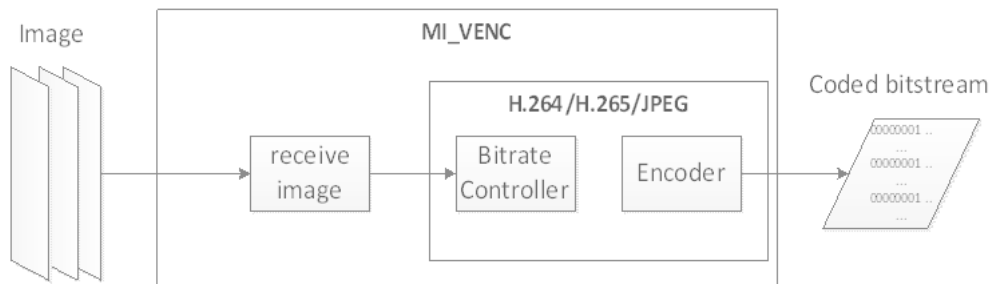


I AM SELLING API

1 Overview

The video encoding module mainly provides functions such as creating and destroying video encoding channels, opening and stopping receiving images, setting and obtaining encoding channel properties, and obtaining and releasing code streams. This module supports multi-channel real-time encoding, each channel is independent, and different encoding protocols and profiles can be set

1.1. Coding process



Picture 1-1

The encoding process includes the process of receiving the input image, encoding the input image, and outputting the encoded stream.

The input image YUV format can be NV12 or YUYV422. The input sources of this module include the following two categories:

1. APP directly injects image data into the encoding module
2. Binding front-end modules such as VPE or DVP send image data to the encoding module

As shown below

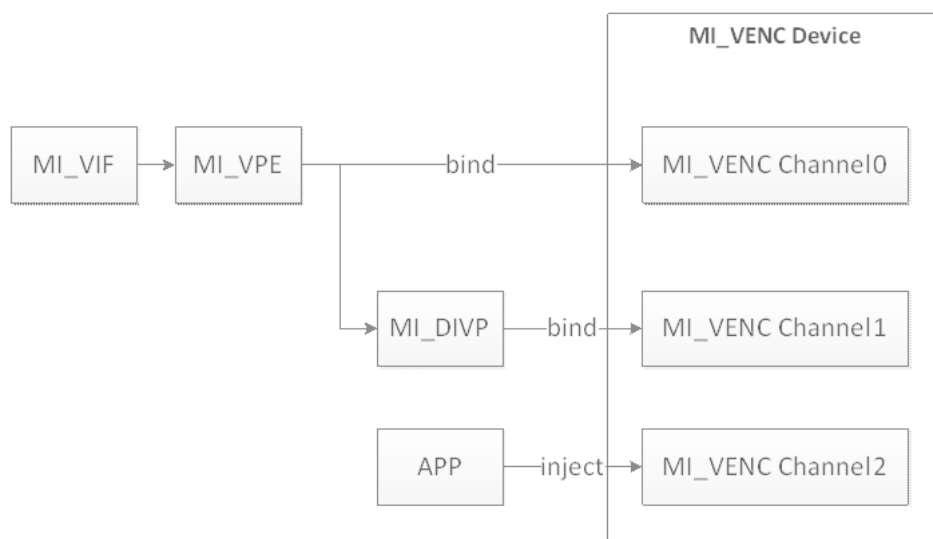


Figure 1-2

1.2. Encoding Channels

As the basic control unit, the encoding channel is completely independent of each other.

The Encoder (HW) completes the function of converting the original image into a code stream, which is specifically controlled by the Rate Controller and the Encoder (SW).

The Rate Controller and Encoder (SW) of each channel will save all the user settings of the current encoding and manage all the resources inside the encoding channel, such as rate control, Buffer requirements and allocation.

The code rate control ensures the balance and stability of the output code rate and the image quality, and the encoder focuses on the implementation of the syntax elements of each Codec.

软件与硬件同时控制，同一个Codec硬件分时复用。以两个编码通道为例，基本框图如下

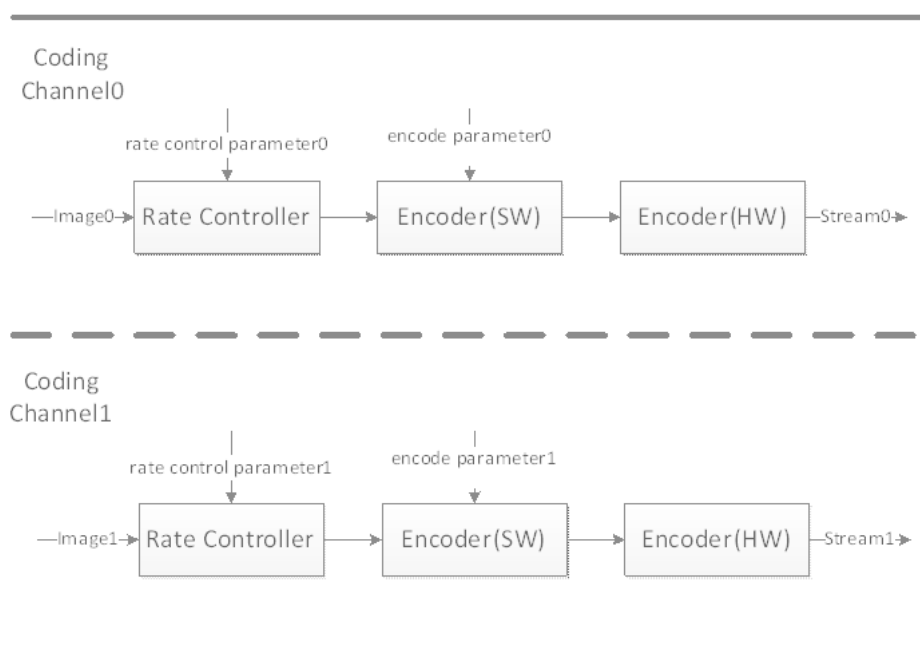


图1-3

1.3. 关键字说明

1.3.1. QP

Quantization 参数：QP值对应量化步长的序号，值越小，量化步长越小，量化的精度就越高，画质也就越好，编码出来的size也越大。

1.3.2. GOP

Group Of Pictures：指的是两个I帧的间隔。视频图像序列由一个或多个图像组GOP组成，GOP之间是独立的。

1.3.3. MB

Macroblock：编码宏块，H.264编码的基本单元。

1.3.4. CU

Coding Unit：编码单元，H.265编码的基本单元。

1.3.5. SPS

Sequence 参数 Set：序列参数集，包含了一个GOP中所有图像的公有信息。

1.3.6. PPS

Picture 参数 Set：图像参数集，包含了一张图像编码所用的参数

1.3.7. SEI

Supplemental Enhancement information:辅助增强信息。

1.3.8. ECS

Entropy-coded segment:JPEG熵编码后的压缩图像条带。

1.3.9. MCU

Minimum code unit:JPEG编码的基本单元。

1.3.10. 码率控制

从信息学角度来看，图像压缩比与质量成反比：压缩比越高，质量越低；压缩比越低，质量越高。以H.264为例，一般图像Qp越低，图像质量越高，码率越高；图像Qp越高，图像质量越低，码率越低。不同编码协议对码率控制算法支持如下表：

表1-1

Chip	Rate Control Algorithm			
	FIXQP	CBR	VBR	AVBR
Pretzel	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	NONE
Macaron	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	NONE
Pudding	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Ispahan	H.264/H.265/JPEG	H.264/H.265/JPEG	H.264/H.265	H.264/H.265
Ikayaki	JPEG	JPEG	None	None

1.3.10.1. FIXQP

FIXQP(Fix Quantization Parameter)固定Qp，在任意时间点，编码图像的所有基本单元Qp 都直接采用用户设定值，H.264/H.265 I帧和P帧的Qp可以分别设置，但有些Chip无法设置 P帧Qp，不同chip的行为如下表：

表1-2

Chip	H.264/H.265	
	Can set I Qp?	Can set P Qp?
Pretzel	Y	Y
Macaron	Y	Y
Pudding	Y	N
Ispahan	Y	N

H.264和H.265的控制流程图如下：

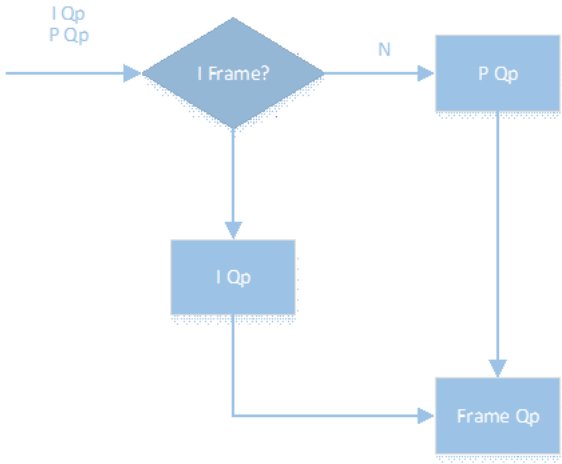


图1-4

JPEG的控制流程图如下：

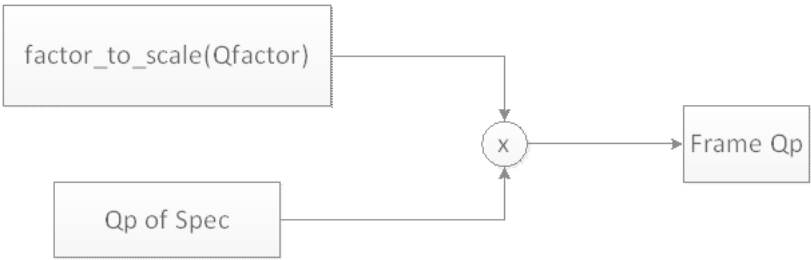


图1-5

采用ITU-t81 K.1节推荐的Qp表，将用户设置的Qfactor按照一定公式转化成比例因子，两者相乘就是最后的Qp。

1.3.10.2. CBR

CBR(Constant Bit Rate)为固定比特率，在码率统计时间内保证编码码率稳定。编码中I帧与P帧因为预测方式的不同会造成编码后Size的明显差异，底层的统计时间以 Gop为基本单位，在Gop之间会实现比特的累积与补偿。

主要步骤如下:

1. 将用户设定的Fps/Gop/Bitrate转换成每个Gop的Bits；

2. I/P区分处理，按照Gop Bits以及分辨率计算Bpp(BitPerPixel)；
3. 通过码率控制模型，将Bpp映射到Frame Qp；
4. HW在Frame Qp基础上通过画面纹理复杂度等信息进一步调整MB/CU Qp；
5. 编码结束后更新码率控制模型，以达到整个序列的不断稳定，同时累积Bit误差，用于后续帧Bit分配的微调；
6. 整个Gop结束后累积整个Gop的Bit误差，用于下一个Gop Bit分配的微调。

控制流程图如下：

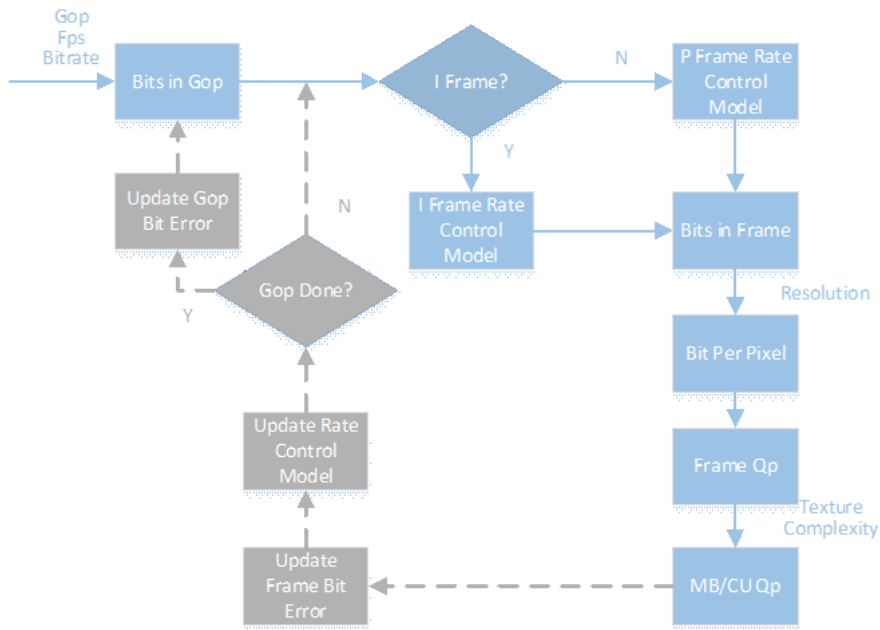


图1-6

1.3.10.3. VBR

VBR(Variable Bit Rate)为可变比特率，允许在码率统计时间内编码码率波动，从而保证编码图像质量平稳。

以H.264为例，用户可设置MaxQp,MinQp,MaxBitRate和ChangePos，具体可以参考MI_VENC_ParamH264Vbr_t [#360-mi_venc_paramh264vbr_t]。MaxQp和MinQp用于控制图像的质量范围; MaxBitRate用于钳位码率统计时间内的最大编码码率;ChangePos用于控制开始调整Qp的码率基准线，是一个相对于最大码率的百分比，即当编码码率大于MaxBitRate*ChangePos时，图像Qp会逐步向MaxQp调整，若图像Qp已到达MaxQp，图像Qp会被钳位到最大值，MaxBitRate的钳位效果失去，编码码率可能会超过MaxBitRate;若编码码率小于MaxBitRate*ChangePos,图像Qp会逐步向MinQp调整，若图像Qp已到达MinQp,编码码率已达到最大值，图像质量最好。

具体流程如下：

1. 将用户设定的Fps/Gop/MaxBitrate/ChangePos以及Resolution转换成序列的初始质量基准设定序列起始Qp
2. HW在Frame Qp基础上通过画面纹理复杂度等信息进一步调整MB/CU Qp
3. 编码完后更新码率控制模型
4. 更新整体序列的质量
5. 根据当前期望的Bit与实际编码的Bit差计算偏差系数Deviation以及 决定是否可以提高或者降低质量
6. 通过偏差系数以及两个质量系数计算当前帧的目标质量
7. 将目标质量映射为Frame Qp，从2循环

控制流程图如下：

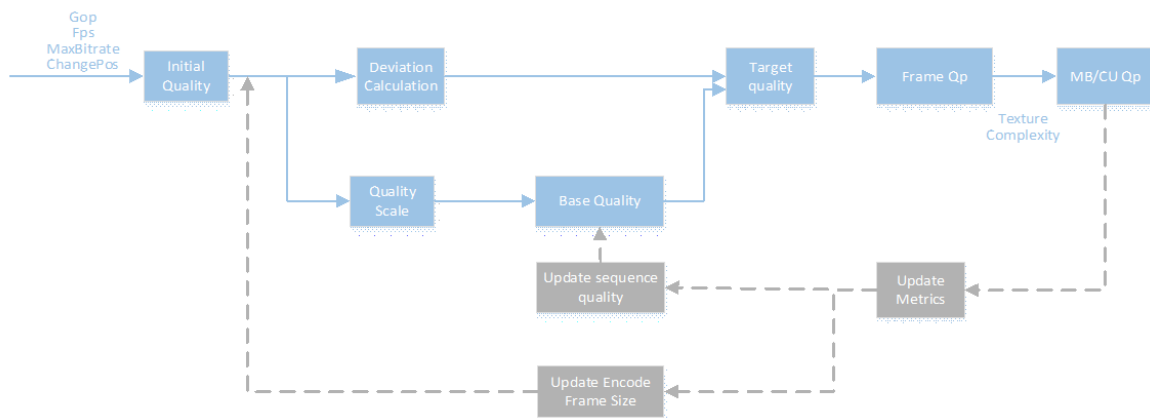


图1-7

1.3.10.4. AVBR

AVBR(Adaptive Variable Bit Rate)自适应可变比特率，允许在码率统计时间内编码码率波动，从而保证编码图像质量平稳。码率控制内部会检测当前场景的运动静止状态，在运动时采用较高码率编码，静止时主动降低码率。以H.264为例，用户可以设置MaxBitRate、ChangePos和MinStillPercent，具体可参考MI_VENC_ParamH264Avbr_t [#361-mi_venc_paramh264avbr_t]。MaxBitRate表示运动场景下的最大码率；MinStillPercent表示静止状态下最小码率相对于调节阈值码率的百分比，MaxBitRate*ChangePos*MinStillPercent表示静止场景下的最小码率，目标码率根据运动程度的不同会在最小码率和最大码率之间调整；MaxQp和MinQp用于控制图像的质量范围，码率控制以Qp钳位为最高优先级，超出MinQp和MaxQp范围码率控制将失效。

1.3.11. QPMAP

QPMAP模式下允许用户自由决定码率控制的策略，支持绝对QpMap和相对QpMap。请参考[MI_VENC_AllocCustomMap](#) [#269-mi_venc_alloccustommap]。

1.3.12. 参考帧结构

H.264/H.265单帧只支持参考1个参考帧，但是整个码流支持多个参考帧缓存。例如：LTR/TSVC3模式，每帧P都只可以参考一个，但是最多会保存2个参考帧供不同P帧参考

参考帧共支持5种模式：NormalP、LTR(VI Ref IDR)、LTR(VI Ref VI)、TSVC-2和TSVC-3。所有参考帧结构由三个参数控制：u32Base、u32Enhance和bEnablePred。具体含义请参考[MI_VENC_ParamRef_t](#) [#345-mi_venc_paramref_t]。u32Enhance设置为0将转换为NormalP参考帧结构，其余结构开启 请参考对应结构图，系统默认为NormalP参考帧结构。以下详细介绍每种参考关系结构图 和参数设定。

1.3.12.1. NORMALP

NormalP为最基本和常见的参考关系，P帧直接参考前一张，其结构图如下图所示：

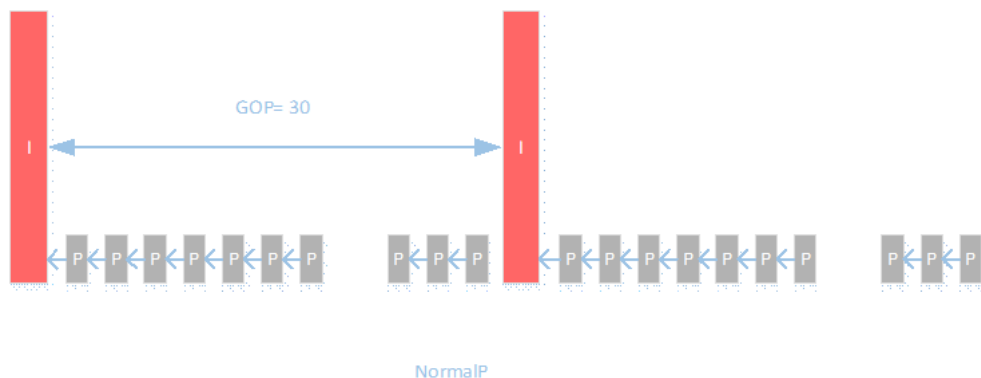


图 1-8

TSVC-3提供三层编码，帧率可以在 $\frac{1}{4}$ ， $\frac{1}{2}$ ， $\frac{3}{4}$ 和全帧率之间变化。其结构图如下：

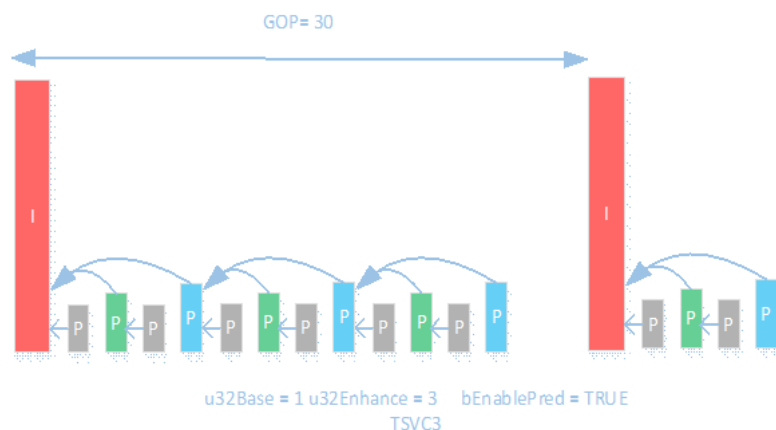


图1-12

注意：开启TSVC-3底层需要额外占用一张Buffer。

1.3.13. 裁剪编码

裁剪编码，即从图像中裁剪出一部分进行编码，用户可以设置裁剪的起始点left/top、width和height，具体功能请参考相关API: `MI_VENC_SetCrop` [#261-mi_venc_setcrop]。示意图如下：

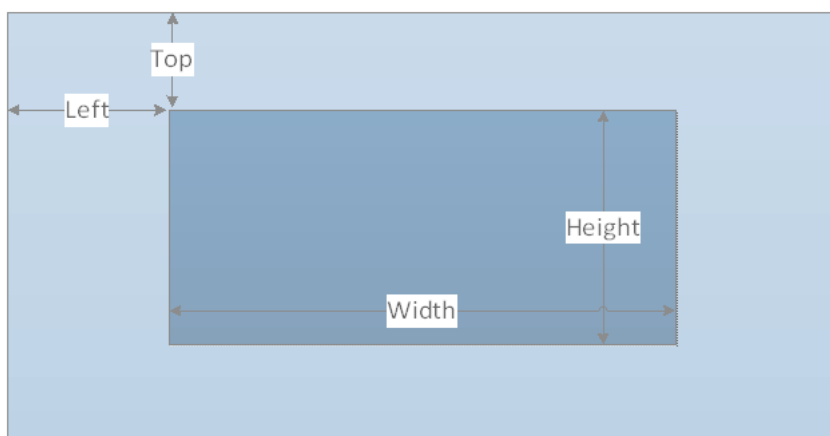


图1-13

1.3.14. ROI

ROI(Region Of Interest)编码，感兴趣区域编码，用户可以通过配置ROI区域，对该区域的图像Qp进行限制，从而实现图像中该区域的Qp与其他图像区域的差异化。系统支持H264 和H265编码设置ROI，且提供8个ROI区域供用户同时使用。

8个ROI区域可以互相叠加，且叠加时的优先级按照0~7的索引号依次提高，也即叠加区域的Qp最终判定只按最高优先级的区域处理。ROI区域可以配置绝对Qp与相对Qp两种模式：

绝对Qp: ROI区域的Qp为用户设定的Qp值

相对Qp: ROI区域的Qp为码率控制产生的Qp与用户设定的Qp偏移值的和

以下示例为编码图像采用FixQp模式，设置图像Qp为30，即图像所有宏块Qp值为30。ROI区域0设置为绝对Qp模式，Qp值为20，索引为0；ROI区域1设置为相对Qp模式，Qp为-15，索引为1。因为ROI区域0的index小于ROI区域1的index，所以在发生重叠的图像区域按高优先级的ROI区域1的Qp设置。除了重叠部分的ROI区域0的Qp值为20，区域1的Qp值为 $30-15=15$ 。

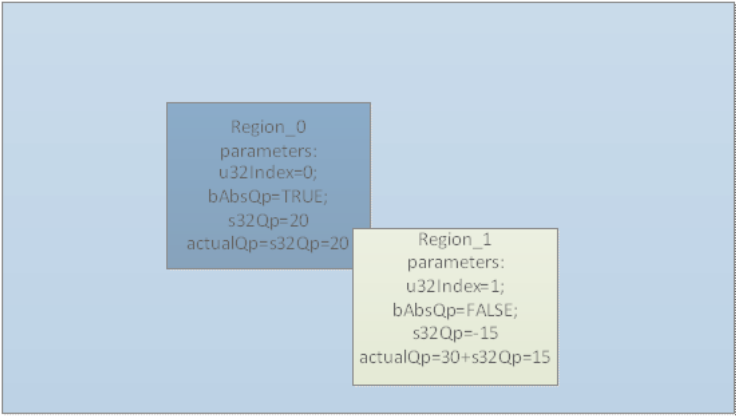


图1-14

1.3.15. 非ROI区域的低帧率编码

为了减低传输码率，对于非ROI区域，可采用低帧率编码，即开启ROI后ROI区域正常编码，而非ROI区域则可以通过设定源和目标的比例关系降低帧率，根据该比例关系，一个GOP 里面一些帧的非ROI区域会直接用上一帧对应区域的数据，用户可以根据实际情况设置 非ROI区域相对帧率，注意只有ROI开启才生效。请参考API：[MI_VENC_SetRoiBgFrameRate](#) [#227-mi_venc_setroibgframerate]，其中s32SrcFrmRate与s32DstFrmRate只表示比例关系，与实际帧率无关

1.3.16. Bind Type

一般来说前级与venc的bind type是E_MI_SYS_BIND_TYPE_FRAME_BASE，即前级完整做完一张 Frame buffer，再输出给venc，这种模式至少需要分配3张Frame buffer。不过若前级是vpe，还有另外两种省内存的bind type：E_MI_SYS_BIND_TYPE_HW_RING和E_MI_SYS_BIND_TYPE_REALTIME。

E_MI_SYS_BIND_TYPE_HW_RING是指vpe与venc只在同一张Frame buffer上边读边写，另外 Ispahan 还支持在半张Frame buffer上读写，可参考[MI_VENC_SetInputSourceConfig](#) [#274-mi_venc_setinputsourceconfig]；

而E_MI_SYS_BIND_TYPE_REALTIME是指vpe与venc之间无需额外分配Dram，硬件直连。

不同Chip和编码协议对bind type的支持如下：

表1-3

Chip	Codec	Bind Type		
		FRAME_BASE	HW_RING	REALTIME
Pretzel	H.264/H.265	Y	N	N
	JPEG	Y	N	N
Macaron	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Pudding	H.264/H.265	Y	N	N
	JPEG	Y	Y	Y
Ispahan	H.264/H.265	Y	Y	N
	JPEG	Y	N	Y
Ikayaki	JPEG	Y	Y	Y

1.3.17. 编码码流输出Buffer配置

不同Chip由于SW架构不同，支持不同的方式，底层的默认分配方式如下表：

表1-4

Chip	H.264	H.265	JPEG
Pretzel	WidthxHeight/2	WidthxHeight/2	WidthxHeight
Macaron	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2
Pudding	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2
Ispahan	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2
Ikayaki	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2	Ring pool:WidthxHeight/2

以上为各Chip的默认输出Buffer配置，用户可以通过MI_VENC_CreateChn [#25-mi_venc_createchn]去指定输出Buffer配置。以H.264为例，可以通过设置stVeAttr.stAttrH264e.u32BufSize去指定输出Buffer配置，若为0，则采用上表默认配置。原先Pretzel采用的是帧模式，即每编码一张，都要重新申请一块 特定大小的Buffer，且通常实际编码使用的size会小于申请的size，会造成内存碎片；而采取 Ring pool模式，每个Channle会单独分配一个output pool，每次编码都从该pool里获取空闲的 最大Buffer，有效提高了Buffer的使用率，减少了内存碎片。

注意：当JPEG使用realtime mode时，Pudding/Ikayaki内部会开两个output pool，以便支持满帧输出，不然只有前端mi_scl输出帧率的一半。若不需要满帧输出，可以在insmod mi_venc.ko后面加上max_jpe_task=1，强制开一个output pool。

2. API参考

2.1. 功能模块API

API 名	功能
MI_VENC_GetChnDevid [#22-mi_venc_getchndevid]	获取编码通道的设备id
MI_VENC_SetModParam [#23-mi_venc_setmodparam]	设置编码相关的模块参数
MI_VENC_GetModParam [#24-mi_venc_getmodparam]	获取编码相关的模块参数
MI_VENC_CreateChn [#25-mi_venc_createchn]	创建编码通道
MI_VENC_DestroyChn [#26-mi_venc_destroychn]	销毁编码通道
MI_VENC_ResetChn [#27-mi_venc_resetchn]	复位编码通道
MI_VENC_StartRecvPic [#28-mi_venc_startrecvpic]	开启编码通道接收输入图像
MI_VENC_StartRecvPicEx [#29-mi_venc_startrecvpicex]	开启编码通道接收输入图像，超出指定的帧数后自动停止接收图像
MI_VENC_StopRecvPic [#210-mi_venc_stoprecvpic]	停止编码通道接收输入图像
MI_VENC_Query [#211-mi_venc_query]	查询编码通道状态
MI_VENC_SetChnAttr [#212-mi_venc_setchnattr]	设置编码通道的编码属性
MI_VENC_GetChnAttr [#213-mi_venc_getchnattr]	获取编码通道的编码属性

MI_VENC_GetStream [#214-mi_venc_getstream]	获取编码码流
MI_VENC_ReleaseStream [#215-mi_venc_releasestream]	释放码流缓存
MI_VENC_InsertUserData [#216-mi_venc_insertuserdata]	插入用户数据
MI_VENC_SetMaxStreamCnt [#217-mi_venc_setmaxstreamcnt]	设置最大码流缓存帧数
MI_VENC_GetMaxStreamCnt [#218-mi_venc_getmaxstreamcnt]	获取最大码流缓存帧数
MI_VENC_RequestIdr [#219-mi_venc_requestidr]	请求IDR帧
MI_VENC_EnableIdr [#220-mi_venc_enableidr]	使能IDR帧
MI_VENC_SetH264IdrPicId [#221-mi_venc_seth264idrpicid]	设置IDR帧的idr_pic_id
MI_VENC_GetH264IdrPicId [#222-mi_venc_geth264idrpicid]	获取IDR帧的idr_pic_id
MI_VENC_GetFd [#223-mi_venc_getfd]	获取编码通道对应的设备文件句柄
MI_VENC_CloseFd [#224-mi_venc_closefd]	关闭编码通道对应的设备文件句柄
MI_VENC_SetRoiCfg [#225-mi_venc_setroicfg]	设置编码通道的感兴趣区域编码配置
MI_VENC_GetRoiCfg [#226-mi_venc_getroicfg]	获取编码通道的感兴趣区域编码配置
MI_VENC_SetRoiBgFrameRate [#227-mi_venc_setroibgframerate]	设置编码通道非感兴趣区域的帧率配置
MI_VENC_GetRoiBgFrameRate [#228-mi_venc_getroibgframerate]	获取编码通道非感兴趣区域的帧率配置
MI_VENC_SetH264SliceSplit [#229-mi_venc_seth264slicesplit]	设置H.264编码的slice分割配置
MI_VENC_GetH264SliceSplit [#230-mi_venc_geth264slicesplit]	获取H.264编码的slice分割配置
MI_VENC_SetH264InterPred [#231-mi_venc_seth264interpred]	设置H.264编码的帧间预测配置
MI_VENC_GetH264InterPred [#232-mi_venc_geth264interpred]	获取H.264编码的帧间预测配置
MI_VENC_SetH264IntraPred [#233-mi_venc_seth264intrapred]	设置H.264编码的帧内预测配置
MI_VENC_GetH264IntraPred [#234-mi_venc_geth264intrapred]	获取H.264编码的帧内预测配置
MI_VENC_SetH264Trans [#235-mi_venc_seth264trans]	设置H.264编码的变换、量化配置
MI_VENC_GetH264Trans [#236-mi_venc_geth264trans]	获取H.264编码的变换、量化配置
MI_VENC_SetH264Entropy [#237-mi_venc_seth264entropy]	设置H.264编码的熵编码配置
MI_VENC_GetH264Entropy [#238-mi_venc_geth264entropy]	获取H.264编码的熵编码配置
MI_VENC_SetH265InterPred [#239-mi_venc_seth265interpred]	设置H.265编码的帧间预测配置
MI_VENC_GetH265InterPred [#240-mi_venc_geth265interpred]	获取H.265编码的帧间预测配置
MI_VENC_SetH265IntraPred [#241-mi_venc_seth265intrapred]	设置H.265编码的帧内预测配置
MI_VENC_GetH265IntraPred [#242-mi_venc_geth265intrapred]	获取H.265编码的帧内预测配置
MI_VENC_SetH265Trans [#243-mi_venc_seth265trans]	设置H.265编码的变换、量化配置
MI_VENC_GetH265Trans [#244-mi_venc_geth265trans]	获取H.265编码的变换、量化配置

MI_VENC_SetH264Dbk [#245-mi_venc_seth264dbk]	设置H.264编码的deblocking配置
MI_VENC_GetH264Dbk [#246-mi_venc_geth264dbk]	获取H.264编码的deblocking配置
MI_VENC_SetH265Dbk [#247-mi_venc_seth265dbk]	设置H.265编码的deblocking配置
MI_VENC_GetH265Dbk [#248-mi_venc_geth265dbk]	获取H.265编码的deblocking配
MI_VENC_SetH264Vui [#249-mi_venc_seth264vui]	设置H.264编码的VUI配置
MI_VENC_GetH264Vui [#250-mi_venc_geth264vui]	获取H.264编码的VUI配置
MI_VENC_SetH265Vui [#251-mi_venc_seth265vui]	设置H.265编码的VUI配置
MI_VENC_GetH265Vui [#252-mi_venc_geth265vui]	获取H.265编码的VUI配置
MI_VENC_SetH265SliceSplit [#253-mi_venc_seth265slicesplit]	设置H.265编码的slice分割配置
MI_VENC_GetH265SliceSplit [#254-mi_venc_geth265slicesplit]	获取H.265编码的slice分割配置
MI_VENC_SetJpegParam [#255-mi_venc_setjpegparam]]	设置JPEG编码的参数集合
MI_VENC_GetJpegParam [#256-mi_venc_getjpegparam]	获取JPEG编码的参数集合
MI_VENC_SetRcParam [#257-mi_venc_setrcparam]	设置通道码率控制高级参数
MI_VENC_GetRcParam [#258-mi_venc_getrcparam]	获取通道码率控制高级参数
MI_VENC_SetRefParam [#259-mi_venc_setrefparam]	设置H.264/H.265编码通道高级跳帧参考参数
MI_VENC_GetRefParam [#260-mi_venc_getrefparam]	获取H.264/H.265编码通道高级跳帧参考参数
MI_VENC_SetCrop [#261-mi_venc_setcrop]	设置VENC的裁剪属性
MI_VENC_GetCrop [#262-mi_venc_getcrop]	获取VENC的裁剪属性
MI_VENC_SetFrameLostStrategy [#263-mi_venc_setframeLoststrategy]	设置瞬时码率超出阈值时丢帧策略的配置
MI_VENC_GetFrameLostStrategy [#264-mi_venc_getframeloststrategy]	获取瞬时码率超出阈值时丢帧策略的配置
MI_VENC_SetSuperFrameCfg [#265-mi_venc_setsuperframecfg]	设置超大帧处理配置
MI_VENC_GetSuperFrameCfg [#266-mi_venc_getsuperframecfg]	获取超大帧处理配置
MI_VENC_SetRcPriority [#267-mi_venc_setrcpriority]	设置码率控制的优先级类型
MI_VENC_GetRcPriority [#268-mi_venc_getrcpriority]	获取码率控制的优先级类型
MI_VENC_AllocCustomMap [#269-mi_venc_alloccustommap]	分配智能编码所用的Custom Map的内存
MI_VENC_ApplyCustomMap [#270-mi_venc_applycustommap]	应用已经配置的Custom Map
MI_VENC_GetLastHistoStaticInfo [#271-mi_venc_getlasthistostaticinfo]	获取最新的图像帧编码后的相关输出信
MI_VENC_ReleaseHistoStaticInfo [#272-mi_venc_releasehistostaticinfo]	释放保存的最新的图像帧编码后的相关输出信息占用的内存
MI_VENC_SetAdvCustRcAttr [#273-mi_venc_setadvcustrcattr]	设置自定义的高级码控相关属性配置
MI_VENC_SetInputSourceConfig [#274-mi_venc_setinputsourceconfig]	设置H.264/H.265的输入源配置信息
MI_VENC_SetSmartDetInfo [#275-mi_venc_setsmartdetinfo]	设置智能侦测算法相关的统计信息

MI_VENC_SetIntraRefresh [#276-mi_venc_setintrarefresh]	设置H.264/H.265的P帧刷新Islice
MI_VENC_GetIntraRefresh [#277-mi_venc_getintrarefresh]	获取H.264/H.265的Intra Refresh参数
MI_VENC_InitDev [#278-mi_venc_initdev]	初始化venc设备
MI_VENC_DeInitDev [#279-mi_venc_deinitdev]	反初始化venc设备
MI_VENC_DupChn [#280-mi_venc_dupchn]	同步VENC通道状态。

2.2. MI_VENC_GetChnDevid

- 功能

获取编码通道的设备id。

- 语法

```
MI_S32 MI_VENC_GetChnDevid(MI_VENC_CHN VeChn, MI_U32 *pu32Devid);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pu32Devid	编码设备id指针。	输出

- 返回值

- 0: 成功
- 非0: 失败, 参照[错误码](#) [#jump]

- 依赖

- 头文件：mi_common.h、mi_venc.h
- 库文件: libmi.a

- 芯片差异

不同Chip下不同编码协议对应的设备id是不同的，如下表所示：

Chip	H.264	H.265	JPEG
Pretzel	0	0	1
Macaron	0	0	1
Pudding	0	0	1
Ispahan	0	0	1

- 注意

- 此接口在通道创建后调用，如果通道未创建，则返回失败
- 由于不同编码协议对应的设备id有可能不同，因此如果接下来调用的接口参数包含设备id，需要先调用该接口获取当前channel的对应设备id

- 举例

```
MI_S32 BindVpeToJpeg()
{
    MI_S32 s32Ret;
    MI_SYS_ChnPort_t stVencInputPort;
    MI_SYS_ChnPort_t stVpeOutputPort;
    MI_U32 u32DevId = 0;
    MI_U32 u32ChnId = 0;

    /*call MI_VPE_CreateChannel to create vpe channel*/
    /*call MI_VENC_CreateChn to create jpeg channel*/

    memset(&stVencInputPort, 0, sizeof(MI_SYS_ChnPort_t));
    s32Ret = MI_VENC_GetChnDevid(u32ChnId, &u32DevId);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetChnDevid err 0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stVencInputPort.eModId = E_MI_MODULE_ID_VENC;
    stVencInputPort.u32DevId = u32DevId;
    stVencInputPort.u32ChnId = u32ChnId;
    stVencInputPort.u32PortId = 0;

    stVpeOutputPort.eModId = E_MI_MODULE_ID_VPE;
    stVpeOutputPort.u32DevId = 0;
    stVpeOutputPort.u32ChnId = 0;
    stVpeOutputPort.u32PortId = 0;

    s32Ret = MI_SYS_BindChnPort2(&stVpeOutputPort, &stVencInputPort, 30, 30,
    E_MI_SYS_BIND_TYPE_FRAME_BASE, 0);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_SYS_BindChnPort2 err 0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return MI_SUCCESS;
}
```

2.3. MI_VENC_SetModParam

- 功能
设置编码相关的模块参数。
- 语法

```
MI_S32 MI_VENC_SetModParam(MI_VENC_ModParam_t *pstModParam)
```

- 形参

参数名称	描述	输入/输出
pstModParam	编码模块参数指针	输入

- 返回值
 - 0: 成功
 - 非0: 失败，参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h、mi_venc.h

- 库文件文件：libmi.a
- 注意
 - 此接口在通道创建前调用，如果通道已经创建，则返回失败
 - 可以设置mi_venc.ko、mi_h264e.ko、mi_h265e.ko、mi_jpge.ko模块的参数。
 - 暂不支持。

2.4. MI_VENC_GetModParam

- 功能
获取编码相关的模块参数。
- 语法

```
MI_S32 MI_VENC_GetModParam(MI_VENC_ModParam_t *pstModParam)
```

- 参数

参数名称	描述	输入/输出
pstModParam	编码模块参数指针。	输出

- 返回值
 - 0: 成功
 - 非0: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h、mi_venc.h
 - 库文件: libmi.a
- 注意
可以获取mi_venc.ko、mi_h264e.ko、mi_h265e.ko、mi_jpge.ko模块的参数。
- 暂不支持

2.5. MI_VENC_CreateChn

- 功能
创建编码通道。
- 语法

```
MI_S32 MI_VENC_CreateChn(MI_VENC_CHN Vehn, MI_VENC_ChnAttr_t *pstAttr);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstAttr	编码通道属性指针	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h 、 mi_comm_rc.h 、 mi_venc.h
 - 库文件: libmi.a
- 芯片差异

不同Chip支持的编码规格如下表所示：

Chip	H.264		H.265		JPEG
	Baseline Profile	Main Profile	High Profile	Main Profile	Baseline Profile
Pretzel	Y	Y	N	Y	Y
Macaron	Y	Y	N	Y	Y
Pudding	Y	Y	Y	Y	Y
Ispahan	Y	Y	Y	Y	Y
Ikayaki	N	N	N	N	Y

不同Chip支持的编码通道宽高差异如下表所示：

Chip	Resolution Spec	H.264	H.265	JPEG
Pretzel	MIN_WIDTHxMIN_HEIGHT	192x128	192x128	16x16
	MAX_WIDTHxMAX_HEIGHT	3840x2176	3840x2176	3840x2176
	MIN_ALIGN	8x2	8x2	8x2
Macaron	MIN_WIDTHxMIN_HEIGHT	192x128	192x128	16x16
	MAX_WIDTHxMAX_HEIGHT	2688x1920	2688x1920	2688x2688
	MIN_ALIGN	8x2	8x2	8x2
Pudding	MIN_WIDTHxMIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTHxMAX_HEIGHT	3840x2176	3840x2176	3840x2176
	MIN_ALIGN	8x2	8x2	8x2

Ispahan	MIN_WIDTHxMIN_HEIGHT	256x128	256x128	16x16
	MAX_WIDTHxMAX_HEIGHT	2592x1952	2592x1952	2688x2688
	MIN_ALIGN	8x2	8x2	8x2
Ikayaki	MIN_WIDTHxMIN_HEIGHT	None	None	16x16
	MAX_WIDTHxMAX_HEIGHT	None	None	1920x1088
	MIN_ALIGN	None	None	8x2

- 注意
 - 编码通道属性由两部分组成，编码器属性和码率控制器属性。
 - 编码器属性首先需要选择编码协议，然后分别对各种协议对应的属性进行赋值。
 - 编码器属性最大宽高，通道宽高必须满足如下约束：
 $MaxPicWidth \in [MIN_WIDTH, MAX_WIDTH]$
 $MaxPicHeight \in [MIN_HEIGHT, MAX_HEIGHT]$
 $PicWidth \in [MIN_WIDTH, MaxPicwidth]$
 $PicHeight \in [MIN_HEIGHT, MaxPicHeight]$
 - 最大宽高，通道宽高必须是MIN_ALIGN的整数倍。
 - 其中MIN_WIDTH、MAX_WIDTH、MIN_HEIGHT、MAX_HEIGHT、MIN_ALIGN分别表示编码通道 支持的最小宽度，最大宽度，最小高度，最大高度，最小对齐单元（像素）。
 - 当输入图像大小不大于通道编码图像最大宽高时，才能启动编码通道进行编码。
 - 推荐的编码宽高为：3840x2160（4k*2k）、1920x1080(1080P)、1280x720（720P）、960x540、640x360、704x576、704x480、352x288、352x240。
 - 编码器属性stVeAttr中除通道宽高（u32PicWidth/u32PicHeight）和profile以外 都是静态属性，一旦创建编码通道成功，静态属性不支持被修改，除非该通道被销毁，重新创建。设置时需要注意的事项请参考[MI_VENC_SetChnAttr](#) [#212-mi_venc_setchnattr]接口说明。
 - 码率控制器属性首先需要配置RC模式，JPEG抓拍通道不需要配置码率控制器属性，其他协议类型通道（H.264/H.265/MJPEG）都必须配置。码率控制器属性RC模式必须与 编码器属性协议类型匹配。
 - 三种编码协议（H.264/H.265/MJPEG），码率控制均支持三种模式：CBR、VBR和FIXQP。并且对于不同协议，相同RC模式的属性变量基本一致。
 - u32SrcFrmRateNum指编码模块帧率的分子部分，u32SrcFrmRateDen指编码模块帧率的 分母部分。u32SrcFrmRateNum/u32SrcFrmRateDen应该设置为产生TimeRef的实际帧率，RC需要根据u32SrcFrmRateNum/u32SrcFrmRateDen统计实际的帧率以及进行码率控制。如果编码图像的源是VI时，则u32SrcFrmRateNum/u32SrcFrmRateDen设置为VI的实际输出帧率，因为TimeRef是在VI输出的时候产生。假如VI的实际输出帧率为30，则u32SrcFrmRateNum设为30，u32SrcFrmRateDen设为1。

CBR除了上述的属性之外，还需要设置平均比特率和波动等级。平均比特率的单位是bps。平均比特率的设置与编码通道宽高以及图像帧率都有关系。典型的平均比特率的设置如下表所示。注意，下表中的平均比特率的设置是在通道编码 帧率为满帧率（30fps）时的设置。当用户设置编码输出帧率不为满帧率时，可以对 下表中的码率按用户设置帧率与满帧率（30fps）的比例进行换算。

Image W/H/bitrate level	D1 (720x576)	720p (1280x720)	1080p (1920x1080)
Low bit rate	< 400Kbps	< 800Kbps	< 2000Kbps
Medium code rate	400 ~ 1000Kbps	800kbps ~ 4000Kbps	2000 ~ 8000Kbps
High code rate	> 1000Kbps	> 4000Kbps	> 8000Kbps

- 波动等级设置分为5档，波动等级越大，系统允许码率的波动范围更大。如果波动等级 设置高，对于一些图像复杂，变化剧烈的场景，图像质量可能会更平稳，适用于网络 带宽富裕的场景；如果波动等级设置低，编码的码率会比较平稳，对于一些图像复杂，变化剧烈的场景，图像质量可能不如高波动等级，适用于带宽不富裕的场景，保留，暂时没有使用。
- VBR除了上述属性之外，还需要设置MaxBitRate，MaxQp，MinQp。编码通道在码率统计时间内允许的最大码率。MaxQp: 图像允许的最大QP。
- MinQp: 图像允许的最小QP。FIXQP除了上述属性之外，还需要设置IQp，PQp。IQp: I帧时，图像固定使用的QP值。PQp: P帧时，图像固定使用的QP值。在设置I帧QP，P帧QP时，可以根据当前的带宽限制对I帧QP和P帧QP同时进行向上或是向下调整。为了减少呼吸效应，推荐I帧QP始终比P帧的QP小2~3。

不同Chip支持的最大VENC_MAX_CHN_NUM 规格如下表所示：

芯片	最大值
Macaron	16
Pudding	16
Ispahan	16
lkayaki	16

- 举例

```
MI_S32 StartVenc()
{
    MI_S32 s32Ret;
    MI_VENC_CHN VeChn = 0;
    MI_VENC_ChnAttr_t stAttr;

    /*set h264 chnnel video encode attribute*/
    stAttr.stVeAttr.eType = E_MI_VENC_MODTYPE_H264E;
    stAttr.stVeAttr.stAttrH264e.u32PicWidth = u32PicWidth;
    stAttr.stVeAttr.stAttrH264e.u32PicHeight = u32PicHeigh;
    stAttr.stVeAttr.stAttrH264e.u32MaxPicWidth = u32MaxPicWidth;
    stAttr.stVeAttr.stAttrH264e.u32MaxPicHeight = u32MaxPicHeigh;
    stAttr.stVeAttr.stAttrH264e.u32Profile = 2;

    /*set h264 chnnel rate control attribute*/
    stAttr.stRcAttr.enRcMode = E_MI_VENC_RC_MODE_H264CBR;
    stAttr.stRcAttr.stAttrH264Cbr.u32BitRate = 10*1024*1024;
    stAttr.stRcAttr.stAttrH264Cbr.fr32DstFrmRate = 30;
    stAttr.stRcAttr.stAttrH264Cbr.u32SrcFrmRate = 30;
    stAttr.stRcAttr.stAttrH264Cbr.u32Gop = 30;
    stAttr.stRcAttr.stAttrH264Cbr.u32FluctuateLevel = 1;
    stAttr.stRcAttr.stAttrH264Cbr.u32StatTime = 1;

    s32Ret = MI_VENC_CreateChn(VeChn, &stAttr);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_CreateChn err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    s32Ret = MI_VENC_StartRecvPic(VeChn);
    if (s32Ret != MI_SUCCESS)
    {
        printf("MI_VENC_StartRecvPic err0x%x\n",s32Ret);
        return E_MI_ERR_FAILED;
    }

    return MI_SUCCESS;
}
```

2.6. MI_VENC_DestroyChn

- 功能
销毁编码通道。
- 语法

```
MI_S32 MI_VENC_DestroyChn(MI_VENC_CHN VeChn);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 销毁并不存在的通道，返回失败。
 - 销毁前必须停止接收图像，否则返回失败。
 - 如果开启了OSD，则编码通道销毁前要调用接口MI_RGN_DetachFrmChn将OSD区域从当前通道撤出。
- 举例

```
MI_S32 StopVenc()
{
    MI_S32 s32Ret;
    MI_VENC_CHN VeChn = 0;

    s32Ret = MI_VENC_StopRecvPic(VeChn);
    if (s32Ret != MI_SUCCESS)
    {
        printf("MI_VENC_StopRecvPic err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    s32Ret = MI_VENC_DestroyChn(VeChn);
    if (s32Ret != MI_SUCCESS)
    {
        printf("MI_VENC_DestroyChn err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return MI_SUCCESS;
}
```

2.7. MI_VENC_ResetChn

- 功能
复位通道，清除调用该接口之前缓存的图像和码流。
- 语法

MI_S32 MI_VENC_ResetChn(MI_VENC_CHN VeChn);

• 参数

参数名称	描述	输入/输出
VeChn	通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

• 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

• 依赖

- 头文件：mi_common.h、mi_venc.h
- 库文件：libmi.a

• 注意

- Reset并不存在的通道，返回失败MI_ERR_VENC_UNEXIST。
- 如果一个通道没有停止接收图像而reset通道，则返回失败。
- 暂不支持

2.8. MI_VENC_StartRecvPic

• 功能

开启编码通道接收输入图像。

• 语法

MI_S32 MI_VENC_StartRecvPic(MI_VENC_CHN VeChn);

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

• 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

• 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

• 注意

- 如果通道未创建，则返回失败MI_ERR_VENC_UNEXIST。
- 此接口不判断当前是否已经开启接收，即允许重复开启不返回错误。
- 只有开启接收之后编码器才开始接收图像编码。

• 举例

请参见[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]的举例。

2.9. MI_VENC_StartRecvPicEx

- 功能
开启编码通道接收输入图像，超出指定的帧数后自动停止接收图像。
- 语法

```
MI_S32 MI_VENC_StartRecvPicEx(MI_VENC_CHN VeChn,MI_VENC_RecvPicParam_t*pstRecvParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRecvParam	接收图像参数结构体指针，用于指定需要接收的图像帧数。	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回失败MI_ERR_VENC_UNEXIST。
 - 如果通道已经调用了[MI_VENC_StartRecvPic](#) [#28-mi_venc_startrecvpic] 开始接收图像而没有停止接收图像，或者上次调用[MI_VENC_StartRecvPicEx](#) [#29-mi_venc_startrecvpicex]后还没有接收到足够的图像，再一次调用此接口返回 操作不允许。
 - 该接口用于连续接收N帧并编码的场景，当N=0时，该接口等同于[MI_VENC_StartRecvPic](#) [#28-mi_venc_startrecvpic]。
 - 如果通道已经调用了[MI_VENC_StartRecvPic](#) [#28-mi_venc_startrecvpic] 开始接收图像，停止接收图像，再次调用[MI_VENC_StartRecvPicEx](#) [#29-mi_venc_startrecvpicex]启动编码时，建议用户调用MI_VENC_ResetChn清除编码模块 在调用该接口之前缓存的图像和码流。
 - 如果创建jpeg通道抓拍，建议用户调用[MI_VENC_StartRecvPicEx](#) [#29-mi_venc_startrecvpicex]，可以设定抓拍通道接收整数张图像后自动停止接收。
- 举例

```
MI_S32 JpegSnapProcess()
{
    MI_S32 s32Ret;
    MI_VENC_CHN VeChn=0;
    MI_VENC_ChnAttr_t stAttr;
    MI_VENC_RecvPicParam_t stRecvParam;

    /*set jpeg channel video encode attribute*/
    stAttr.stVeAttr.eType = E_MI_VENC_MODTYPE_JPEGE;
    stAttr.stVeAttr.stAttrJpeg.u32PicWidth = u32PicWidth;
    stAttr.stVeAttr.stAttrJpeg.u32PicHeight = u32PicHeigh;
    stAttr.stVeAttr.stAttrJpeg.u32MaxPicWidth = u32MaxPicWidth;
    stAttr.stVeAttr.stAttrJpeg.u32MaxPicHeight = u32MaxPicHeigh;
```

```
//...omit other video encode assignments here.

//create jpeg channel
s32Ret = MI_VENC_CreateChn(VeChn, &stAttr);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VENC_CreateChn err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}
//start snapping
stRecvParam.s32RecvPicNum = 2;
s32Ret = MI_VENC_StartRecvPicEx(VeChn, &stRecvParam);
if (s32Ret != MI_SUCCESS)
{
    printf("MI_VENC_StartRecvPicEx err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}

//...wait until all pictures have been encoded.

s32Ret = MI_VENC_StopRecvPic(VeChn);
if (s32Ret != MI_SUCCESS)
{
    printf("MI_VENC_StopRecvPic err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}

//destroy jpeg channel
s32Ret = MI_VENC_DestroyChn(VeChn);
if (s32Ret != MI_SUCCESS)
{
    printf("MI_VENC_DestroyChn err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}

return MI_SUCCESS;
}
```

2.10. MI_VENC_StopRecvPic

- 功能
停止编码通道接收输入图像。
- 语法

```
MI_S32 MI_VENC_StopRecvPic(MI_VENC_CHN VeChn);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意

- 如果通道未创建，则返回失败。
- 此接口并不判断当前是否停止接收，即允许重复停止接收不返回错误。
- 此接口用于编码通道停止接收图像来编码，在编码通道销毁或复位前必须停止接收图像。
- 调用此接口仅停止接收原始数据编码，码流buffer并不会被清除。
- 调用[MI_VENC_StartRecvPic](#) [#28-mi_venc_startrecvpic] 和[MI_VENC_StartRecvPicEx](#) [#29-mi_venc_startrecvpicex] 接口开始接收图像，都可以调用 该接口来停止接收。
- 举例
请参见[MI_VENC_DestroyChn](#) [#26-mi_venc_destroychn]的举例。

2.11. MI_VENC_Query

- 功能
查询编码通道状态。
- 语法

```
MI_S32 MI_VENC_Query(MI_VENC_CHN VeChn, MI_VENC_ChnStat_t *pstStat);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstStat	编码通道的状态指针	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回失败。
 - 此接口用于查询此函数调用时刻的编码器状态，pstStat包含四个主要的信息：
 - a. u32LeftPics 表示待编码的帧个数。在复位通道前，可以通过查询是否还有图像待 编码来决定复位时机，防止复位时将可能需要编码的帧清理出去。
 - b. u32LeftStreamBytes 表示码流buffer中剩余的byte数目。在复位通道前，可以通过 查询是否还有码流没有被处理来决定复位时机，防止复位时将可能需要的码流清理出去。
 - c. u32LeftStreamFrames 表示码流buffer中剩余的帧数目。在复位通道前，可以通过 查询是否还有图像的码流没有被取走来决定复位时机，防止复位时将可能需要的码流 清理出去。
 - d. u32CurPacks 表示当前帧的码流包个数。在调用[MI_VENC_GetStream](#) [#214-mi_venc_getstream]之前应确保 u32CurPacks大于0，否则会返回无buffer可取的error。
 - 当通过packet获取时，当前的帧可能不是一个完整的帧(作为一部分)；当通过frame获取时（或者如果没有帧数据时），表示当前完整帧的数量。当用户想要以帧为单位获取码流，需要查询一个完整帧的包的数量。在这种情况下，查询操作通常可以在选择成功之后执行。此时，u32CurPacks是当前完整帧的包数。

- 在编码通道状态结构体中，u32LeftRecvPics表示调用MI_VENC_StartRecvPicEx [#29-mi_venc_startrecvpicex]接口后剩余等待接收的帧数目。
 - 在编码通道状态结构体中，u32LeftEncPics表示调用MI_VENC_StartRecvPicEx [#29-mi_venc_startrecvpicex]接口后剩余等待编码的帧数目。
 - 如果没有调用MI_VENC_StartRecvPicEx [#29-mi_venc_startrecvpicex]，u32LeftRecvPics和u32LeftEncPics数目始终为0。
- 举例
请参见MI_VENC_GetStream [#214-mi_venc_getstream]的举例。

2.12. MI_VENC_SetChnAttr

- 功能
设置编码通道动态属性，包含编码宽高，profile，以及码控参数。
- 语法

```
MI_S32 MI_VENC_SetChnAttr(MI_VENC_CHN VeChn, MI_VENC_ChnAttr_t *pstAttr);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstAttr	编码通道属性指针	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照错误码 [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 不能动态设置编码通道最大宽、最大高等属性。
 - 编码通道属性包括了编码器属性和码率控制器属性两部分。
 - 设置未创建的通道的属性，则返回失败。
 - 如果pstAttr为空，则返回失败。
 - 设置编码图像大小时的限制和创建通道时的限制一样。
 - 编码通道属性分为动态属性和静态属性两种。其中，动态属性的属性值在通道创建时配置，在通道销毁之前可以被修改；静态属性的属性值在通道创建时配置，在通道创建之后不能被修改。
 - 此接口只能设置编码通道属性中的动态属性，如果设置静态属性，则返回失败。编码通道的编码协议、获取码流的方式（按帧还是按包获取码流）、编码图像最大宽高属性属于静态属性。另外，各个编码协议的静态属性由各个协议模块指定，具体请参见MI_VENC_ChnAttr_t [#321-mi_venc_chnattr_t]。
 - 设置编码通道属性中码率控制器属性，码率控制模式为VBR，当码率控制高级参数中u32MinIQp小于u32MinQp，其中u32MinQp是码率控制器属性，此接口将u32MinIQp的值修改为u32MinQp。
 - 设置编码器属性中通道宽高属性时，其中通道的优先级并不会恢复默认，编码通道的其它所有参数配置恢复默认值，并且清空码流buffer和缓存图像队列。

2.13. MI_VENC_GetChnAttr

- 功能

获取编码通道属性。

- 语法

```
MI_S32 MI_VENC_GetChnAttr(MI_VENC_CHN VeChn, MI_VENC_ChnAttr_t *pstAttr);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstAttr	编码通道属性指针	输出

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

- 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

- 注意

- 获取未创建的通道的属性，返回失败MI_ERR_VENC_UNEXIST。
- 如果pstAttr为空，则返回失败。

2.14. MI_VENC_GetStream

- 功能

获取编码的码流。

- 语法

```
MI_S32 MI_VENC_GetStream(MI_VENC_CHN VeChn, MI_VENC_Stream_t *pstStream, MI_S32 s32MilliSec);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstStream	码流结构体指针。上层需要将该指针指向一块分配好的内存，而且其内的成员MI_VENC_Pack_t *pstPack同样需要自行配置相应PackCount*sizeof(MI_VENC_Pack_t)的内存。	输入/输出
s32MilliSec	T调用该api直到获取到码流的超时等待时间。 取值范围：>=-1 -1：阻塞 0：非阻塞 >0：超时等待时间，该时间为相对时间，单位ms	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建, 返回失败。
 - 如果pstStream为空, 返回MI_ERR_VENC_NULL_PTR。
 - 如果s32MilliSec小于-1, 返回MI_ERR_VENC_ILLEGAL_PARAM。
 - 支持超时方式获取。支持select/poll系统调用。
 - s32MilliSec=0时, 则为非阻塞获取, 即如果缓冲无数据, 则返回失败MI_ERR_VENC_BUF_EMPTY。s32MilliSec=-1时, 则为阻塞, 即如果缓冲无数据, 则会等待有数据时才返回获取成功。s32MilliSec>0时, 则为超时, 即如果缓冲无数据, 则会等待用户设定的超时时间, 若在设定的时间内有数据则返回获取成功, 否则返回超时失败。
 - 码流结构体MI_VENC_Stream_t [#315-mi_venc_stream_t]包含4个部分:
 - a. 码流包信息指针pstPack指向一组MI_VENC_Pack_t [#311-mi_venc_pack_t]的内存空间, 该空间由调用者分配。大小为u32PackCount x sizeof(MI_VENC_Pack_t), 可以在select之后通过调用MI_VENC_Query [#211-mi_venc_query] 获得。
 - b. 码流包个数u32PackCount指定pstPack中MI_VENC_Pack_t [#311-mi_venc_pack_t]的个数。一张完整的帧是由u32PackCount个码流包构成。
 - c. 序列号u32Seq表示帧序列号。
 - d. 码流特征信息stH264Info/stJpegInfo/stH265Info, 数据类型为联合体, 包含了不同编码协议 对应的码流特征信息, 码流特征信息的输出用于支持用户的上层应用。
 - 此接口应当和MI_VENC_ReleaseStream 配对起来使用, 用户获取码流后系统不会主动释放码流 缓存, 需要用户及时释放已经获取的码流缓存, 否则可能会导致码流buffer满, 影响编码器 编码, 并且用户必须按先获取先释放的顺序释放已经获取的码流缓存。
 - 建议用户使用select方式获取码流, 流程如下:
 - a. 调用MI_VENC_Query [#211-mi_venc_query]函数查询编码通道状态;
 - b. 确保u32CurPacks和u32LeftStreamFrames同时大于0;
 - c. 调用malloc分配u32CurPacks个包信息结构体;
 - d. 调用MI_VENC_GetStream [#214-mi_venc_getstream]获取编码码流;
 - e. 调用MI_VENC_ReleaseStream [#215-mi_venc_releasestream]释放码流缓存。
 - 以H.264为例介绍pstPack[0]存储的码流结构, pstPack[0]对应信息如下:

```
pstPack[0].u32DataNum=3,
pstPack[0].stPackInfo[0].u32PackType.enH264EType=E_MI_VENC_H264E_NALU_SPS,
pstPack[0].stPackInfo[1].u32PackType.enH264EType=E_MI_VENC_H264E_NALU_PPS,
pstPack[0].stPackInfo[2].u32PackType.enH264EType=E_MI_VENC_H264E_NALU_SEI ;
```

即pstPack[0]包含三个NALU包: SPS/PPS/SEI。如下图所示:



- 举例

```

MI_S32 VencGetH264Stream()
{
    MI_S32 i;
    MI_S32 s32Ret;
    MI_S32 s32VencFd;
    MI_U32 u32FrameIdx = 0;
    MI_VENC_CHN VeChn = 0;
    MI_VENC_ChnAttr_t stStat;
    MI_VENC_Stream_t stStream;
    fd_set read_fds;
    FILE* pFile=NULL;

    pFile = fopen("stream.h264","wb");
    if (pFile == NULL)
    {
        return E_MI_ERR_FAILED;
    }
    s32VencFd = MI_VENC_GetFd(VeChn);
    do{
        FD_ZERO(&read_fds);
        FD_SET(s32VencFd, &read_fds);
        s32Ret = select(s32VencFd+1, &read_fds, NULL, NULL, NULL);
        if (s32Ret < 0)
        {
            printf("select err\n");
            return E_MI_ERR_FAILED;
        }
        else if (0 == s32Ret)
        {
            printf("timeout\n");
            return E_MI_ERR_FAILED;
        }
        else
        {
            if (FD_ISSET(s32VencFd, &read_fds))
            {
                s32Ret = MI_VENC_Query(VeChn, &stStat);
                if (s32Ret != MI_SUCCESS)
                {
                    return E_MI_ERR_FAILED;
                }

                /*****
                suggest to check both u32CurPacks and u32LeftStreamFrames at
                the same time,for 举例:
                if (0 == stStat.u32CurPacks || 0 == stStat.u32LeftStreamFrames)
                {
                    continue;
                }
                *****/
                if (0 == stStat.u32CurPacks)
                {
                    continue;
                }
                stStream.pstPack =
                (MI_VENC_Pack_t*)malloc(sizeof(MI_VENC_Pack_t)*stStat.u32CurPacks);
                if (NULL == stStream.pstPack)
                {
                    return E_MI_ERR_FAILED;
                }
                stStream.u32PackCount = stStat.u32CurPacks;
                s32Ret = MI_VENC_GetStream(VeChn, &stStream, -1);
                if (MI_SUCCESS != s32Ret)
                {
                    free(stStream.pstPack);
                    stStream.pstPack = NULL;
                    return E_MI_ERR_FAILED;
                }
                for (i=0; i<stStream.u32PackCount; i++)
                {
                    fwrite(stStream.pstPack[i].pu8Addr+stStream.pstPack[i].u320ffset, 1,
                        stStream.pstPack[i].u32Len-stStream.pstPack[i].u320ffset, pFile);
                }
                s32Ret = MI_VENC_ReleaseStream(VeChn, &stStream);
            }
        }
    } while (1);
}

```

```
        if (MI_SUCCESS != s32Ret)
        {
            free(stStream.pstPack);
            stStream.pstPack=NULL;
            return E_MI_ERR_FAILED;
        }
        free(stStream.pstPack);
        stStream.pstPack = NULL;
    }
    u32FrameIdx++;
}while(u32FrameIdx<0xff);
fclose(pFile);

return s32Ret;
}
```

2.15. MI_VENC_ReleaseStream

- 功能
释放码流缓存。
- 语法

```
MI_S32 MI_VENC_ReleaseStream(MI_VENC_CHN VeChn, MI_VENC_Stream_t *pstStream);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstStream	码流结构体指针	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回[错误码](#) [#jump]MI_ERR_VENC_UNEXIST。
 - 如果pstStream为空，则返回[错误码](#) [#jump]MI_ERR_VENC_NULL_PTR。
 - 此接口应当和[MI_VENC_GetStream](#) [#214-mi_venc_getstream]配对起来使用，用户获取码流后必须及时释放已经获取的码流缓存，否则可能会导致码流buffer满，影响编码器编码，并且用户必须按先获取先释放的顺序释放已经获取的码流缓存。
 - 在编码通道复位以后，所有未释放的码流包均无效，不能再使用或者释放这部分 无效的码流缓存。
 - 无效的缓存。
 - 释放无效的码流会返回失败MI_ERR_VENC_ILLEGAL_PARAM。
- 举例
请参见MI_VENC_GetStream的举例。

2.16. MI_VENC_InsertUserData

- 功能

插入用户数据。用户首先申请一块空间，填充自定义的信息，传入对应的指针和数据长度，在下一个SEI的NAL（跟在下一个I帧之前）就会有相应的信息。

- 语法

```
MI_S32 MI_VENC_InsertUserData(MI_VENC_CHN VeChn, MI_U8 *pu8Data, MI_U32 u32Len);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pu8Data	用户数据指针	输入
u32Len	用户数据长度 取值范围：(0,1024]，以byte为单位	输入

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

- 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

- 注意

- 如果通道未创建，则返回失败。
- 如果pu8Data为空，则返回失败。
- 插入用户数据，只支持H.264/H.265编码协议。
- H.264 协议通道最多同时分配4块内存空间用于缓存用户数据，且每段用户数据大小不超过 1kbyte。如果用户插入的数据多余4块，或插入的一段用户数据大于1kbyte时，此接口会返回 错误。每段用户数据以SEI包的形式被插入到下一个I帧之前。在某段用户数据包被编码发送 之后，H.264通道内缓存这段用户数据的内存空间被清零，用于存放新的用户数据。

- 举例

```
MI_S32 VencInsertUserData()  
{  
    MI_S32 s32Ret;  
    MI_VENC_CHN VeChn = 0;  
    MI_U8 au8UserData[] = "sigmastar2020";  
    s32Ret = MI_VENC_InsertUserData(VeChn, au8UserData, sizeof(au8UserData));  
    if (MI_SUCCESS != s32Ret)  
    {  
        printf("MI_VENC_InsertUserData err0x%x\n", s32Ret);  
        return E_MI_ERR_FAILED;  
    }  
  
    return s32Ret;  
}
```

2.17. MI_VENC_SetMaxStreamCnt

- 功能
设置码流最大缓存帧数。
- 语法

```
MI_S32 MI_VENC_SetMaxStreamCnt(MI_VENC_CHN VeChn, MI_U32 u32MaxStrmCnt);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
u32MaxStrmCnt	最大码流缓存帧数 取值范围：>=1	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 此接口用于设置编码通道的码流buffer中能够缓存的最大码流帧数。
 - 缓存码流帧数已达到最大码流帧数，当前待编码图像因码流buffer满而直接丢掉不编码。
 - 最大码流帧数在创建通道时由系统内部指定默认值，默认值为3。
 - 此接口在创建通道之后，销毁编码通道之前均可以被调用。在下一帧开始编码之前生效。此接口允许被多次调用。建议在创建编码通道成功之后，启动编码前进行设置，不建议在编码过程中动态调整。

2.18. MI_VENC_GetMaxStreamCnt

- 功能
获取码流最大缓存帧数。
- 语法

```
MI_S32 MI_VENC_GetMaxStreamCnt(MI_VENC_CHN VeChn, MI_U32 *pu32MaxStrmCnt);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pu32MaxStrmCnt	最大码流缓存帧数的指针	输出

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h, mi_venc.h
 - 库文件: libmi.a

2.19. MI_VENC_RequestIdr

- 功能
请求一张IDR帧。
- 语法

```
MI_S32 MI_VENC_RequestIdr(MI_VENC_CHN VeChn, MI_BOOL bInstant);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
bInstant	是否使能立即编码IDR帧: 0: 在尽可能短的时间内编出IDR帧; 1: 下帧立即编码IDR帧	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回失败。
 - IDR帧请求，只支持H.264/H.265编码协议。
 - 由于此接口不受帧率控制影响，每调用一次即编出一个IDR，调用频繁会影响码流帧率和码率 的稳定，使用时需注意
- 举例

```
MI_S32 VencRequestIdr()  
{  
    MI_S32 s32Ret;  
    MI_VENC_CHN VeChn = 0;  
    MI_BOOL bInstant;  
  
    bInstant = TRUE;  
    s32Ret = MI_VENC_RequestIdr(VeChn, bInstant);  
    if (MI_SUCCESS != s32Ret)  
    {  
        printf("MI_VENC_RequestIDR err0x%x\n", s32Ret);  
        return E_MI_ERR_FAILED;  
    }  
  
    return s32Ret;  
}
```

2.20. MI_VENC_EnableIdr

- 功能
是否使能IDR帧。
- 语法

```
MI_S32 MI_VENC_EnableIdr(MI_VENC_CHN VeChn, MI_BOOL bEnableIdr);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
bEnableIDR	是否使能的标志	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 注意
 - 如果通道未创建，则返回失败。
 - 若不使能IDR帧，则在下一帧之后都编不出IDR帧或I帧，直到再次使能为止。
 - 本接口只支持H.264/H.265编码协议。
- 举例

```
MI_S32 VencEnableIdr()  
{  
    MI_S32 s32Ret;  
    MI_VENC_CHN VeChn = 0;  
    s32Ret = MI_VENC_EnableIdr(VeChn);  
    if (MI_SUCCESS != s32Ret)  
    {  
        printf("MI_VENC_EnableIdr err0x%x\n", s32Ret);  
        return E_MI_ERR_FAILED;  
    }  
    return s32Ret;  
}
```

2.21. MI_VENC_SetH264IdrPicId

- 功能
设置IDR帧的idr_pic_id属性。
- 语法

```
MI_S32 MI_VENC_SetH264IdrPicId(MI_VENC_CHN VeChn, MI_VENC_H264IdrPicIdCfg_t *pstH264eIdrPicIdCfg);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstH264IdrPicIdCfg	idr_pic_id的参数指针	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回失败。
 - Idr_pic_id 的参数主要由两个参数决定：
 - a. enH264eIdrPicIdMode: 设置IDR帧idr_pic_id的模式，enH264eIdrPicIdMode=E_MI_VENC_H264E_IDR_PIC_ID_MODE_AUTO表示由编码内部流程自动计算生成idr_pic_id，enH264eIdrPicIdMode=E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR表示由用户设定idr_pic_id的值。
 - b. u32H264eIdrPicId: 设置IDR帧idr_pic_id的值，当enH264eIdrPicIdMode=E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR时该值才有效。
 - 设置IDR帧的idr_pic_id，只支持H.264编码协议。
 - 此接口在创建通道之后，销毁编码通道之前均可以被调用。在下一帧开始编码之前生效。此接口允许被多次调用。建议在创建编码通道成功之后，启动编码前进行设置，不建议在编码过程中动态调整。
 - 暂不支持

2.22. MI_VENC_GetH264IdrPicId

- 功能

获取IDR帧的idr_pic_id的配置属性。
- 语法

```
MI_S32 MI_VENC_GetH264IdrPicId(MI_VENC_CHN VeChn, MI_VENC_H264IdrPicIdCfg_t *pstH264eIdrPicIdCfg);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstH264eIdrPicIdCfg	idr_pic_id的参数指针	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回失败。
 - 此接口在创建通道之后，销毁编码通道之前均可以被调用。
 - 暂不支持

2.23. MI_VENC_GetFd

- 功能

获取编码通道对应的设备文件句柄，用于调用select/poll监听是否有编码数据，与轮询CPU相比有效降低了cpu使用率。
- 语法

```
MI_S32 MI_VENC_GetFd(MI_VENC_CHN VeChn);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 举例

请参见[MI_VENC_GetStream](#) [#214-mi_venc_getstream]的举例。

2.24. MI_VENC_CloseFd

- 功能

关闭编码通道对应的设备文件句柄。
- 语法

```
MI_S32 MI_VENC_CloseFd(MI_VENC_CHN VeChn);
```

- 参数

参数名称	描述	输入/输出
VeChn	视频编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h, mi_venc.h
 - 库文件: libmi.a

2.25. MI_VENC_SetRoiCfg

- 功能
设置H.264/H.265通道的ROI属性。
- 语法

```
MI_S32 MI_VENC_SetRoiCfg(MI_VENC_CHN VeChn, MI_VENC_RoiCfg_t *pstVencRoiCfg);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围 : [0,VENC_MAX_CHN_NUM)	输入
pstVencRoiCfg	ROI区域参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 芯片差异
不同Chip和编码协议对ROI参数的限制也不同，如下表：

Chip	Codec	Alignment				bAbsQp	
		u32Left	u32Top	u32Width	u32Height	AbsQp	RelQP

Pretzel	H.264	16	16	16	16	[12,48]	[-7,7]
	H.265	32	32	32	32	[12,48]	[-7,7]
Macaron	H.264	16	16	16	16	[12,48]	[-7,7]
	H.265	32	32	32	32	[12,48]	[-7,7]
Pudding	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32	No support	[-32,31]
Ispahan	H.264	16	16	16	16	No support	[-32,31]
	H.265	32	32	32	32	No support	[-32,31]

- 注意
 - 本接口用于设置H.264/H.265协议编码通道ROI区域的参数。ROI参数主要由5个参数决定：
 - a. 系统支持每个通道可设置8个ROI区域，系统内部按照0~7的索引号对ROI区域进行管理，u32Index表示的用户设置ROI的索引号。ROI区域之间可以互相叠加，且当发生叠加时，ROI区域之间的优先级按照索引号0~7依次提高。
 - b. bEnable: 指定当前的ROI区域是否使能。
 - c. bAbsQp: 指定当前的ROI区域采用绝对QP方式或是相对QP。
 - d. s32Qp: 当bAbsQp为true时，s32Qp表示ROI区域内部的所有宏块采用的QP值，当bAbsQp为false时，s32Qp表示ROI区域内部的所有宏块采用的相对QP值。
 - e. tRect: 指定当前的ROI区域的位置坐标和区域的大小。ROI区域的起始点坐标必须在图像范围内。
 - 本接口属于高级接口，系统默认没有ROI区域使能，用户必须调用此接口启动ROI。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。此接口在编码过程中被调用时，等到下一个帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用MI_VENC_GetRoiCfg接口，获取当前通道的ROI配置，然后再进行设置。
- 举例

```

MI_S32 VencSetRoi()
{
    MI_S32 s32Ret;
    MI_VENC_RoiCfg_t stRoiCfg;
    MI_S32 index=0;
    MI_VENC_CHN VeChnId=0;
    //...omit other thing
    s32Ret = MI_VENC_GetRoiCfg(VeChnId, index, &stRoiCfg);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetRoiCfg err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stRoiCfg.bEnable = TRUE;
    stRoiCfg.bAbsQp = FALSE;
    stRoiCfg.s32Qp = 10;
    stRoiCfg.stRect.u32Left = 16;
    stRoiCfg.stRect.u32Top = 16;
    stRoiCfg.stRect.u32Width = 16;
    stRoiCfg.stRect.u32Height = 16;
    stRoiCfg.u32Index = 0;
    s32Ret = MI_VENC_SetRoiCfg(VeChnId, &stRoiCfg);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetRoiCfg err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
}

```



```
}

    return s32Ret;
}
```

2.26. MI_VENC_GetRoiCfg

- 功能
获取H.264/H.265通道的Roi配置属性。
- 语法

```
MI_S32 MI_VENC_GetRoiCfg(MI_VENC_CHN VeChn, MI_U32 u32Index, MI_VENC_RoiCfg_t *pstVencRoiCfg);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
u32Index	H.264协议编码通道ROI区域索引	输入
pstVencRoiCfg	对应ROI区域的配置	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.264协议编码通道的index的ROI配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例
请参见[MI_VENC_GetRoiCfg](#) [#226-mi_venc_getroicfg] 的举例。

2.27. MI_VENC_SetRoiBgFrameRate

- 功能
设置H.264/H.265通道的非ROI区域帧率属性。
- 语法

```
MI_S32 MI_VENC_SetRoiBgFrameRate(MI_VENC_CHN VeChn, MI_VENC_RoiBgFrameRate_t *pstRoiBgFrmRate);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRoiBgFrmRate	非ROI区域帧率控制参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.264协议编码通道非ROI区域帧率控制的参数。
 - 本接口调用之前必须首先使能ROI区域。
 - 本接口属于高级接口，系统默认没有使能非ROI区域帧率属性，用户必须调用此接口使能。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。此接口在编码过程中被调用时，等到下一帧时生效。
 - 设置通道帧率控制属性时，输入帧率SrcFrmRate大于输出帧率DstFrmRate且DstFrmRate大于等于0或同时等于-1。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用[MI_VENC_GetRoiBgFrameRate](#) [#228-mi_venc_getroibgframerate]接口，
 - 获取当前通道的非ROI区域帧率配置，然后再进行设置。
 - 设置非ROI区域的帧率低于ROI区域时，不建议使用跳帧参考或者svc-t编码。跳帧参考或svc-t编码场景下，如果设置了非ROI区域的帧率低于ROI区域，则实际编码的非ROI区域目标帧率可能大于用户配置的目标帧率，因为base层的非ROI区域不能编码为pskip块。例如在调用了此接口设置了非ROI区域帧率比例为2:1之前，还调用了[MI_VENC_SetRefParam](#) [#259-mi_venc_setrefparam]设置长参考帧，会导致实际编码的非ROI区域帧率比例低于2:1。
- 举例

```
MI_S32 VencSetRoiFrameRate()
{
    MI_S32 s32Ret;
    MI_VENC_RoiBgFrameRate_t stRoiBgFrameRate;
    MI_S32 index = 0;
    MI_VENC_CHN VeChnId=0;

    //...omit other thing
    stRoiBgFrameRate.s32SrcFrmRate=30;
    stRoiBgFrameRate.s32DstFrmRate=15;
    s32Ret = MI_VENC_SetRoiBgFrameRate(VeChnId, &stRoiBgFrameRate);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetRoiBgFrameRate err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.28. MI_VENC_GetRoiBgFrameRate

- 功能

获取H.264/H.265通道的非Roi区域帧率配置属性。

- 语法

```
MI_S32 MI_VENC_GetRoiBgFrameRate(MI_VENC_CHN VeChn, MI_VENC_RoiBgFrameRate_t *pstRoiBgFrmRate);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRoiBgFrmRate	非ROI区域帧率的配置	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetRoiBgFrameRate](#) [#227-mi_venc_setroibgframerate]的举例。

2.29. MI_VENC_SetH264SliceSplit

- 功能

设置H.264通道的slice分割属性。
- 语法

```
MI_S32 MI_VENC_SetH264SliceSplit(MI_VENC_CHN VeChn, MI_VENC_ParamH264SliceSplit_t *pstSliceSplit);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstSliceSplit	H.264码流slice分割参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h

- 库文件: libmi.a
- 注意
 - 本接口用于设置H.264协议编码通道码流的分割方式
 - Slice 分割属性主要由两个参数决定
 - a. bSplitEnable: 当前帧是否进行slice分割。
 - b. u32SliceRowCount: Slice 按照宏块行进行分割。u32SliceRowCount表示每个slice占图像宏块行数。且当编码至图像的最后几行，不足u32SliceRowCount时，剩余的宏块行被划分为一个slice。
 - 系统默认bSplitEnable为false，当设置bSplitEnable为true时，以u32SliceRowCount宏块行作为一个slice单独编码，每编码完成一个slice用户即可获取该slice的码流数据。建议u32SliceRowCount不要设置过小，u32SliceRowCount越小用户收到通知越频繁，系统开销越大，且容易造成码流输出内存碎片，降低码流输出内存的使用效率。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。此接口在编码过程中被调用时，等到下一帧时生效。
 - 建议用户在调用此接口之前，先调用MI_VENC_GetH264SliceSplit [#230-mi_venc_geth264slicesplit]接口，获取当前通道的slicesplit配置，然后再进行设置。
- 举例

```
MI_S32 VencSetSliceSplit()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264SliceSplit_t stSlice;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264SliceSplit(VeChnId, &stSlice);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264SliceSplit err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stSlice.bSplitEnable = TRUE;
    stSlice.u32SliceRowCount = 8;
    s32Ret = MI_VENC_SetH264SliceSplit(VeChnId, &stSlice);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264SliceSplit err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.30. MI_VENC_GetH264SliceSplit

- 功能

获取H.264通道的slice分割属性。
- 语法

```
MI_S32 MI_VENC_GetH264SliceSplit(MI_VENC_CHN VeChn, MI_VENC_ParamH264SliceSplit_t *pstSliceSplit);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstSliceSplit	H.264码流slice分割参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 此接口用于获取H.264协议编码通道的slice分割属性。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH264SliceSplit](#) [#229-mi_venc_seth264slicesplit]的举例。

2.31. MI_VENC_SetH264InterPred

- 功能

设置H.264协议编码通道的帧间预测属性。
- 语法

```
MI_S32 MI_VENC_SetH264IntraPred(MI_VENC_CHN VeChn, MI_VENC_ParamH264IntraPred_t *pstH264IntraPred);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264InterPred	H.264协议编码通道的帧间预测配置	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.264协议编码通道的帧间预测的配置
 - 帧间预测的属性主要由七个参数决定：

- a. u32HWSIZE: 帧间预测时的水平搜索窗的大小。
 - b. u32VWSIZE: 帧间预测时的垂直搜索窗的大小。
 - c. bInter16x16PredEn: 16x16块帧间预测使能标识。
 - d. bInter16x8PredEn: 16x8块帧间预测使能标识。
 - e. bInter8x16PredEn: 8x16块帧间预测使能标识。
 - f. bInter8x8PredEn: 8x8块帧间预测使能标识。
 - g. bExtedgeEn: 帧间预测扩边搜索使能。对于图像边界上的宏块进行帧间预测时，搜索窗的范围会超出图像的边界，此时，扩边搜索使能会对图像进行扩边，进行帧间预测。如果扩边搜索使能被关闭，那么对于超出图像范围的搜索窗，不会进行预测。
- 此接口属于高级接口，用户可以选择性调用，建议不调用，系统会有默认值。默认搜索窗的大小会根据图像的分辨率不同，其他5个预测使能开关全部默认使能。
 - bInter16x16PredEn的预测开关只能开启。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用[MI_VENC_GetH264InterPred](#) [#232-mi_venc_geth264interpreted]接口，获取当前编码通道的InterPred配置，然后再进行设置。
- 举例

```
MI_S32 VencSetInterPred()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264InterPred_t stInterPred;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264InterPred(VeChnId, &stInterPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264InterPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stInterPred.bExtedgeEn = TRUE;
    stInterPred.bInter16x16PredEn = TRUE;
    stInterPred.bInter16x8PredEn = FALSE;
    stInterPred.bInter8x16PredEn = FALSE;
    stInterPred.bInter8x8PredEn = FALSE;
    stInterPred.u32HWSIZE = 4;
    stInterPred.u32VWSIZE = 2;
    s32Ret = MI_VENC_SetH264InterPred(VeChnId, &stInterPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264InterPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.32. MI_VENC_GetH264InterPred

- 功能

获取H.264协议编码通道的帧间预测属性。
- 语法

```
MI_S32 MI_VENC_GetH264IntraPred(MI_VENC_CHN VeChn, MI_VENC_ParamH264IntraPred_t *pstH264IntraPred);
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264InterPred	H.264协议编码通道的帧间预测参数	输出

• 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

• 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

• 注意

- 本接口用于获取H.264协议编码通道的帧间预测方式。
- 本接口可在编码通道创建之后，编码通道销毁之前调用。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。

• 举例

请参见[MI_VENC_SetH264InterPred](#) [#231-mi_venc_seth264interpred]的举例。

2.33. MI_VENC_SetH264IntraPred

• 功能

设置H.264协议编码通道的帧内预测属性。

• 语法

```
MI_S32 MI_VENC_SetH264IntraPred(MI_VENC_CHN VeChn,MI_VENC_ParamH264IntraPred_t *pstH264IntraPred);
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264IntraPred	H.264协议编码通道的帧内预测配置	输入

• 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

• 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

• 注意

- 本接口用于设置H.264协议编码通道的帧内预测的配置

- 帧间预测的属性主要由四个参数决定：
 - a. bIntra16x16PredEn: 16x16块帧内预测使能标识。
 - b. bIntraNxNPredEn: NxN块帧内预测使能标识。其中，NxN表示4x4和8x8。
 - c. blpcmEn: IPCM块帧内预测使能标识。
 - d. constrained_intra_pred_flag: 具体的含义，请参见H.264协议。
- 本接口属于高级接口，用户可以选择性调用，不建议调用，系统会有默认值。系统默认使能bIntra16x16PredEn和bIntraNxNPredEn，blpcmEn对于不同芯片有不同默认值，constrained_intra_pred_flag默认为0。
- bIntra16x16PredEn、bIntraNxNPredEn两种帧内预测使能开关必须有一个是开启的，系统不支持两个开关全部关闭。
- 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 建议用户在调用此接口之前，先调用MI_VENC_GetH264IntraPred [#234-mi_venc_geth264intrapred]接口，获取当前编码通道的IntraPred配置，然后再进行设置。
- 举例

```
MI_S32 VencSetInterPred()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264InterPred_t stInterPred;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264InterPred(VeChnId, &stInterPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264InterPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stInterPred.bExtedgeEn = TRUE;
    stInterPred.bInter16x16PredEn = TRUE;
    stInterPred.bInter16x8PredEn = FALSE;
    stInterPred.bInter8x16PredEn = FALSE;
    stInterPred.bInter8x8PredEn = FALSE;
    stInterPred.u32HWSIZE = 4;
    stInterPred.u32VWSIZE = 2;
    s32Ret = MI_VENC_SetH264InterPred(VeChnId, &stInterPred);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264InterPred err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.34. MI_VENC_GetH264IntraPred

- 功能
获取H.264协议编码通道的帧内预测属性。
- 语法

```
MI_S32 MI_VENC_GetH264IntraPred(MI_VENC_CHN VeChn, MI_VENC_ParamH264IntraPred_t *pstH264IntraPred);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264IntraPred	H.264协议编码通道的帧内预测参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.264协议编码通道的帧内预测方式。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH264IntraPred](#) [#233-mi_venc_seth264intrapred]的举例。

2.35. MI_VENC_SetH264Trans

- 功能

设置H.264协议编码通道的变换、量化的属性。
- 语法

```
MI_S32 MI_VENC_SetH264Trans(MI_VENC_CHN VeChn, MI_VENC_ParamH264Trans_t *pstH264Trans);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264Trans	H.264协议编码通道的变换、量化属性	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: mi_venc.ko, libmi_venc.so
- 注意
 - 本接口用于设置H.264协议编码通道的变换、量化的配置。
 - 变换、量化属性主要由三个参数组成：

- a. `u32IntraTransMode`: 帧内预测宏块的变换属性。`u32IntraTransMode=0`表示支持对帧内预测宏块支持4x4变换和8x8变换；`u32IntraTransMode=1`表示只支持对帧内预测宏块支持4x4变换；`u32IntraTransMode=2`表示只支持对帧内预测宏块支持8x8变换。只有在编码通道协议为highprofile时，才能选择8x8变换，即在baselineprofile和mainprofile下，系统只支持的配置。
 - b. `u32InterTransMode`: 帧间预测宏块的变换属性。`u32InterTransMode=0`表示支持对帧间预测宏块支持4x4变换和8x8变换；`u32InterTransMode=1`表示只支持对帧间预测宏块支持4x4变换；`u32InterTransMode=2`表示只支持对帧间预测宏块支持8x8变换。只有在编码通道协议为highprofile时，才能选择8x8变换，即在baselineprofile和mainprofile下，系统只支持的配置。
 - c. `s32ChromaQpIndexOffset`: 具体含义请参见H.264协议关于chroma_qp_index_offset的解释。
- 此接口属于高级接口，用户可以选择性调用，不建议调用，系统会有默认值。系统会根据不同的profile设置默认参数。不同profile下系统默认trans参数如下表：

Profile	u32IntraTransMode	u32InterTransMode	bScalingListValid
Baseline/ mainprofile	1	1	false
Highprofile	0	0	false

- 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 建议用户在调用此接口之前，先调用MI_VENC_GetH264Trans [#236-mi_venc_geth264trans]接口，获取当前编码通道的trans配置，然后再进行设置。

- 举例

```

MI_S32 VencSetTrans()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264Trans_t stTrans;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264Trans(VeChnId, &stTrans);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264Trans err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stTrans.u32IntraTransMode = 2;
    stTrans.u32InterTransMode = 2;
    stTrans.s32ChromaQpIndexOffset = 2;
    s32Ret = MI_VENC_SetH264Trans(VeChnId, &stTrans);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Trans err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}

```

2.36. MI_VENC_GetH264Trans

- 功能

获取H.264协议编码通道的变换、量化属性。
- 语法

```
MI_S32 MI_VENC_GetH264Trans(MI_VENC_CHN VeChn, MI_VENC_ParamH264Trans_t *pstH264Trans);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264Trans	H.264协议编码通道的变换、量化参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.264协议编码通道的变换、量化配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH264Trans](#) [#235-mi_venc_seth264trans]的举例。

2.37. MI_VENC_SetH264Entropy

- 功能

设置H.264协议编码通道的熵编码模式。
- 语法

```
MI_S32 MI_VENC_SetH264Entropy(MI_VENC_CHN VeChn, MI_VENC_ParamH264Entropy_t *pstH264EntropyEnc);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264EntropyEnc	H.264协议编码通道的熵编码模式	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a

- 注意
 - 本接口用于设置H.264协议编码通道的熵编码的配置。
 - 变换、量化属性主要由两个参数组成：
 - a. u32EntropyEncModel: I帧的熵编码方式，u32EntropyEncModel=0表示I帧使用cavlc编码，u32EntropyEncModel=1表示I帧使用cabac编码方式。
 - b. u32EntropyEncModeP: P帧的熵编码方式，u32EntropyEncModeP=0表示P帧使用cavlc编码，u32EntropyEncModeP=1表示P帧使用cabac编码方式。
 - I帧的熵编码方式与P帧的熵编码方式可以分别设置。
 - Baselineprofile不支持cabac编码方式，支持cavlc编码方式，mainprofile和highprofile支持cabac编码方式和cavlc编码方式。
 - Cabac编码方式相对于cavlc编码方式，需要更大的计算量，可是，会产生更少的码流。如果系统性能不够富裕，建议用户可以采取I帧cavlc编码，P帧cabac编码。
 - 此接口属于高级接口，用户可以选择性调用，不建议调用，系统会有默认值。系统会根据不同的profile设置默认参数。不同profile下系统默认熵编码参数如下表：

Profile	u32EntropyEncModel	u32EntropyEncModeP
Baseline	0	0
Mainprofile/Highprofile	1	1

- 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 建议用户在调用此接口之前，先调用MI_VENC_GetH264Entropy [#238-mi_venc_geth264entropy]接口，获取当前编码通道的Entropy配置，然后再进行设置。
- 举例

```
MI_S32 VencSetEntropy()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264Entropy_t stEntropy;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264Entropy(VeChnId, &stEntropy);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264Entropy err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stEntropy.u32EntropyEncModeI = 0;
    stEntropy.u32EntropyEncModeP = 0;
    s32Ret = MI_VENC_SetH264Entropy(VeChnId, &stEntropy);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Entropy err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.38. MI_VENC_GetH264Entropy

- 功能

获取H.264协议编码通道的熵编码属性。

- 语法

```
MI_S32 MI_VENC_GetH264Entropy(MI_VENC_CHN VeChn,MI_VENC_ParamH264Entropy_t *pstH264EntropyEnc);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264EntropyEnc	H.264协议编码通道的熵编码属性	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.264协议编码通道的熵编码配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH264Entropy](#) [#237-mi_venc_seth264entropy]的举例。

2.39. MI_VENC_SetH265InterPred

- 功能

设置H.265协议编码通道的帧间预测属性。
- 语法

```
MI_S32 MI_VENC_SetH265InterPred(MI_VENC_CHN VeChn,MI_VENC_ParamH265InterPred_t *pstH265InterPred);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265InterPred	H.265协议编码通道的帧间预测配置	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a
- 注意
 - 本接口用于设置H.265协议编码通道的帧间预测的配置
 - 帧间预测的属性主要由七个参数决定：
 - a. u32HWSIZE: 帧间预测时的水平搜索窗的大小。
 - b. u32VWSIZE: 帧间预测时的垂直搜索窗的大小。
 - c. bInter16x16PredEn: 16x16块帧间预测使能标识。
 - d. bInter16x8PredEn: 16x8块帧间预测使能标识。
 - e. bInter8x16PredEn: 8x16块帧间预测使能标识。
 - f. bInter8x8PredEn: 8x8块帧间预测使能标识。
 - g. bExtedgeEn: 帧间预测扩边搜索使能。对于图像边界上的宏块进行帧间预测时，搜索窗的范围会超出图像的边界，此时，扩边搜索使能会对图像进行扩边，进行帧间预测。如果扩编搜索使能被关闭，那么对于超出图像范围的搜索窗，不会进行预测。
 - 此接口属于高级接口，用户可以选择性调用，建议不调用，系统会有默认值。默认搜索窗的大小会根据图像的分辨率不同，其他5个预测使能开关全部默认使能。
 - bInter16x16PredEn的预测开关只能开启。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用MI_VENC_GetH265InterPred [#240-mi_venc_geth265interpred]接口，获取当前编码通道的InterPred配置，然后再进行设置。
- 举例

请参考MI_VENC_SetH264InterPred [#231-mi_venc_seth264interpred]的举例。

2.40. MI_VENC_GetH265InterPred

- 功能

获取H.265协议编码通道的帧间预测属性。
- 语法

```
MI_S32 MI_VENC_GetH265InterPred(MI_VENC_CHN VeChn,MI_VENC_ParamH265InterPred_t *pstH265InterPred);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265InterPred	H.265协议编码通道的帧间预测参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照错误码 [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h

- 库文件: libmi.a
- 注意
 - 本接口用于获取H.265协议编码通道的帧间预测方式。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH265InterPred](#) [#239-mi_venc_seth265interpred]的举例。

2.41. MI_VENC_SetH265IntraPred

- 功能
设置H.265协议编码通道的帧内预测属性。
- 语法

```
MI_S32 MI_VENC_SetH265IntraPred(MI_VENC_CHN VeChn,MI_VENC_ParamH265IntraPred_t *pstH265IntraPred);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265IntraPred	H.265协议编码通道的帧内预测配置	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.265协议编码通道的帧内预测的配置
 - 帧内预测的属性主要由三个参数决定：
 - a. u32Intra32x32Penalty: 32x32块帧内预测被选中概率，值越大概率越低。
 - b. u32Intra16x16Penalty: 16x16块帧内预测被选中概率，值越大概率越低。
 - c. u32Intra8x8Penalty: 8x8块帧内预测被选中概率，值越大概率越低。
 - 此接口属于高级接口，用户可以选择性调用，不建议调用，系统会有默认值。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用[MI_VENC_GetH265IntraPred](#) [#242-mi_venc_geth265intrapred]接口，获取当前编码通道的IntraPred配置，然后再进行设置。
- 举例

```
MI_S32 H265SetIntraPred()  
{  
    MI_S32 s32Ret;
```

```
MI_VENC_ParamH265IntraPred_t stIntraPred;
MI_VENC_CHN VeChnId = 0;

//...omit other thing
s32Ret = MI_VENC_GetH265IntraPred(VeChnId, &stIntraPred);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VENC_GetH265IntraPred err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}
stIntraPred.u32Intra32x32Penalty = 0;
stIntraPred.u32Intra16x16Penalty = 0;
stIntraPred.u32Intra8x8Penalty = 10;
s32Ret = MI_VENC_SetH265IntraPred(VeChnId, &stIntraPred);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VENC_SetH265IntraPred err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}

return s32Ret;
}
```

2.42. MI_VENC_GetH265IntraPred

- 功能
获取H.265协议编码通道的帧内预测属性。
- 语法

```
MI_S32 MI_VENC_GetH265IntraPred(MI_VENC_CHN VeChn,MI_VENC_ParamH265IntraPred_t *pstH265IntraPred);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265IntraPred	H.265协议编码通道的帧内预测参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.265协议编码通道的帧内预测方式。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH265IntraPred](#) [#241-mi_venc_seth265intrapred]的举例。

2.43. MI_VENC_SetH265Trans

- 功能
设置H.265协议编码通道的变换、量化的属性。
- 语法

```
MI_S32 MI_VENC_SetH265Trans(MI_VENC_CHN VeChn,MI_VENC_ParamH265Trans_t *pstH265Trans);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265Trans	H.265协议编码通道的变换、量化属性	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.265协议编码通道的变换、量化的配置。
 - 变换、量化属性主要由三个参数组成：
 - u32IntraTransMode: 帧内预测宏块的变换属性。u32IntraTransMode=0表示支持对帧内预测宏块支持4x4变换和8x8变换；u32IntraTransMode=1表示只支持对帧内预测宏块支持4x4变换；u32IntraTransMode=2表示只支持对帧内预测宏块支持8x8变换。只有在编码通道协议为highprofile时，才能选择8x8变换，即在baselineprofile和mainprofile下，系统只支持u32IntraTransMode=1的配置。
 - u32InterTransMode: 帧间预测宏块的变换属性。u32InterTransMode=0表示支持对帧间预测宏块支持4x4变换和8x8变换；u32InterTransMode=1表示只支持对帧间预测宏块支持4x4变换；u32InterTransMode=2表示只支持对帧间预测宏块支持8x8变换。只有在编码通道协议为highprofile时，才能选择8x8变换，即在baselineprofile和mainprofile下，系统只支持u32InterTransMode=1的配置。
 - s32ChromaQpIndexOffset: 具体含义请参见H.265协议关于slice_cb_qp_offset的解释。
 - 此接口属于高级接口，用户可以选择性调用，不建议调用，系统会有默认值。系统会根据不同的profile设置默认参数。不同profile下系统默认trans参数如下表：

Profile	u32IntraTransMode	u32InterTransMode	bScalingListValid
Baseline/ mainprofile	1	1	false
Highprofile	0	0	false

- 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 建议用户在调用此接口之前，先调用MI_VENC_GetH265Trans [#244-mi_venc_geth265trans]接口，获取当前编码通道的trans配置，然后再进行设置。

- 举例

```
MI_S32 H265SetTrans()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH265Trans_t stTrans;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH265Trans(VeChnId, &stTrans);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH265Trans err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stTrans.u32IntraTransMode = 2;
    stTrans.u32InterTransMode = 2;
    stTrans.s32ChromaQpIndexOffset = 2;
    s32Ret = MI_VENC_SetH265Trans(VeChnId, &stTrans);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH265Trans err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.44. MI_VENC_GetH265Trans

- 功能
获取H.265协议编码通道的变换、量化属性。
- 语法

```
MI_S32 MI_VENC_GetH265Trans(MI_VENC_CHN VeChn MI_VENC_ParamH265Trans_t *pstH265Trans);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265Trans	H.265协议编码通道的变换、量化参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.265协议编码通道的变换、量化配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见[MI_VENC_SetH265Trans](#) [#243-mi_venc_seth265trans]的举例。

2.45. MI_VENC_SetH264Dbk

- 功能
设置H.264协议编码通道的Deblocking类型。
- 语法

```
MI_S32 MI_VENC_SetH264Dbk(MI_VENC_CHN VeChn, MI_VENC_ParamH264Dbk_t *pstH264Dbk);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264Dbk	H.264协议编码通道的Deblocking参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.264协议编码通道的Deblocking的配置
 - 变换、量化属性主要由三个参数组成：
 - a. disable_deblocking_filter_idc: 具体含义请参见H.264协议。
 - b. Slice_alpha_c0_offset_div2: 具体含义请参见H.264协议。
 - c. Slice_beta_offset_div2: 具体含义请参见H.264协议。
 - 系统默认打开deblocking功能，默认disable_deblocking_filter_idc=0，slice_alpha_c0_offset_div2=0，slice_beta_offset_div2=0。
 - 如果用户想关闭deblocking功能，可以将disable_deblocking_filter_idc置为1。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用[MI_VENC_GetH264Dbk](#) [#246-mi_venc_geth264dbk]接口，获取当前编码通道的Dbk配置，然后再进行设置。
- 举例

```
MI_S32 VencSetDbk()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264Dbk_t stDbk;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264Dbk(VeChnId, &stDbk);
    if (MI_SUCCESS != s32Ret)
    {
```

```
        printf("MI_VENC_GetH264Db1k err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stDb1k.disable_deblocking_filter_idc = 0;
    stDb1k.slice_alpha_c0_offset_div2 = 6;
    stDb1k.slice_beta_offset_div2 = 5;
    s32Ret = MI_VENC_SetH264Db1k(VeChnId,&stDb1k);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Db1k err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.46. MI_VENC_GetH264Db1k

- 功能
获取H.264协议编码通道的db1k类型。
- 语法

```
MI_S32 MI_VENC_GetH264Db1k(MI_VENC_CHN VeChn MI_VENC_ParamH264Db1k_t *pstH264Db1k);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264Db1k	H.264协议编码通道的db1k属性	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.264协议编码通道的db1k配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例
请参见[MI_VENC_SetH264Db1k](#) [#245-mi_venc_seth264db1k]的举例。

2.47. MI_VENC_SetH265Db1k

- 功能
设置H.265协议编码通道的Deblocking类型。
- 语法

```
MI_S32 MI_VENC_SetH265Db1k(MI_VENC_CHN VeChn,MI_VENC_ParamH265Db1k_t *pstH265Db1k);
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265Db1k	H.265协议编码通道的Deblocking参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.265协议编码通道的Deblocking的配置
 - 变换、量化属性主要由三个参数组成：
 - a. disable_deblocking_filter_idc: 具体含义请参见H.265协议关于slice_deblocking_filter_disabled_flag的解释。
 - b. slice_tc_offset_div2: 具体含义请参见H.265协议。
 - c. slice_beta_offset_div2: 具体含义请参见H.265协议。
 - 系统默认打开deblocking功能，默认disable_deblocking_filter_idc=0， slice_tc_offset_div2=0， slice_beta_offset_div2=0。
 - 如果用户想关闭deblocking功能，可以将disable_deblocking_filter_idc置为1。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用[MI_VENC_GetH265Db1k](#) [#248-mi_venc_geth265db1k] 接口，获取当前编码通道的Db1k配置，然后再进行设置。
- 举例

请参见[MI_VENC_SetH264Db1k](#) [#245-mi_venc_seth264db1k]的举例。

2.48. MI_VENC_GetH265Db1k

- 功能

获取H.265协议编码通道的db1k类型。
- 语法

```
MI_S32 MI_VENC_GetH265Db1k(MI_VENC_CHN VeChn,MI_VENC_ParamH265Db1k_t *pstH265Db1k);
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstH265Dbk	H.265协议编码通道的dbk属性	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.265协议编码通道的dbk配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见MI_VENC_SetH264Dbk的举例。

2.49. MI_VENC_SetH264Vui

- 功能

设置H.264协议编码通道的vui参数。
- 语法

```
MI_S32 MI_VENC_SetH264Vui(MI_VENC_CHN VeChn, MI_VENC_ParamH264Vui_t *pstH264Vui);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstH264Vui	H.264协议编码通道的Vui参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.264协议编码通道的VUI的配置
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。

- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用MI_VENC_GetH264Vui [#250-mi_venc_geth264vui]接口，获取当前编码通道的Vui配置，然后再进行设置。
- 举例

```
MI_S32 VencSetDb1k()
{
    MI_S32 s32Ret;
    MI_VENC_ParamH264Db1k_t stDb1k;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetH264Db1k(VeChnId, &stDb1k);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetH264Db1k err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stDb1k.disable_deblocking_filter_idc = 0;
    stDb1k.slice_alpha_c0_offset_div2 = 6;
    stDb1k.slice_beta_offset_div2 = 5;
    s32Ret = MI_VENC_SetH264Db1k(VeChnId,&stDb1k);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetH264Db1k err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    return s32Ret;
}
```

2.50. MI_VENC_GetH264Vui

- 功能
获取H.264协议编码通道的Vui配置。
- 语法

```
MI_S32 MI_VENC_GetH264Vui(MI_VENC_CHN VeChn,MI_VENC_ParamH264Vui_t *pstH264Vui);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH264Vui	H.264协议编码通道的Vui属性	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.264协议编码通道的Vui配置。

- 本接口可在编码通道创建之后，编码通道销毁之前调用。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例
请参见 [MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]的举例

2.51. MI_VENC_SetH265Vui

- 功能
设置H.265协议编码通道的vui参数。
- 语法

```
MI_S32 MI_VENC_SetH265Vui(MI_VENC_CHN VeChn, MI_VENC_ParamH265Vui_t *pstH265Vui);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265Vui	H.265协议编码通道的Vui参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.265协议编码通道的VUI的配置
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个I帧时生效。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 建议用户在调用此接口之前，先调用[MI_VENC_GetH265Vui](#) [#252-mi_venc_geth265vui]接口，获取当前编码通道的Vui配置，然后再进行设置。
- 举例
请参见 [MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]的举例

2.52. MI_VENC_GetH265Vui

- 功能
获取H.265协议编码通道的Vui配置。
- 语法

```
MI_S32 MI_VENC_GetH265Vui(MI_VENC_CHN VeChn MI_VENC_ParamH265Vui_t *pstH265Vui);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstH265Vui	H.265协议编码通道的Vui属性	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.265协议编码通道的Vui配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

请参见 [MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]的举例

2.53. MI_VENC_SetH265SliceSplit

- 功能

设置H.265通道的slice分割属性。
- 语法

```
MI_S32 MI_VENC_SetH265SliceSplit(MI_VENC_CHN VeChn,MI_VENC_ParamH265SliceSplit_t *pstSliceSplit);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstSliceSplit	H.265码流slice分割参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置H.265协议编码通道码流的分割方式
 - Slice分割属性主要由两个参数决定：

- a. **bSplitEnable**: 当前帧是否进行**slice**分割。
- b. **u32SliceRowCount**: **slice**按照宏块行进行分割。**u32SliceRowCount**表示每个**slice**占图像宏块行数。且当编码至图像的最后几行，不足**u32SliceRowCount**时，剩余的宏块行被划分为一个**slice**。
- 本接口属于高级接口，用户可以选择性调用，建议不调用，系统默认**bSplitEnable**为**false**。本接口可在编码通道创建之后，编码通道销毁之前设置。此接口在编码过程中被调用时，等到下一帧时生效。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数
- 建议用户在调用此接口之前，先调用**MI_VENC_GetH265SliceSplit** [#254-mi_venc_geth265slicesplit]接口，获取当前通道的**slicesplit**配置，然后再进行设置。
- 举例
请参见**MI_VENC_SetH264SliceSplit** [#229-mi_venc_seth264slicesplit]的举例。

2.54. MI_VENC_GetH265SliceSplit

- 功能
获取H.265通道的**slice**分割属性。
- 语法

```
MI_S32 MI_VENC_GetH265SliceSplit(MI_VENC_CHN VeChn,MI_VENC_ParamH265SliceSplit_t *pstSliceSplit);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstSliceSplit	H.265码流 slice 分割参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照**错误码** [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取H.265协议编码通道的**slice**分割属性。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例
请参见**MI_VENC_SetH264SliceSplit** [#229-mi_venc_seth264slicesplit]的举例。

2.55. MI_VENC_SetJpegParam

- 功能
设置JPEG协议编码通道的高级参数。
- 语法

```
MI_S32 MI_VENC_SetJpegParam(MI_VENC_CHN VeChn, MI_VENC_ParamJpeg_t *pstJpegParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstJpegParam	JPEG协议编码通道的高级参数集合	输入

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

- 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

- 注意

- 本接口用于设置JPEG协议编码通道的高级参数。
- 高级参数主要由4个参数组成：
 - a. u32Qfactor: 量化表因子范围为[1,99]，u32Qfactor越大，量化表中的量化系数越小，得到的图像质量会更好，同时，编码压缩率更低。同理u32Qfactor越小，量化表中的量化系数越大，得到的图像质量会更差，同时，编码压缩率更高。具体的u32Qfactor与量化表的关系请见RFC2435标准。
 - b. au8YQt [64], au8CbCrQt [64]: 这两个参数对应两个量化表空间，用户可以通过这两个参数设置用户的量化表。
 - c. u32McuPerEcs: 每个Ecs中包含多少Mcu。系统模式u32MCUPerECS=0，表示当前帧的所有的MCU被编码为一个ECS。u32MCUPerECS的最小值为0，最大值不超过 $(picwidth+15)>>4 \times (picheight+15)>>4 \times 2$ 。
- 如果用户想使用自己的量化表，在设置量化表的同时，请将Qfactor设置为50。
- 本接口可在编码通道创建之后，编码通道销毁之前设置。本接口在编码过程中被调用时，等到下一个帧编码时生效。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 建议用户在调用此接口之前，先调用[MI_VENC_GetJpegParam](#) [#256-mi_venc_getjpegparam]接口，获取当前编码通道的JpegParam配置，然后再进行设置。

- 举例

```
MI_S32 SetJpegParam()
{
    MI_S32 s32Ret;
    MI_VENC_ParamJpeg_t stParamJpeg;
    MI_VENC_CHN VeChnId = 0;
    int i;

    //...omit other thing
    s32Ret = MI_VENC_GetJpegParam(VeChnId, &stParamJpeg);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetJpegParam err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    stParamJpeg.u32MCUPerEcs = 100;
    for (i = 0; i < 64; i++)
    {
        stParamJpeg.au8YQt[i] = 16;
        stParamJpeg.au8CbCrQt[i] = 17;
    }
}
```

```
s32Ret=MI_VENC_SetJpegParam(VeChnId, &stParamJpeg);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VENC_SetJpegParam err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}

return s32Ret;
}
```

2.56. MI_VENC_GetJpegParam

- 功能
获取JPEG协议编码通道的高级参数配置。
- 语法

```
MI_S32 MI_VENC_GetJpegParam(MI_VENC_CHN VeChn, MI_VENC_ParamJpeg_t *pstJpegParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)。	输入
pstJpegParam	Jpeg协议编码通道的高级参数配置。	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取JPEG协议编码通道的高级参数配置。
 - 本接口可在编码通道创建之后，编码通道销毁之前调用。
 - 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例
请参见[MI_VENC_SetJpegParam](#) [#255-mi_venc_setjpegparam]]的举例。

2.57. MI_VENC_SetRcParam

- 功能
设置编码通道码率控制器的高级参数。
- 语法

```
MI_S32 MI_VENC_SetRcParam(MI_VENC_CHN VeChn, MI_VENC_RcParam_t *pstRcParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRcParam	编码通道码率控制器的高级参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 芯片差异

不同Chip和不同编码协议支持的码控算法是不一样的，如下表：

Chip	H.264	H.265	JPEG
Pretzel	FIXQP/CBR/VBR	FIXQP/CBR/VBR	FIXQP/CBR
Macaron	FIXQP/CBR/VBR	FIXQP/CBR/VBR	FIXQP/CBR
Pudding	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Ispahan	FIXQP/CBR/VBR/AVBR	FIXQP/CBR/VBR/AVBR	FIXQP/CBR
Ikayaki	None	None	FIXQP/CBR

- 注意
 - 编码通道码率控制器的高级参数都有默认值，而不是必须调用这个接口才能启动编码通道。
 - 建议用户先调用[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]接口，获取RC高级参数，然后修改相应参数，再调用本接口对高级参数进行设置。
 - RC高级参数现仅支持H.264/H.265/Mjpeg码率控制模式。
 - 码率控制器的高级参数由以下参数组成：u32Thrdl[RC_TEXTURE_THR_SIZE]，u32ThrdP[RC_TEXTURE_THR_SIZE]：分别衡量I帧，P帧的宏块复杂度的一组阈值。这组阈值按照从小到大的顺序依次排列，每个阈值的取值范围为[0,255]。这组阈值用于在进行宏块级码率控制时，根据图像复杂度对每个宏块的Qp进行适当的调整。对于H.264，宏块级码率控制只有加方向（最大加12），即如果当前宏块的图像复杂度处于某两阈值之间时，当前宏块的Qp值就在宏块行起始Qp值的基础上加上x。
x取值如下表（C表示图像复杂度）：

C Range	x
$C \leq u32Thrd[0]$	0
$u32Thrd[0] < C \leq u32Thrd[1]$	1
$u32Thrd[1] < C \leq u32Thrd[2]$	2
$u32Thrd[2] < C \leq u32Thrd[3]$	3
$u32Thrd[3] < C \leq u32Thrd[4]$	4
$u32Thrd[4] < C \leq u32Thrd[5]$	5
$u32Thrd[5] < C \leq u32Thrd[6]$	6
$u32Thrd[6] < C \leq u32Thrd[7]$	7
$u32Thrd[7] < C \leq u32Thrd[8]$	8
$u32Thrd[8] < C \leq u32Thrd[9]$	9
$u32Thrd[9] < C \leq u32Thrd[10]$	10
$u32Thrd[10] < C \leq u32Thrd[11]$	11
$u32Thrd[11] < C$	12

对于H.265，宏块级码率控制既有加方向（最大加8），也有减方向（最大减4），即如果当前宏块的图像复杂度小于等于u32Thr[3]阈值时，当前宏块的Qp值就在宏块行起始Qp值的基础上减去x；如果当前宏块的图像复杂度大于u32Thr[3]阈值时，当前宏块的Qp值就在宏块行起始Qp值的基础上加上y，x、y的取值如下表（C表示图像复杂度）：

C Range	x or y
$C < u32Thrd[0]$	x=4
$u32Thrd[0] \leq C < u32Thrd[1]$	x=3
$u32Thrd[1] \leq C < u32Thrd[2]$	x=2
$u32Thrd[2] \leq C < u32Thrd[3]$	x=1
$u32Thrd[3] \leq C \leq u32Thrd[4]$	x=y=0
$u32Thrd[4] < C \leq u32Thrd[5]$	y=1
$u32Thrd[5] < C \leq u32Thrd[6]$	y=2
$u32Thrd[6] < C \leq u32Thrd[7]$	y=3
$u32Thrd[7] < C \leq u32Thrd[8]$	y=4
$u32Thrd[8] < C \leq u32Thrd[9]$	y=5
$u32Thrd[9] < C \leq u32Thrd[10]$	y=6
$u32Thrd[10] < C \leq u32Thrd[11]$	y=7
$u32Thrd[11] < C$	y=8

- **u32RowQpDelta**: 在宏块级码率控制时，每一行宏块的起始QP相对于帧起始QP的波动幅度值。对于码率波动较严格的场景下，可以尝试将此参数调大，实现更加精确的码率控制，但可能会导致某些帧图像内部的图像质量有差异。在高码率时，该值推荐为0；中码率时推荐该值为0或1；低码率时推荐该值为2~5。
- CBR参数如下：
 - a. **u32MinIprop&u32MaxIprop**: CBR高级参数，分别表示的是最小IP比例和最大IP比例。IP比例表示I帧和P帧的Bits数的比例。这两个值用于钳位IP比例的范围。当**u32MinIprop**被调整较大时，会导致I帧清晰，P帧模糊。当**u32MaxIprop**被调整较小时，会导致I帧模糊，P帧清晰。在正常情况下不建议对IP大小比进行约束，避免带来呼吸效应和码率波动，默认**u32MinIprop**设为1，**u32MaxIprop**设为20。在对I帧大小有约束的场景时，可以根据对I帧大小波动的依赖来设置**u32MinIprop**和**u32MaxIprop**的值。
 - b. **u32MaxQp&u32MinQp&u32MaxStartQp**: CBR高级参数，**u32MaxQp**、**u32MinQp**表示的是当前帧的最大QP和最小QP。这个钳位效果最强烈，所有其他对图像QP的调整，如宏块级码率控制，最终都会被约束到这个最大QP和最小QP。**u32MaxStartQp**表示的是帧级码率控制输出的最大QP，取值范围是[**u32MinQp**,**u32MaxQp**]。宏块级码率控制可能会使某些宏块的QP大于或小于**u32MaxStartQp**，但都会被钳位到[**u32MinQp**,**u32MaxQp**]。默认值**u32MinQp**为10，**u32MaxQp**为51，**u32MaxStartQp**为51。在对质量无特殊需求下，建议不更改此组参数。
 - c. **u32MaxPPDeltaQp&u32MaxIPDeltaQp**: CBR高级参数，表示的是连续两个P帧起始Qp之间的最大差值和连续IP帧起始QP之间的最大差值。当出现大运动，场景切换等变化时，保证码率平稳条件下，可能会导致连续P帧之间QP差异过大，导致图像马赛克等，可以通过调整这两个参数对QP变化进行钳位，避免出现图像质量波动。**u32MaxPPDelta**默认值为3；**u32MaxIPDeltaQP**默认值为5，可以根据场景对质量与码率要求的差异修改。
 - d. **s32IPQPDelta**: CBR高级参数，表示的是平均QP值与当前I帧QP的差值，此参数可为负值。可用于调整I帧过大和呼吸效应。系统默认值为2，增大此值，I帧变清晰。**u32RQRatio[8]**: CBR高级参数，表示的是码率稳定和质量稳定在进行码率控制中的权重。**u32RQRatio[i]/100**表示质量稳定权重，**1-u32RQRatio[i]/100**表示码率稳定权重，譬如：**u32RQRatio[i]=75**，表示质量稳定占75%的权重，码率稳定占25%的权重。CBR提供了8种场景模式，分别是：**Normal**(正常场景),**Move**(运动场景),**Still**(静止场景),**StillToMove**(静止转为运动场景),**MoveToStill**(运动转为静止场景),**SceneSwitch**(场景切换),**SharpMove**(剧烈运动场景),**Init**(程序初始化)，分别对应了**u32RQRatio**中0~7的索引号。现在系统默认值为**u32RQRatio[]={75,75,75,50,50,20,30,0}**。用户可以设置不同场景下，码率控制的权重，以满足特定的依赖，譬如：在大运动场景下，质量稳定的比例为30%，码率稳定的比例为70%，用户可以将质量比例继续调小，码率的波动变的更小，保留，暂不使用。
- VBR 参数如下：
 - a. **s32DeltaQP**: VBR高级参数，表示的是在VBR质量出现波动的过程中，帧与帧之间的最大的QP变化。这个参数可用于防止出现马赛克。系统默认为2。
 - b. **s32ChangePos**: VBR高级参数，表示的是VBR开始调整QP时码率与最大码率的比值。系统默认为90，如果在内容变化剧烈且要求不超出最大码率的场景，建议减小此值，同时增大**s32DeltaQP**，但码率控制稳定时的码率偏小和质量偏差。
- AVBR 参数如下：
 - a. **s32ChangePos**: AVBR高级参数，表示的是AVBR开始调整Qp时码率与最大码率的比值。系统默认为80，如果在内容变化剧烈且要求不超出最大码率的场景，建议减小此值，同时增大**s32DeltaQP**，但码率控制稳定时的码率偏小和质量偏差。
 - b. **u32MaxQp&u32MinQp**: AVBR高级参数，**u32MaxQp**、**u32MinQp**表示的是当前帧的最大Qp和最小Qp。这个钳位效果最强烈，所有其他对图像Qp的调整，如宏块级码率控制，最终都会被约束到这个最大Qp和最小Qp。默认值**u32MinQp**为12，**u32MaxQp**为48在对质量无特殊需求下，建议不更改此组参数。
 - c. **u32MaxIQp&u32MinIQp**: AVBR高级参数，**u32MaxIQp**、**u32MinIQp**表示的是当前序列IDR帧的最大Qp和最小Qp。这个钳位效果最强烈，所有其他对图像Qp的调整，如宏块级码率控制，最终都会被约束到这个最大Qp和最小Qp。在对质量无特殊需求下，建议不更改此组参数。
 - d. **u32MotionSensitivity**: AVBR高级参数，表示的是运动敏感度，该值越大表示码率控制对画面运动程度的反应越快，但是同时对噪声也更敏感。
- **pRcParam**: 由用户制定的RC高级参数，保留，暂时没有使用。
- 建议用户在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
- 举例

```

MI_S32 VencSetRcParam()
{
    MI_S32 s32Ret;
    MI_VENC_RcParam_t stVencRcPara;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing
    s32Ret = MI_VENC_GetRcParam(VeChnId, &stVencRcPara);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_GetRcParam err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }

    stVencRcPara.stParamH264Cbr.s32IPQPDelta = 2;
    s32Ret = MI_VENC_SetRcParam(VeChnId, &stVencRcPara);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetRcParam err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    //...omit other thing

    return s32Ret;
}

```

2.58. MI_VENC_GetRcParam

- 功能

获取通道码率控制高级参数。

- 语法

```
MI_S32 MI_VENC_GetRcParam(MI_VENC_CHN VeChn, MI_VENC_RcParam_t *pstRcParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRcParam	通道码率控制参数指针	输出

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

- 依赖

- 头文件：mi_common.h, mi_venc.h
- 库文件: libmi.a

- 注意

- 本接口用于获取H.264编码通道码率控制的高级参数。各参数的含义请具体参见MI_VENC_RcParam_t。
- 如果pstRcParam为空，则返回失败。

- 举例

```

MI_S32 VencSetRcParam()
{

```



```
MI_S32 s32Ret;
MI_VENC_RcParam_t stVencRcPara;
MI_VENC_CHN VeChnId = 0;

//...omit other thing
s32Ret = MI_VENC_GetRcParam(VeChnId, &stVencRcPara);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VENC_GetRcParam err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}

stVencRcPara.stParamH264Cbr.s32IPQPDelta = 2;
s32Ret = MI_VENC_SetRcParam(VeChnId, &stVencRcPara);
if (MI_SUCCESS != s32Ret)
{
    printf("MI_VENC_SetRcParam err0x%x\n", s32Ret);
    return E_MI_ERR_FAILED;
}
//...omit other thing

return s32Ret;
}
```

2.59. MI_VENC_SetRefParam

- 功能
设置H.264/H.265编码通道高级跳帧参考参数。
- 语法

```
MI_S32 MI_VENC_SetRefParam(MI_VENC_CHN VeChn,MI_VENC_ParamRef_t *pstRefParam)
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRefParam	H.264/H.265编码通道高级跳帧参考参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果pstRefParam为空，则返回失败。
 - 创建H.264/H.265协议编码通道时，默认跳帧参考模式是1倍跳帧参考模式。如果用户需要修改编码通道的跳帧参考，建议在创建通道之后，启动编码之前调用此接口，减少在编码过程中调用的次数。
 - 本接口在编码过程中被调用时，等到下一个I帧时生效。
 - 如果用户设置1倍跳帧参考模式，对应的配置为：bEnablePred = TRUE，u32Enhance=0，u32Base=1；
 - 如果设置2倍跳帧参考模式，对应的配置为：bEnablePred = TRUE，u32Enhance=1，u32Base=1；
 - 如果设置4倍跳帧参考模式，对应的配置为：bEnablePred = TRUE，u32Enhance=3，u32Base=1。

- 举例

```
MI_S32 VencSetRefParam()
{
    MI_S32 s32Ret;
    MI_VENC_ParamRef_t stRefParam;
    MI_VENC_CHN VeChnId = 0;

    //...omit other thing

    stRefParam.u32Base = 1;
    stRefParam.u32Enhance = 3;
    stRefParam.bEnablePred = TRUE;
    s32Ret = MI_VENC_SetRefParam(VeChn, &stRefParam);
    if (MI_SUCCESS != s32Ret)
    {
        printf("MI_VENC_SetRefParam err0x%x\n", s32Ret);
        return E_MI_ERR_FAILED;
    }
    //...omit other thing

    return s32Ret;
}
```

2.60. MI_VENC_GetRefParam

- 功能
获取H.264/H.265编码通道高级跳帧参考参数。
- 语法

```
MI_S32 MI_VENC_GetRefParam(MI_VENC_CHN VeChn, MI_VENC_ParamRef_t *pstRefParam)
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstRefParam	H.264/H.265编码通道高级跳帧参考参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
如果pstRefParam为空，则返回失败。

2.61. MI_VENC_SetCrop

- 功能
设置通道的裁剪属性。

- 语法

MI_S32 MI_VENC_SetCrop (VENC_CHN VeChn, MI_VENC_CropCfg_t *pstCropCfg)

- 参数

参数名称	描述	输入/输出
VeChn	通道号	输入
pstCropCfg	通道裁剪属性	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 芯片差异

不同Chip支持的Crop对齐规则如下表所示：

Chip	H.264	H.265	JPEG
	CropWidth x CropHeight : CropLeft x CropTop	CropWidth x CropHeight : CropLeft x CropTop	CropWidth x CropHeight : CropLeft x CropTop
Pretzel	8x2:16x16	8x2:16x16	8x2:256x16
Macaron	8x2:16x16	8x2:16x16	8x2:256x16
Pudding	8x2:16x16	8x2:16x16	8x2:256x16
Ispahan	8x2:16x16	8x2:16x16	8x2:256x16
Ikayaki	None	None	8x2:256x16

- 注意
 - 本接口用于设置通道的裁剪属性。
 - 本接口必须在通道创建之后，通道销毁前调用。
 - 裁剪属性由两部分组成：
 - a. bEnable: 是否使能通道裁剪功能。
 - b. stRect: 裁剪区域属性，包括裁剪区域起始点坐标，以及裁剪区域的尺寸。

2.62. MI_VENC_GetCrop

- 功能
 - 获取通道的裁剪属性。
- 语法

MI_S32 MI_VENC_GetCrop (MI_VENC_CHN VeChn, MI_VENC_CropCfg_t *pstCropCfg)

• 参数

参数名称	描述	输入/输出
VeChn	通道号	输入
pstCropCfg	通道裁剪属性	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取通道的裁剪属性。
 - 本接口必须在通道创建之后，通道销毁之前调用。

2.63. MI_VENC_SetFrameLostStrategy

- 功能
设置编码通道瞬时码率超过阈值时丢帧策略。
- 语法

```
MI_S32 MI_VENC_SetFrameLostStrategy (MI_VENC_CHN VeChn,MI_VENC_ParamFrameLost_t *pstFrmLostParam)
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0, VENC_MAX_CHN_NUM)	输入
pstFrmLostParam	编码通道丢帧策略的参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件 : mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果pstFrmLostParam为空，则返回失败。
 - pstFrmLostParam 主要由四个参数决定:
 - a. enFrmLostMode: 丢帧策略模式。
 - b. u32EncFrmGaps: 丢帧间隔。
 - c. bFrmLostOpen: 丢帧开关。

d. u32FrmLostBpsThr: 丢帧阈值。

- 本接口属于高级接口，用户可以选择性调用，系统有默认值，默认在瞬时码率超出阈值时为丢帧。
- 本接口提供瞬时码率超过阈值时两种处理方式：丢帧和编码pskip帧。u32EncFrmGaps控制是否均匀丢帧或均匀编码pskip帧。如下图所示：

 -> Frames whose instantaneous bit rate does not exceed the threshold during editing and are retained

 -> Frames whose instantaneous bit rate exceeds the threshold during editing and discarded

 -> When editing, the instantaneous bit rate exceeds the threshold, and the encoding is pskip frame

enFrmLostMode=E_MI_VENC_FRMLOST_NORMAL, u32EncFrmGaps=0



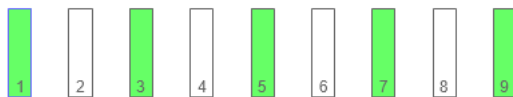
enFrmLostMode=E_MI_VENC_FRMLOST_NORMAL, u32EncFrmGaps=1



enFrmLostMode=E_MI_VENC_FRMLOST_PSKIP, u32EncFrmGaps=0



enFrmLostMode=E_MI_VENC_FRMLOST_PSKIP, u32EncFrmGaps=1



- 本接口可在编码通道创建之后，编码通道销毁之前设置。此接口在编码过程中被调用时，等到下一帧时生效。

2.64. MI_VENC_GetFrameLostStrategy

- 功能

获取编码通道瞬时码率超过阈值时丢帧策略。

- 语法

```
MI_S32 MI_VENC_GetFrameLostStrategy(MI_VENC_CHN VeChn,MI_VENC_ParamFrameLost_t*pstFrmLostParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstFrmLostParam	编码通道丢帧策略的参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意

如果pstFrmLostParam为空，则返回失败。

2.65. MI_VENC_SetSuperFrameCfg

- 功能

设置编码超大帧配置。
- 语法

```
MI_S32 MI_VENC_SetSuperFrameCfg(MI_VENC_CHN VeChn, MI_VENC_SuperFrameCfg_t *pstSuperFrmParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstSuperFrmParam	编码超大帧配置参数	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_comm_rc.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建，则返回失败。
 - 本接口属于高级接口，用户可以选择性调用，系统默认值。系统默认值为eSuperFrmMode=E_MI_VENC_SUPERFRM_NONE。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置。
 - 超大帧模式选择E_MI_VENC_SUPERFRM_NONE时，MI不做任何处理。
 - 超大帧模式选择E_MI_VENC_SUPERFRM_DISCARD时，MI会将超大帧丢弃，并继续编下一帧；如果是Macaron,Isphahan 等版本，因为内存不足的关系，丢弃超大帧后会强制申请1帧。

- 超大帧模式选择E_MI_VENC_SUPERFRM_REENCODE时，MI会将这一帧QP提高4并重新编码，最多重复4次，如果还是超出阈值，就把这一帧丢弃。
- 超大帧处理主要是防止因为某一帧过大造成的画面卡顿。阈值单位是bit，建议根据实际场景设定。

2.66. MI_VENC_GetSuperFrameCfg

- 功能
获取编码超大帧配置。
- 语法

```
MI_S32 MI_VENC_GetSuperFrameCfg(MI_VENC_CHN VeChn,MI_VENC_SuperFrameCfg_t *pstSuperFrmParam);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstSuperFrmParam	编码超大帧配置参数	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
如果pstSuperFrmParam为空，则返回失败。

2.67. MI_VENC_SetRcPriority

- 功能
设置码率控制的优先级类型。
- 语法

```
MI_S32 MI_VENC_SetRcPriority(MI_VENC_CHN VeChn,MI_VENC_RcPriority_e eRcPriority);
```

- 参数

参数名称	描述	输入/输出
VeChn	通道号	输入
enRcPriority	优先级类型枚举	输入

- 返回值
 - MI_OK: 成功

- Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于设置码率控制是以目标码率还是以超大帧阈值为高优先级。
 - 当码率控制以目标码率为高优先级时，在码率不足时可能编码出超大帧以补偿码率，这时超大帧不会重编。当码率控制以超大帧阈值为高优先级时，超大帧则会重编以降低比特数，结果可能导致码率不足。
 - 本接口必须在编码通道创建之后，编码通道销毁之前调用。
 - 本接口只支持H.264和H.265两种协议的码率控制模式。

2.68. MI_VENC_GetRcPriority

- 功能
获取码率控制的优先级类型。
- 语法

```
MI_S32 MI_VENC_GetRcPriority(MI_VENC_CHN VeChn,MI_VENC_RcPriority_e *peRcPriority);
```

- 参数

参数名称	描述	输入/输出
VeChn	通道号	输入
penRcPriority	优先级类型指针	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
本接口必须在编码通道创建之后，编码通道销毁之前调用。

2.69. MI_VENC_AllocCustomMap

- 功能
分配智能编码所用的Custom Map的内存，Custom Map包含 **QP Map** 和 **Mode Map**。
- 语法

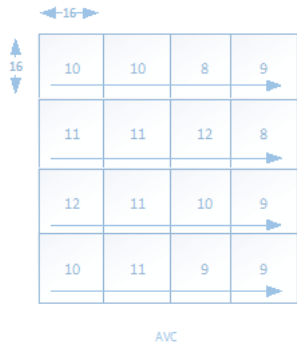
```
MI_S32 MI_VENC_ AllocCustomMap (MI_VENC_CHN VeChn, MI_PHY *pstPhyAddr, MI_VOID **ppCpuAddr);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstPhyAddr	分配的物理地址的指针	输入/输出
ppCpuAddr	分配的CPU虚拟地址的指针	输入/输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于分配Custom Map所需的内存，并返回分配好的内存的地址，包括物理地址和CPU地址，同时返回图像帧的大小。
 - 如果地址分配失败，pstPhyAddr或ppCpuAddr为空，则返回失败。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置，分配的内存会在通道销毁时候同步释放。
 - Custom Map包含QP Map和Mode Map两种，可以同时使用两种Map,不同的Map使用不同的位来区分。
 - QPMap中QP的设置是在H.264中以16*16块为单位，每一个16*16的QP值，采用用户设定的相应块的QP值。所有这些块的QP值组成QP表，该表中QP值的组织方式如下图示。QP值依次填入为QPMap所分配的内存地址中。QP值根据设定（参考[MI_VENC_SetAdvCustRcAttr](#) [#273-mi_venc_setadvcustrcattr]）使用相对或绝对值填入，如果是绝对QP值，QP的取值范围为[12,48]。如果是相对QP值，QP的取值范围为[0,63]（实际生效的相对QP值为配置值减32，即实际生效的范围为[-32，31]）。
 - Mode Map中Mode的设置是在H.264中同样以16*16块为单位，每一个16*16的Mode值(0:not use,1:skip mode,2:intra mode)，采用用户设定的相应块的Mode值。所有这些块的Mode值组成Mode表，该表中Mode值的组织方式和QP值的组织方式一样，Mode值依次填入为ModeMap所分配的内存地址中。
 - 每个16*16块可以配置1 byte的Mode/QP值，Mode值占据所填写值的高2位，QP值占据低6位。配置填写示例如下：

```
struct {  
    uint8 qp : 6;  
    uint8 mode : 2;  
} H264CustomMapConf;
```

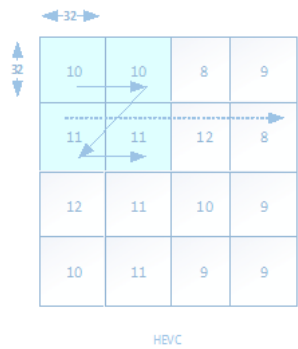


- QPMap中QP的设置是在H.265中以4个32*32块为单位，每一个32*32块的QP值，采用用户设定的相应块的QP值。所有这些块的QP值组成QP表，该表中QP值的组织方式如下图示，QP值按照上下两个的次序依次填入为QPMap所分配

的内存地址中。QP值同样根据设定使用相对或绝对值填入。

- ModeMap中Mode的设置与H.265中同样以4个32*32块为单位，每4个32*32块的Mode值(0:not use,1:skip mode,2:intra mode)，采用用户设定的相应块的Mode值。所有这些块的Mode值组成Mode表，该表中Mode值的组织方式和QP值的组织方式一样，Mode值依次填入为ModeMap所分配的内存地址中。
- 以上4个32*32块中，可以配置4bytes的QP/Mode值。其中Mode值占2位，每个32*32的块对应的QP值占6位（共24位）。配置填写示例如下：

```
struct {  
  
    uint32 sub_ctu_qp_0      : 6;  
  
    uint32 mode      : 2;  
  
    uint32 sub_ctu_qp_1      : 6;  
  
    uint32 reserved_0 : 2;  
  
    uint32 sub_ctu_qp_2      : 6;  
  
    uint32 reserved_1      : 2;  
  
    uint32 sub_ctu_qp_3      : 6;  
  
    uint32 reserved_2      : 2;  
  
} H265CustomMapConf;
```



2.70. MI_VENC_ApplyCustomMap

- 功能
应用已经配置的Custom Map。
- 语法

```
MI_S32 MI_VENC_ApplyCustomMap (MI_VENC_CHN VeChn, MI_PHY PhyAddr);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号	输入
	取值范围：[0,VENC_MAX_CHN_NUM)	
PhyAddr	分配的虚拟物理地址	输入

- 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于在配置自定义的Custom Map到内存后，应用相关的配置。
 - 本接口可在初始化或修改Custom Map后使用。

2.71. MI_VENC_GetLastHistoStaticInfo

- 功能
获取最新的图像帧编码后的相关输出信息。
- 语法

```
MI_S32 MI_VENC_GetLastHistoStaticInfo (MI_VENC_CHN VeChn, MI_VENC_FrameHistoStaticInfo_t
**ppFrmHistoStaticInfo);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
ppFrmHistoStaticInfo	保存的最新已编码帧相关输出的地址的指针	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 本接口用于获取保存的最近已经编码的图像帧输出相关信息的地址，其中输出信息包括编码后的帧类型，块的数量，帧大小等等。
 - 如果获取相关输出内容失败，则返回失败。
 - 本接口可在编码通道创建之后，编码通道销毁之前设置，获取的相关输出内容会在通道销毁时候或者调用下面的释放接口时候释放。

2.72. MI_VENC_ReleaseHistoStaticInfo

- 功能
释放保存的最新的图像帧编码后的相关输出信息占用的内存。
- 语法

```
MI_S32 MI_VENC_ReleaseHistoStaticInfo (MI_VENC_CHN VeChn);
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入

• 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

• 依赖

- 头文件: mi_common.h, mi_venc.h
- 库文件: libmi.a

• 注意

本接口用于释放之前保存的最近的已经编码的图像帧的输出相关信息占用的内存。

2.73. MI_VENC_SetAdvCustRcAttr

• 功能

设置自定义的高级码控相关属性配置。

• 语法

```
MI_S32 MI_VENC_SetAdvCustRcAttr (MI_VENC_CHN Vehn,MI_VENC_AdvCustRcAttr_t sAdvCustRcAttr);
```

• 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
sAdvCustRcAttr	自定义的高级码控相关功能的开关参数配置	输入

• 返回值

- MI_OK: 成功
- Not MI_OK: 失败, 参照[错误码](#) [#jump]

• 依赖

- 头文件: mi_common.h, mi_venc.h
- 库文件: libmi.a

• 注意

- 本接口用于设置自定义的高级码控相关属性配置。
- 本接口在编码通道创建之后，图像开始接收之前设置。
- 设置了自定义的高级码控功能的开关后，才能使用[MI_VENC_AllocCustomMap](#) [#269-mi_venc_alloccustommap]和[MI_VENC_GetLastHistoStaticInfo](#)来启动相关功能。

2.74. MI_VENC_SetInputSourceConfig

- 功能
设置H.264/H.265的输入配置信息。
- 语法

```
MI_S32 MI_VENC_SetInputSourceConfig(MI_VENC_CHN VeChn,MI_VENC_InputSourceConfig_t
*pstInputSourceConfig);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstInputSourceConfig	输入配置信息	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件：mi_common.h, mi_venc.h
 - 库文件: libmi.a

- 注意
不同Chip在使用Input ring mode或者hw auto sync mode的时候，需不需要调用此接口的限制是不一样的，如下表：

Chip	使用ring mode/hw auto sync mode时需要调用此接口？
Pretzel	N
Macaron	N
Pudding	N
Ispahan	Y
Ikayaki	Y

- 如果通道未创建，则返回失败。
- 本接口在编码通道创建之后，图像开始接收之前设置。
- 允许对最多一个通道设置成input ring mode或者input ring hw auto sync mode。
- 如果设置了E_MI_VENC_INPUT_MODE_RING_ONE_FRM/E_MI_VENC_INPUT_MODE_RING_HALF_FRM/E_MI_VENC_INPUT_MODE_那APP在调用MI_SYS_BindChnPort2需要设置E_MI_SYS_BIND_TYPE_HW_RING和相应ring buffer高度。比如分辨率为1920x1080，如果设置为E_MI_VENC_INPUT_MODE_RING_ONE_FRM，则ring buffer高度需设置1080;如果为E_MI_VENC_INPUT_MODE_RING_HALF_FRM，则ring buffer高度需设置540。

2.75. MI_VENC_SetSmartDetInfo

- 功能

用于第三方智能侦测算法向VENC提供智能编码所需的统计信息。

- 语法

```
MI_S32 MI_VENC_SetSmartDetInfo(MI_VENC_CHN VeChn, MI_VENC_SmartDetInfo_t *pstSmartDetInfo);
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstSmartDetInfo	智能侦测算法相关的统计信息	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 如果通道未创建时，则返回失败。
 - 本接口在编码通道创建之后，且图像开始接收之后设置。
 - 本接口参数设置只在Pudding、Ispahan生效。

2.76. MI_VENC_SetIntraRefresh

- 功能

设置H.264/H.265的P帧刷新Islice，开启IntraRefresh模式后，只会在编码开始位置有一张IDR，后续只有P帧，开启IntraRefresh模式的好处是不改变IDR帧质量，把原本IDR编码的Intra LCU/宏块分散在若干个P帧中，使每帧的大小相对平均。

优点：- 码率非常平稳，对网络冲击小，适合无线传输环境。- 编码，解码及网络延迟非常小 - 不会降低I帧质量，不会引起严重呼吸效应

缺点：

- 只支持normalP的GOP结构，其他GOP结构不支持
 - 该技术主要针对码率平稳要求高的场景，并不能够降低太多码率，对低码率场景优化不大，仅支持宏块/LCU按行刷新
- 语法

```
MI_S32 MI_VENC_SetIntraRefresh(MI_VENC_CHN VeChn, MI_VENC_IntraRefresh_t *pstIntraAttr)
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstIntraAttr	输入Intra配置信息	输入

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 目前h264 和h265 support intra refresh 的chip如下

Chip	是否支持intra refresh
Pudding	Y
Ispahan	Y

- 如果通道未创建，则返回失败。
 - 本接口在编码通道创建之后，图像开始接收之前设置。
- 举例
 - 参考mi_demo里面st_main_venc.c
 - 注意uRefreshLineNum是取值范围是：IntraRefreshLine * Gop > MbH 例如：1080P gop 30 mb_height:16，那取值范围是 1088 / 16 = 68 个MB行 68/gop = 2.26，取值要大 2<=68 为了有降低码率，u32RefreshLineNum 取值建议不要取值太大，不然后回导致整个gop的P frame都刷到Islice.
- 参考代码

```
MI_VENC_IntraRefresh_t stIntraAttr;

stIntraAttr.bEnable = conf->gdr.enable;

stIntraAttr.u32RefreshLineNum = conf->gdr.refreshmaxNum;

stIntraAttr.u32ReqIQp = conf->gdr.reqst_IQP;

s32Ret = MI_VENC_SetIntraRefresh(conf->venc_chn, &stIntraAttr);
```

2.77. MI_VENC_GetIntraRefresh

- 功能
获取H.264/H.265的Intra Refresh参数
- 语法

```
MI_S32 MI_VENC_GetIntraRefresh(MI_VENC_CHN VeChn, MI_VENC_IntraRefresh_t *pstIntraAttr)
```

- 参数

参数名称	描述	输入/输出
VeChn	编码通道号 取值范围：[0,VENC_MAX_CHN_NUM)	输入
pstIntraAttr	输出Intra配置信息	输出

- 返回值
 - MI_OK: 成功
 - Not MI_OK: 失败, 参照[错误码](#) [#jump]
- 依赖
 - 头文件: mi_common.h, mi_venc.h
 - 库文件: libmi.a
- 注意
 - 目前h264 和h265 encoder support intra refresh 的chip如下

Chip	是否支持intra refresh
Pudding	Y
Ispahan	Y

- 如果通道未创建，则返回失败。
- 本接口在编码通道创建之后，图像开始接收之前设置。

2.78. MI_VENC_InitDev

- 描述

初始化venc设备。
- 语法

```
MI_S32 MI_VENC_InitDev(MI_VENC_InitParam_t *pstInitParam);
```

- 参数

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值
 - MI_OK成功。
 - 非MI_OK失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_common.h、mi_venc.h
 - 库文件：libmi.a
- 注意
 - 此函数必须在[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]前调用，否则返回失败。

- 如果本接口在MI_VENC_CreateChn [#25-mi_venc_createchn]之前没有调用，内部会使用默认的参数初始化设备。
- 本接口必须和MI_VENC_DeInitDev成对使用，不可单独重复调用，否则返回失败。
- 本接口参数设置只在Pudding、Ispahan生效。

2.79. MI_VENC_DeInitDev

- 描述
反初始化venc设备。
- 语法
MI_S32 MI_VENC_DeInitDev(void);
- 返回值
 - MI_OK成功。
 - 非MI_OK失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_common.h、mi_venc.h
 - 库文件：libmi.a
- 注意
 - 此函数必须在初始化设备后调用，否则返回失败。
 - 如果本接口在app退出前没有调用，内部会自动反初始化设备。
 - 本接口必须和MI_VENC_InitDev [#278-mi_venc_initdev]成对使用，不可单独重复调用，否则返回失败。
 - 本接口参数设置只在Pudding 和Ispahan生效。

2.80. MI_VENC_DupChn

- 描述
同步VENC通道状态。
- 语法

MI_S32 MI_VENC_DupChn(MI_VENC_CHN VeChn)

- 参数

参数名称	描述	输入/输出
VeChn	通道号	输入

- 返回值
 - MI_OK成功。
 - 非MI_OK失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件: mi_common.h mi_venc.h
 - 库文件: libmi_venc.a/libmi_venc.so
- 注意
 - 此接口仅适用于dual os环境。在RTK环境下已初始化VENC通道，切换到Linux环境时，用于同步VENC通道的状态。

- Linux dup chn后 · RTK端app不可以再操作该VENC通道。

3. VENC 数据类型

3.1. 数据类型定义

相关数据类型、数据结构定义如下表。

数据结构	定义
RC_TEXTURE_THR_SIZE [#32-rc_texture_thr_size]	定义纹理级码控的阈值个数
MI_VENC_H264eNaluType_e [#33-mi_venc_h264enalutype_e]	定义H.264码流NALU类型
MI_VENC_H264eRefSliceType_e [#34-mi_venc_h264erefslicetype_e]	定义获取的H.264码流属于何种跳帧参考模式下的参考帧
MI_VENC_H264eRefType_e [#35-mi_venc_h264ereftype_e]	定义H.264跳帧参考码流的帧类型以及参考属性
MI_VENC_JpegePackType_e [#36-mi_venc_jpegepacktype_e]	定义JPEG码流的PACK类型
MI_VENC_H265eNaluType_e [#37-mi_venc_h265enalutype_e]	定义H.265码流NALU类型
MI_VENC_Rect_t [#38-mi_venc_rect_t]	定义编码描述的矩形框
MI_VENC_DataType_t [#39-mi_venc_datatype_t]	定义码流结果联合体
MI_VENC_PackInfo_t [#310-mi_venc_packinfo_t]	定义当前码流包数据中包含的其他类型码流包数据的结构体
MI_VENC_Pack_t [#311-mi_venc_pack_t]	定义帧码流包结构体
MI_VENC_StreamInfoH264_t [#312-mi_venc_streaminfoh264_t]	定义H.264协议码流特征信息
MI_VENC_StreamInfoJpeg_t [#313-mi_venc_streaminfojpeg_t]	定义JPEG协议码流特征信息
MI_VENC_StreamInfoH265_t [#314-mi_venc_streaminfoh265_t]	定义H.265协议码流特征信息
MI_VENC_Stream_t [#315-mi_venc_stream_t]	定义帧码流类型结构体
MI_VENC_StreamBufInfo_t [#316-mi_venc_streambufinfo_t]	定义码流buffer信息的结构体
MI_VENC_AttrH264_t [#317-mi_venc_attrh264_t]	定义H.264编码器属性结构体
MI_VENC_AttrJpeg_t [#318-mi_venc_attrjpeg_t]	定义JPEG抓拍编码器属性结构体
MI_VENC_AttrH265_t [#319-mi_venc_attrh265_t]	定义H.265编码器属性结构体
MI_VENC_Attr_t [#320-mi_venc_attr_t]	定义编码器属性结构体
MI_VENC_ChnAttr_t [#321-mi_venc_chnattr_t]	定义编码通道属性结构体
MI_VENC_ChnStat_t [#322-mi_venc_chnstat_t]	定义编码通道的状态结构体
MI_VENC_ParamH264SliceSplit_t [#323-mi_venc_paramh264slicesplit_t]	定义H.264编码通道slice分割属性
MI_VENC_ParamH265SliceSplit_t [#324-mi_venc_paramh265slicesplit_t]	定义H.265协议编码通道SLICE分割结构体
MI_VENC_ParamH264InterPred_t [#325-mi_venc_paramh264interpreted_t]	定义H.264编码通道帧间预测属性

MI_VENC_ParamH264IntraPred_t [#326-mi_venc_paramh264intrapred_t]	定义H.264协议编码通道帧内预测结构体
MI_VENC_ParamH264Trans_t [#327-mi_venc_paramh264trans_t]	定义H.264编码通道变换、量化属性
MI_VENC_ParamH264Entropy_t [#328-mi_venc_paramh264entropy_t]	定义H.264编码通道熵编码属性
MI_VENC_ParamH265InterPred_t [#329-mi_venc_paramh265interpred_t]	定义H.264编码通道帧间预测属性
MI_VENC_ParamH265IntraPred_t [#330-mi_venc_paramh265intrapred_t]	定义H.264编码通道帧内预测属性
MI_VENC_ParamH265Trans_t [#331-mi_venc_paramh265trans_t]	定义H.264编码通道变换、量化属性
MI_VENC_ParamH264Dbk_t [#332-mi_venc_paramh264dbk_t]	定义H.264编码通道Deblocking属性
MI_VENC_ParamH265Dbk_t [#333-mi_venc_paramh265dbk_t]	定义H.265协议编码通道Dbk结构体
MI_VENC_ParamH264Vui_t [#334-mi_venc_paramh264vui_t]	定义H.264编码通道VUI属性
MI_VENC_ParamH264VuiAspectRatio_t [#335-mi_venc_paramh264vuiaspectratio_t]	定义H.264协议编码通道Vui中AspectRatio信息的结构体
MI_VENC_ParamH264VuiTimelInfo_t [#336-mi_venc_paramh264vuitimeinfo_t]	定义H.264协议编码通道Vui中TIME_INFO信息的结构体
MI_VENC_ParamH264VuiVideoSignal_t [#337-mi_venc_paramh264vuivideosignal_t]	定义H.264协议编码通道Vui中VIDEO_SIGNAL信息的结构体
MI_VENC_ParamH265Vui_t [#338-mi_venc_paramh265vui_t]	定义H.265协议编码通道Vui结构体
MI_VENC_ParamH265VuiAspectRatio_t [#339-mi_venc_paramh265vuiaspectratio_t]	定义H.265协议编码通道Vui中AspectRatio信息的结构体
MI_VENC_ParamH265VuiTimelInfo_t [#340-mi_venc_paramh265vuitimeinfo_t]	定义H.265协议编码通道Vui中TIME_INFO信息的结构体
MI_VENC_ParamH265VuiVideoSignal_t [#341-mi_venc_paramh265vuivideosignal_t]	定义H.265协议编码通道Vui中VIDEO_SIGNAL信息的结构体
MI_VENC_ParamJpeg_t [#342-mi_venc_paramjpeg_t]	定义JPEG编码参数集合
MI_VENC_RoiCfg_t [#343-mi_venc_roicfg_t]	定义编码通道感兴趣区域编码属性
MI_VENC_RoiBgFrameRate_t [#344-mi_venc_roibgframerate_t]	定义非Roi区域的帧率属性
MI_VENC_ParamRef_t [#345-mi_venc_paramref_t]	定义H.264/H.265编码的高级跳帧参考参数
MI_VENC_RcAttr_t [#346-mi_venc_rcattr_t]	定义编码通道码率控制器属性
MI_VENC_RcMode_e [#347-mi_venc_rcmode_e]	定义编码通道码率控制器模式
MI_VENC_AttrH264Cbr_t [#348-mi_venc_attrh264cbr_t]	定义H.264编码通道CBR属性结构
MI_VENC_AttrH264Vbr_t [#349-mi_venc_attrh264vbr_t]	定义H.264编码通道VBR属性结构
MI_VENC_AttrH264FixQp_t [#350-mi_venc_attrh264fixqp_t]	定义H.264编码通道Fixqp属性结构
MI_VENC_AttrH264Abr_t [#351-mi_venc_attrh264abr_t]	定义H.264编码通道ABR属性结构
MI_VENC_AttrH264Avbr_t [#352-mi_venc_attrh264avbr_t]	定义H.264编码通道AVBR属性结构
MI_VENC_AttrMjpegCbr_t [#353-mi_venc_attrmjpegcbr_t]	定义MJPEG编码通道CBR属性结构
MI_VENC_AttrMjpegFixQp_t [#354-mi_venc_attrmjpegfixqp_t]	定义MJPEG编码通道FixQp属性结构
数据结构	定义

MI_VENC_AttrH265Cbr_t [#355-mi_venc_attrh265cbr_t]	定义H.265编码通道CBR属性结构
MI_VENC_AttrH265Vbr_t [#356-mi_venc_attrh265vbr_t]	定义H.265编码通道VBR属性结构
MI_VENC_AttrH265FixQp_t [#357-mi_venc_attrh265fixqp_t]	定义H.265编码通道Fixqp属性结构
MI_VENC_AttrH265Avbr_t [#358-mi_venc_attrh265avbr_t]	定义H.265编码通道AVBR属性结构
MI_VENC_SuperFrmMode_e [#359-mi_venc_superfrmmode_e]	定义码率控制中超大帧处理模式
MI_VENC_ParamH264Vbr_t [#360-mi_venc_paramh264vbr_t]	定义H264协议编码通道VBR码率控制模式高级参数配置
MI_VENC_ParamH264Avbr_t [#361-mi_venc_paramh264avbr_t]	定义H264协议编码通道AVBR码率控制模式高级参数配置
MI_VENC_ParamMjpegCbr_t [#362-mi_venc_parammjpegcbr_t]	定义MJPEG协议编码通道CBR码率控制模式高级参数配置
MI_VENC_ParamH264Cbr_t [#363-mi_venc_paramh264cbr_t]	定义H264协议编码通道CBR新版码率控制模式高级参数配置
MI_VENC_ParamH265Vbr_t [#364-mi_venc_paramh265vbr_t]	定义H265协议编码通道VBR码率控制模式高级参数配置
MI_VENC_ParamH265Avbr_t [#365-mi_venc_paramh265avbr_t]	定义H265协议编码通道AVBR码率控制模式高级参数配置
MI_VENC_ParamH265Cbr_t [#366-mi_venc_paramh265cbr_t]	定义H265协议编码通道CBR新版码率控制模式高级参数配置
MI_VENC_RcParam_t [#367-mi_venc_rcparam_t]	定义编码通道的码率控制高级参数
MI_VENC_CropCfg_t [#368-mi_venc_cropcfg_t]	定义通道截取（Clip）参数
MI_VENC_RecvPicParam_t [#369-mi_venc_recvpicparam_t]	接收指定帧数图像编码
MI_VENC_H264eldrPicIdMode_e [#370-mi_venc_h264eldrpicidmode_e]	设置IDR帧或I帧的idr_pic_id的模式
MI_VENC_H264ldrPicIdCfg_t [#371-mi_venc_h264ldrpicidcfg_t]	IDR帧或I帧的idr_pic_id参数
MI_VENC_FrameLostMode_e [#372-mi_venc_framelostmode_e]	定义编码通道瞬时码率超过阈值时的丢帧模式
MI_VENC_ParamFrameLost_t [#373-mi_venc_paramframelost_t]	定义编码通道瞬时码率超过阈值时的丢帧策略
MI_VENC_SuperFrameCfg_t [#374-mi_venc_superframecfg_t]	超大帧处理策略参数
MI_VENC_RcPriority_e [#375-mi_venc_rcpriority_e]	码率控制优先级枚举
MI_VENC_ModParam_t [#376-mi_venc_modparam_t]	定义编码模块参数
MI_VENC_ModType_e [#377-mi_venc_modtype_e]	定义模块参数类型
MI_VENC_ParamModH265e_t [#378-mi_venc_parammodh265e_t]	定义mi_h265.ko模块参数
MI_VENC_ParamModJpege_t [#379-mi_venc_parammodjpege_t]	定义mi_jpege.ko模块参数
MI_VENC_AdvCustRcAttr_t [#380-mi_venc_advcustrcattr_t]	自定义的高级码控相关功能的开关参数
MI_VENC_FrameHistoStaticInfo_t [#381-mi_venc_framehistostaticinfo_t]	定义图像帧编码的相关配置属性信息
MI_VENC_InputSourceConfig_t [#382-mi_venc_inputsourceconfig_t]	定义H.264/H.265编码通道输入配置参数结构体
MI_VENC_InputSrcBufferMode_e [#383-mi_venc_inputsrcbuffermode_e]	定义H.264/H.265输入buffer模式
VENC_MAX_SAD_RANGE_NUM [#384-venc_max_sad_range_num]	定义智能检测相关之统计SAD值落入区间范围的最大区间个数
数据结构 SmartDetType_e [#385-mi_venc_smartdettype_e]	定义智能侦测类型

MI_VENC_MdInfo_t [#386-mi_venc_mdinfo_t]	定义运动侦测相关统计信息
MI_VENC_SmartDetInfo_t [#387-mi_venc_smartdetinfo_t]	定义智能侦测算法所需的统计信息
MI_VENC_IntraRefresh_t [#388-mi_venc_intrarefresh_t]	支持P帧刷Islice的控制参数
MI_VENC_InitParam_t [#389-mi_venc_initparam_t]	Venc 设备初始化参数

3.2. RC_TEXTURE_THR_SIZE

- 描述
定义RC宏块复杂度的阈值的个数。
- 定义

```
#define RC_TEXTURE_THR_SIZE 1
```

3.3. MI_VENC_H264eNaluType_e

- 描述
定义H.264码流NALU类型。
- 定义

```
typedef enum
{
    E_MI_VENC_H264E_NALU_PSLICE = 1,
    E_MI_VENC_H264E_NALU_ISLICE = 5,
    E_MI_VENC_H264E_NALU_SEI = 6,
    E_MI_VENC_H264E_NALU_SPS = 7,
    E_MI_VENC_H264E_NALU_PPS = 8,
    E_MI_VENC_H264E_NALU_IPSLICE = 9,
    E_MI_VENC_H264E_NALU_PREFIX = 14,
    E_MI_VENC_H264E_NALU_MAX
}MI_VENC_H264eNaluType_e;
```

- 成员

成员名称	描述
E_MI_VENC_H264E_NALU_PSLICE	PSLICE类型
E_MI_VENC_H264E_NALU_ISLICE	ISLICE类型
E_MI_VENC_H264E_NALU_SEI	SEI类型
E_MI_VENC_H264E_NALU_SPS	SPS类型
E_MI_VENC_H264E_NALU_PPS	PPS类型
E_MI_VENC_H264E_NALU_PREFIX	PREFIX类型
E_MI_VENC_H264E_NALU_IPSLICE	P帧刷ISLICE类型（暂不支持）

3.4. MI_VENC_H264eRefSliceType_e

- 描述
定义获取的H.264码流属于何种跳帧参考模式下的参考帧。
- 定义

```
typedef enum
{
    E_MI_VENC_H264E_REFSLICE_FOR_1X = 1,
    E_MI_VENC_H264E_REFSLICE_FOR_2X = 2,
    E_MI_VENC_H264E_REFSLICE_FOR_4X,
    E_MI_VENC_H264E_REFSLICE_FOR_MAX = 5
}MI_VENC_H264eRefSliceType_e;
```

- 成员

成员名称	描述
E_MI_VENC_H264E_REFSLICE_FOR_1X	1倍跳帧参考时的参考帧
E_MI_VENC_H264E_REFSLICE_FOR_2X	2倍跳帧参考时的参考帧或4倍跳帧参考时用于2倍跳帧参考的参考帧
E_MI_VENC_H264E_REFSLICE_FOR_4X	4倍跳帧参考时的参考帧
E_MI_VENC_H264E_REFSLICE_MAX	非参考帧

3.5. MI_VENC_H264eRefType_e

- 描述
定义H.264跳帧参考码流的帧类型以及参考属性。
- 定义

```
typedef enum
{
    E_MI_VENC_BASE_IDR = 0,

    E_MI_VENC_BASE_P_REFTOIDR,

    E_MI_VENC_BASE_P_REFBYBASE,

    E_MI_VENC_BASE_P_REFBYENHANCE,

    E_MI_VENC_ENHANCE_P_REFBYENHANCE,

    E_MI_VENC_ENHANCE_P_NOTFORREF,

    E_MI_VENC_REF_TYPE_MAX

}MI_VENC_H264eRefType_e;
```

- 成员

成员名称	描述
E_MI_VENC_BASE_IDR	base层中的IDR帧
E_MI_VENC_BASE_P_REFTOIDR	Base层中的P帧，用于参考I帧
E_MI_VENC_BASE_P_REFBYBASE	base层中的P帧，用于base层中其他帧的参考
E_MI_VENC_BASE_P_REFBYENHANCE	base层中的P帧，用于enhance层中的帧的参考
E_MI_VENC_ENHANCE_P_REFBYENHANCE	enhance层中的P帧，用于enhance层中其他帧的参考
E_MI_VENC_ENHANCE_P_NOTFORREF	enhance层中的P帧，不用于参考

3.6. MI_VENC_JpegePackType_e

- 描述
定义JPEG码流的PACK类型。
- 定义

```
typedef enum
{
    E_MI_VENC_JPEGE_PACK_ECS = 5,

    E_MI_VENC_JPEGE_PACK_APP = 6,

    E_MI_VENC_JPEGE_PACK_VDO = 7,

    E_MI_VENC_JPEGE_PACK_PIC = 8,

    E_MI_VENC_JPEGE_PACK_MAX

}MI_VENC_JpegePackType_e;
```

- 成员

成员名称	描述
E_MI_VENC_JPEGE_PACK_ECS	ECS 类型
E_MI_VENC_JPEGE_PACK_APP	APP 类型
E_MI_VENC_JPEGE_PACK_VDO	VDO 类型
E_MI_VENC_JPEGE_PACK_PIC	PIC 类型

3.7. MI_VENC_H265eNaluType_e

- 描述
定义H.265码流NALU类型。
- 定义

```
typedef enum
{
    E_MI_VENC_H265E_NALU_PSLICE = 1,
    E_MI_VENC_H265E_NALU_ISLICE = 19,
    E_MI_VENC_H265E_NALU_VPS = 32,
    E_MI_VENC_H265E_NALU_SPS = 33,
    E_MI_VENC_H265E_NALU_PPS = 34,
    E_MI_VENC_H265E_NALU_SEI = 39,
    E_MI_VENC_H265E_NALU_MAX
} MI_VENC_H265eNau1Type_e;
```

- 成员

成员名称	描述
E_MI_VENC_H265E_NALU_PSLICE	PSLICE 类型
E_MI_VENC_H265E_NALU_ISLICE	ISLICE 类型
E_MI_VENC_H265E_NALU_VPS	VPS 类型
E_MI_VENC_H265E_NALU_SPS	SPS 类型
E_MI_VENC_H265E_NALU_PPS	PPS 类型
E_MI_VENC_H265E_NALU_SEI	SEI 类型

3.8. MI_VENC_Rect_t

- 描述
定义编码描述的矩形框。

- 定义

```
typedef struct MI_VENC_Rect_s
{
    MI_U32 u32Left;

    MI_U32 u32Top;

    MI_U32 u32Width;

    MI_U32 u32Height;

}MI_VENC_Rect_t;
```

- 成员

成员名称	描述
u32Left	矩形框左侧与实际画面左侧的距离，单位为pixel
u32Top	矩形框上边缘与实际画面上边缘的距离，单位为pixel
u32Width	矩形框的宽度，单位为pixel
u32Height	矩形框的高度，单位为pixel

- 注意

矩形框不能超出编码实际画面的范围。

- 相关数据类型及接口

```
MI_VENC_SetRoiCfg [#225-mi_venc_setroicfg]
MI_VENC_SetCrop [#261-mi_venc_setcrop]
MI_VENC_SetRoiBgFrameRate [#227-mi_venc_setroibgframerate]
```

3.9. MI_VENC_DataType_t

- 描述

定义码流结果类型。

- 定义

```
typedef union MI_VENC_DataType_s
{
    MI_VENC_H264eNaluType_e eH264EType;

    MI_VENC_JpegePackType_e eJPEGEType;

    MI_VENC_H265eNau1Type_e eH265EType;

}MI_VENC_DataType_t;
```

- 成员

成员名称	描述
enH264EType	H.264码流包类型。
enJPEGEType	JPEG码流包类型。
enH265EType	H.265码流包类型。

- 相关数据类型及接口
[MI_VENC_H264eNaluType_e](#) [#33-mi_venc_h264enalutype_e]
[MI_VENC_JpegePackType_e](#) [#36-mi_venc_jpegepacktype_e]

3.10. MI_VENC_PackInfo_t

- 描述
定义当前码流包数据中包含的其他类型码流包数据的结构体。
- 定义

```
typedef struct MI_VENC_PackInfo_s
{
    MI_VENC_DataType_t stPackType;

    MI_U32 u32PackOffset;

    MI_U32 u32PackLength;

    MI_U32 u32SliceId;
}MI_VENC_PackInfo_t;
```

- 成员

成员名称	描述
u32PackType	当前码流包数据包含其他码流包的类型。
u32PackOffset	当前码流包数据包含其他码流包数据的偏移。
u32PackLength	当前码流包数据包含其他码流包数据的大小。
u32Sliceld	当前码流包数据包含其他码流包的slice id。

3.11. MI_VENC_Pack_t

- 描述
定义帧码流包结构体。
- 定义

```
typedef struct MI_VENC_Pack_s
{
    MI_PHY phyAddr;
```

```
MI_U8 *pu8Addr;

MI_U32 u32Len;

MI_U64 u64PTS;

MI_BOOL bFrameEnd;

MI_VENC_DataType_t stDataType;

MI_U32 u32Offset;

MI_U32 u32DataNum;

MI_U8 u8FrameQP;

MI_S32 s32PocNum;

MI_U32 u32Gradient;

MI_VENC_PackInfo_t asackInfo[8];

} MI_VENC_Pack_t;
```

成员

成员名称	描述
pu8Addr	码流包首地址
phyAddr	码流包物理地址
u32Len	码流包长度
stDataType	码流类型，支持H.264/JPEG/MPEG-4协议类型的数据包。
u64PTS	时间戳。单位：us
bFrameEnd	帧结束标识 取值范围： TRUE：该码流包是该帧的最后一个包。 FALSE：该码流包不是该帧的最后一个包。
u32Offset	码流包中有效数据与码流包首地址pu8Addr的偏移。
u32DataNum	当前码流包（当前包的类型由DataType指定）数据中包含其他类型码流包的个数
u8FrameQP	当前码流包的frame qp。
s32PocNum	当前码流包的poc number。
u32Gradient	当前码流包的梯度信息（用于码率控制）
asackInfo[8]	当前码流包数据中包含其他类型码流包数据信息

注意事项

u8FrameQP和s32PocNum只对H26X的encoder有效，同时只有在这张frame的bFrameEnd为1时去读这位才是有效值。

相关数据类型及接口

[MI_VENC_DataType_t](#) [#39-mi_venc_datatype_t]

3.12. MI_VENC_StreamInfoH264_t

- 描述

定义H.264协议码流特征信息。

- 定义

```
typedef struct MI_VENC_StreamInfoH264_s
{
    MI_U32 u32PicBytesNum;

    MI_U32 u32PSkipMbNum;

    MI_U32 u32IpcmMbNum;

    MI_U32 u32Inter16x8MbNum;

    MI_U32 u32Inter16x16MbNum;

    MI_U32 u32Inter8x16MbNum;

    MI_U32 u32Inter8x8MbNum;

    MI_U32 u32Intra16MbNum;

    MI_U32 u32Intra8MbNum;

    MI_U32 u32Intra4MbNum;

    MI_VENC_H264eRefSliceType_e eRefSliceType;

    MI_VENC_H264eRefType_e eRefType;

    MI_U32 u32UpdateAttrCnt;

    MI_U32 u32StartQp;
}MI_VENC_StreamInfoH264_t;
```

- 成员

成员名称	描述
u32PicBytesNum	编码当前帧的字节 (BYTE) 数
u32PSkipMbNum	编码当前帧中采用跳跃 (SKIP) 编码模式的宏块数
u32IpcmMbNum	编码当前帧中采用IPCM编码模式的宏块数
u32Inter16x8MbNum	编码当前帧中采用Inter16x8预测模式的宏块数
u32Inter16x16MbNum	编码当前帧中采用Inter16x16预测模式的宏块数
u32Inter8x16MbNum	编码当前帧中采用Inter8x16预测模式的宏块数
u32Inter8x8MbNum	编码当前帧中采用Inter8x8预测模式的宏块数
u32Intra16MbNum	编码当前帧中采用Intra16预测模式的宏块数
u32Intra8MbNum	编码当前帧中采用Intra8预测模式的宏块数
u32Intra4MbNum	编码当前帧中采用Intra4预测模式的宏块数
enRefSliceType	编码当前帧属于何种跳帧参考模式下的参考帧
enRefType	高级跳帧参考下的编码帧类型
u32UpdateAttrCnt	通道属性或参数 包含RC参数 被设置的次数
u32StartQp	编码采用的起始Qp值

- 注意
 - 保存H.264码流时，可只选择相应跳帧参考模式下的参考帧：
 - 当跳帧参考模式是1倍跳帧参考模式时，可只保存enRefSliceType等于 E_MI_VENC_H264E_REFSLICE_FOR_1X的码流。
 - 当跳帧参考模式是2倍跳帧参考模式时，可只保存enRefSliceType等于 E_MI_VENC_H264E_REFSLICE_FOR_2X的码流。
 - 当跳帧参考模式是4倍跳帧参考模式时，可只保存enRefSliceType等于E_MI_VENC_H264E_REFSLICE_FOR_4X的码流
 - 或同时保存enRefSliceType等于E_MI_VENC_H264E_REFSLICE_FOR_2X和E_MI_VENC_H264E_REFSLICE_FOR_4X的码流。
- 相关数据类型及接口

[MI_VENC_DataType_t](#) [#39-mi_venc_datatype_t]

[MI_VENC_H264eNaluType_e](#) [#33-mi_venc_h264enalutype_e]

.....

3.13. MI_VENC_StreamInfoJpeg_t

- 描述
定义JPEG协议码流特征信息。
- 定义

```
typedef struct MI_VENC_StreamInfoJpeg_s
{
    MI_U32 u32PicBytesNum;
```

```
MI_U32 u32UpdateAttrCnt;

MI_U32 u32Qfactor;

}MI_VENC_StreamInfoJpeg_t
```

- 成员

成员名称	描述
u32PicBytesNum	一帧jpeg码流大小，以字节（byte）为单位
u32UpdateAttrCnt	通道属性或参数 包含RC参数 被设置的次数
u32Qfactor	编码当前帧的Qfactor

- 相关数据类型及接口

[MI_VENC_DataType_t](#) [#39-mi_venc_datatype_t]

3.14. MI_VENC_StreamInfoH265_t

- 描述

定义H.265协议码流特征信息。

- 定义

```
typedef struct MI_VENC_StreamInfoH265_s
{
    MI_U32 u32PicBytesNum;

    MI_U32 u32Inter64x64CuNum;

    MI_U32 u32Inter32x32CuNum;

    MI_U32 u32Inter16x16CuNum;

    MI_U32 u32Inter8x8CuNum;

    MI_U32 u32Intra32x32CuNum;

    MI_U32 u32Intra16x16CuNum;

    MI_U32 u32Intra8x8CuNum;

    MI_U32 u32Intra4x4CuNum;

    MI_VENC_H265eRefType_e eRefType;

    MI_U32 u32UpdateAttrCnt;

    MI_U32 u32StartQp;

}MI_VENC_StreamInfoH265_t;
```

- 成员

成员名称	描述
u32PicBytesNum	编码当前帧的字节（BYTE）数
u32Inter64x64CuNum	编码当前帧中采用Inter64x64预测模式的CU块数
u32Inter32x32CuNum	编码当前帧中采用Inter32x32预测模式的CU块数
u32Inter16x16CuNum	编码当前帧中采用Inter16x16预测模式的CU块数
u32Inter8x8CuNum	编码当前帧中采用Inter8x8预测模式的CU块数
u32Intra32x32CuNum	编码当前帧中采用Intra32x32预测模式的CU块数
u32Intra16x16CuNum	编码当前帧中采用Intra16x16预测模式的CU块数
u32Intra8x8CuNum	编码当前帧中采用Intra8x8预测模式的CU块数
u32Intra4x4CuNum	编码当前帧中采用Intra4x4预测模式的CU块数
eRefType	高级跳帧参考下的编码帧类型
u32UpdateAttrCnt	通道属性或参数 包含RC参数 被设置的次数
u32StartQp	编码采用的起始Qp值

- 注意
enRefType请参见[MI_VENC_StreamInfoH264_t](#) [#312-mi_venc_streaminfoh264_t]中关于enRefType变量的说明。
- 相关数据类型及接口
[MI_VENC_DataType_t](#) [#39-mi_venc_datatype_t]
[MI_VENC_H264eNaluType_e](#) [#33-mi_venc_h264enalutype_e]
[MI_VENC_H264eRefType_e](#) [#35-mi_venc_h264ereftype_e]

3.15. MI_VENC_Stream_t

- 描述
定义帧码流类型结构体。
- 定义

```
typedef struct MI_VENC_Stream_s
{
    MI_VENC_Pack_t *pstPack;

    MI_U32  u32PackCount;

    MI_U32  u32Seq;

    MI_SYS_BUF_HANDLE hMiSys;

    union
    {
        MI_VENC_StreamInfoH264_t stH264Info;

        MI_VENC_StreamInfoJpeg_t stJpegInfo;
```

```
MI_VENC_StreamInfoH265_t stH265Info;

};

} MI_VENC_Stream_t;
```

- 成员

成员名称	描述
pstPack	帧码流包结构
u32PackCount	一帧码流的所有包的个数
u32Seq	码流序列号
hMiSys	Buffer句柄
stH624Info/stJpegInfo/ /stH265Info	码流特征信息

- 相关数据类型及接口

[MI_VENC_Pack_t](#) [#311-mi_venc_pack_t]

[MI_VENC_GetStream](#) [#214-mi_venc_getstream]

3.16. MI_VENC_StreamBufInfo_t

- 描述

定义码流buffer信息的结构体。

- 定义

```
typedef struct MI_VENC_StreamBufInfo_s
{
    MI_PHY phyAddr;

    void* pUserAddr;

    MI_U32 u32BufSize;
}MI_VENC_StreamBufInfo_t;
```

- 成员

成员名称	描述
u32PhyAddr	码流buffer的物理地址
pUserAddr	码流buffer的虚拟地址
u32BufSize	码流buffer的大小

- 相关数据类型及接口

[MI_VENC_GetStreamBufInfo](#)

3.17. MI_VENC_AttrH264_t

- 描述

定义H.264编码属性结构体。

- 定义

```
typedef struct MI_VENC_AttrH264_s
{
    MI_U32 u32MaxPicWidth;

    MI_U32 u32MaxPicHeight;

    MI_U32 u32BufSize;

    MI_U32 u32Profile;

    MI_BOOL bByFrame;

    MI_U32 u32PicWidth;

    MI_U32 u32PicHeight;

    MI_U32 u32BFrameNum;

    MI_U32 u32RefNum;
}MI_VENC_AttrH264_t;
```

- 成员

成员名称	描述
u32MaxPicWidth	编码图像最大宽度。 取值范围：[MIN_WIDTH,MAX_WIDTH]，以像素为单位。必须是MIN_ALIGN的整数倍。 静态属性。
u32PicWidth	编码图像宽度 取值范围：[MIN_WIDTH,u32MaxPicWidth]，以像素为单位。必须是MIN_ALIGN的整数倍。 动态属性。
u32MaxPicHeight	编码图像最大高度。取值范围：[MIN_HEIGHT,MAX_HEIGHT]，以像素为单位。必须是MIN_ALIGN的整数倍 静态属性。
u32PicHeight	编码图像高度 取值范围：[MIN_HEIGHT,u32MaxPicHeight]，以像素为单位。必须是MIN_ALIGN的整数倍 动态属性。
u32BufSize	码流buffer大小。 取值范围：[Min,Max]，以byte为单位。必须是64的整数倍。推荐值：一幅最大编码图像大小。推荐值为u32MaxPicWidthx，u32MaxPicHeightx1.5byte。 Min：一幅最大编码图像大小的½。 Max：无限制，但是会消耗更多的内存。 静态属性。
bByFrame	帧/包模式获取码流。取值范围：{TRUE,FALSE}。 TRUE：按帧获取。 FALSE：按包获取。 静态属性。
u32Profile	编码的等级。取值范围：[0,3]。0：Baseline。1：MainProfile。2：HighProfile。3：Svc-T。 动态属性。
u32BFrameNum	编码支持B帧的个数。 取值范围：[0,Max]。 Max：无限制。 静态属性，保留字段，暂不支持。
u32RefNum	编码支持参考帧的个数。 取值范围：[1,2]。 静态属性，保留字段，暂不支持。

3.18. MI_VENC_AttrJpeg_t

- 描述
定义JPEG抓拍属性结构体。
- 定义

```
typedef struct MI_VENC_AttrJpeg_s
{

    MI_U32  u32MaxPicWidth;

    MI_U32  u32MaxPicHeight;

    MI_U32  u32BufSize;

    MI_BOOL bByFrame;

    MI_U32  u32PicWidth;
```

```
MI_U32 u32PicHeight;

MI_BOOL bSupportDCF;

MI_U32 u32RestartMakerPerRowCnt;

}MI_VENC_AttrJpeg_t;
```

- 成员

成员名称	描述
u32MaxPicWidth	编码图像最大宽度 取值范围：[MIN_WIDTH,MAX_WIDTH]，以像素为单位。必须是MIN_ALIGN的整数倍。 静态属性。
u32PicWidth	编码图像宽度。 取值范围：[MIN_WIDTH,u32MaxPicWidth]，以像素为单位。必须是MIN_ALIGN的整数倍。 动态属性。
u32MaxPicHeight	编码图像最大高度。 取值范围：[MIN_HEIGHT,MAX_HEIGHT]，以像素为单位。必须是MIN_ALIGN的整数倍 静态属性。
u32PicHeight	编码图像高度。 取值范围：[MIN_HEIGHT,u32MaxPicHeight]，以像素为单位。必须是MIN_ALIGN的整数倍 动态属性。
u32BufSize	配置buffer大小。 取值范围：不小于图像最大宽高16对齐后的乘积。必须是64的整数倍。 静态属性。
bByFrame	获取码流模式,帧或包。 取值范围：{TRUE,FALSE}。 TRUE：按帧获取。 FALSE：按包获取。 静态属性。
bSupportDCF	是否支持Jpeg的缩略图。 静态属性。
u32RestartMakerPerRowCnt	每隔一定行数就重新开始一个marker。

3.19. MI_VENC_AttrH265_t

- 描述

定义H.265编码属性结构体。

- 定义

```
typedef struct MI_VENC_AttrH265_s
{

MI_U32 u32MaxPicWidth;

MI_U32 u32MaxPicHeight;

MI_U32 u32BufSize;

MI_U32 u32Profile;

MI_BOOL bByFrame;
```

```
MI_U32 u32PicWidth;

MI_U32 u32PicHeight;

MI_U32 u32BFrameNum;

MI_U32 u32RefNum;

}MI_VENC_AttrH265_t;
```

- 成员

成员名称	描述
u32MaxPicWidth	编码图像最大宽度。 取值范围：[MIN_WIDTH,MAX_WIDTH]，以像素为单位。必须是MIN_ALIGN的整数倍。 静态属性。
u32PicWidth	编码图像宽度。 取值范围：[MIN_WIDTH,u32MaxPicWidth]，以像素为单位。必须是MIN_ALIGN的整数倍。 动态属性。
u32MaxPicHeight	编码图像最大高度。 取值范围：[MIN_HEIGHT,MAX_HEIGHT]，以像素为单位。必须是MIN_ALIGN的整数倍 静态属性。
u32PicHeight	编码图像高度。 取值范围：[MIN_HEIGHT,u32MaxPicHeight]，以像素为单位。必须是MIN_ALIGN的整数倍 动态属性。
u32BufSize	码流buffer大小。 取值范围：[Min,Max]，以byte为单位。必须是64的整数倍。 推荐值：一幅最大编码图像大小。推荐值为u32MaxPicWidth, u32MaxPicHeight 1.5byte。 Min：一幅最大编码图像大小的½。 Max：无限制，但是会消耗更多的内存。 静态属性。
bByFrame	帧/包模式获取码流。取值范围：{TRUE,FALSE}。 TRUE：按帧获取。 FALSE：按包获取。 静态属性。
u32Profile	编码的等级。 取值范围：0。 0：MainProfile。 动态属性。
u32BFrameNum	编码支持B帧的个数。取值范围：[0,Max]。Max：无限制。静态属性，保留字段，暂不支持。
u32RefNum	编码支持参考帧的个数。 取值范围：[1,2]。 静态属性，保留字段，暂不支持。

3.20. MI_VENC_Attr_t

- 描述
定义编码器属性结构体。
- 定义

```
typedef struct MI_VENC_Attr_s
{
    MI_VENC_ModType_e      eType;

    union
    {
        MI_VENC_AttrH264_t  stAttrH264e;

        MI_VENC_AttrJpeg_t   stAttrJpeg;

        MI_VENC_AttrH265_t  stAttrH265e;
    };
}MI_VENC_Attr_t;
```

- 成员

成员名称	描述
eType	编码协议类型
stAttrH264e/stAttrMjpeg/stAttrJpeg/ /stAttrH265e	某种协议的编码器属性

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.21. MI_VENC_ChnAttr_t

- 描述

定义编码通道属性结构体。

- 定义

```
typedef struct MI_VENC_ChnAttr_s
{
    MI_VENC_Attr_t stVeAttr;

    MI_VENC_RcAttr_t stRcAttr;
}MI_VENC_ChnAttr_t;
```

- 成员

成员名称	描述
stVeAttr	编码器属性
stRcAttr	码率控制器属性

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.22. MI_VENC_ChnStat_t

- 描述
定义编码通道的状态结构体。
- 定义

```
typedef struct MI_VENC_CHN Stat_s
{
    MI_U32 u32LeftPics;

    MI_U32 u32LeftStreamBytes;

    MI_U32 u32LeftStreamFrames;

    MI_U32 u32LeftStreamMillisec;

    MI_U32 u32CurPacks;

    MI_U32 u32LeftRecvPics;

    MI_U32 u32LeftEncPics;

    MI_U32 u32FrmRateNum;

    MI_U32 u32FrmRateDen;

    MI_U32 u32Bitrate;
}MI_VENC_ChnStat_t;
```

- 成员

成员名称	描述
u32LeftPics	待编码的图像数
u32LeftStreamBytes	码流buffer剩余的byte数
u32LeftStreamFrames	码流buffer剩余的帧数
u32LeftStreamMillisec	码流buffer剩余的时长，单位为ms
u32CurPacks	当前帧的码流包个数
u32LeftRecvPics	剩余待接收的帧数，在用户设置 MI_VENC_StartRecvPicEx后有效
u32LeftEncPics	剩余待编码的帧数，在用户设置 MI_VENC_StartRecvPicEx后有效
u32FrmRateNum	最近1s内统计的帧率分子，以整数为单位
u32FrmRateDen	最近1s内统计的帧率分母，以整数为单位
u32Bitrate	最近1s内统计的码率，单位为kbps

- 相关数据类型及接口
[MI_VENC_Query](#) [#211-mi_venc_query]

3.23. MI_VENC_ParamH264SliceSplit_t

- 描述

定义H.264协议编码通道SLICE分割结构体。

- 定义

```
typedef struct MI_VENC_ParamH264SliceSplit_s
{
    MI_BOOL bSplitEnable;

    MI_U32 u32SliceRowCount;
}MI_VENC_ParamH264SliceSplit_t;
```

- 成员

成员名称	描述
bSplitEnable	Slice分割是否使能
u32SliceRowCount	表示每个slice占的宏块行数。 最小值为：1，最大值为：图像高\+15/16

- 相关数据类型及接口

[MI_VENC_SetH264SliceSplit](#) [#229-mi_venc_seth264slicesplit]

[MI_VENC_GetH264SliceSplit](#) [#230-mi_venc_geth264slicesplit]

3.24. MI_VENC_ParamH265SliceSplit_t

- 描述

定义H.265协议编码通道SLICE分割结构体。

- 定义

```
typedef struct MI_VENC_ParamH265SliceSplit_s
{
    MI_BOOL bSplitEnable;

    MI_U32 u32SliceRowCount;
}MI_VENC_ParamH265SliceSplit_t;
```

- 成员

成员名称	描述
bSplitEnable	Slice分割是否使能
u32SliceRowCount	表示每个slice占的宏块行数 最小值为：1，最大值为：图像高\+15/16

- 相关数据类型及接口

[MI_VENC_SetH265SliceSplit](#) [#253-mi_venc_seth265slicesplit]

[MI_VENC_GetH265SliceSplit](#) [#254-mi_venc_geth265slicesplit]

3.25. MI_VENC_ParamH264InterPred_t

- 描述
定义H.264协议编码通道帧间预测结构体。
- 定义

```
typedef struct MI_VENC_ParamH264InterPred_s
{
    /* search window */

    MI_U32 u32HWSize;

    MI_U32 u32VWSize;

    MI_BOOL bInter16x16PredEn;

    MI_BOOL bInter16x8PredEn;

    MI_BOOL bInter8x16PredEn;

    MI_BOOL bInter8x8PredEn;

    MI_BOOL bInter8x4PredEn;

    MI_BOOL bInter4x8PredEn;

    MI_BOOL bInter4x4PredEn;

    MI_BOOL bExtedgeEn;
} MI_VENC_ParamH264InterPred_t;
```

- 成员

成员名称	描述
u32HWSize	水平搜索窗大小
u32VWSize	垂直搜索窗大小
bInter16x16PredEn	16x16帧间预测使能开关，默认使能
bInter16x8PredEn	16x8帧间预测使能开关，默认使能
bInter8x16PredEn	8x16帧间预测使能开关，默认使能
bInter8x8PredEn	8x8帧间预测使能开关，默认使能
bInter8x4PredEn	8x4帧间预测使能开关，默认使能
bInter4x8PredEn	4x8帧间预测使能开关，默认使能
bInter4x4PredEn	4x4帧间预测使能开关，默认使能
bExtedgeEn	当搜索遇见图像边界时，超出了图像范围，是否进行补搜的使能开关，默认使能

- 相关数据类型及接口
[MI_VENC_SetH264InterPred](#) [#231-mi_venc_seth264interpred]
[MI_VENC_GetH264InterPred](#) [#232-mi_venc_geth264interpred]

3.26. MI_VENC_ParamH264IntraPred_t

- 描述

定义H.264协议编码通道帧内预测结构体。

- 定义

```
typedef struct MI_VENC_ParamH264IntraPred_s
{
    MI_BOOL bIntra16x16PredEn;

    MI_BOOL bIntraNxNPredEn;

    MI_BOOL bConstrainedIntraPredFlag;

    MI_BOOL bIpcmEn;

    MI_U32 u32Intra16x16Penalty;

    MI_U32 u32Intra4x4Penalty;

    MI_BOOL bIntraPlanarPenalty;
}MI_VENC_ParamH264IntraPred_t;
```

- 成员

成员名称	描述
bIntra16x16PredEn	16x16帧内预测使能，默认使能 取值范围：0或1 0：不使能； 1：使能
bIntraNxNPredEn	NxN帧内预测使能，默认使能 取值范围：0或1 0：不使能；1：使能
bConstrainedIntraPredFlag	默认为0。 取值范围：0或1。 请参见H.264协议关于constrained_intra_pred_flag的解释。
blpcmEn	IPCM预测使能，默认值根据不同的芯片有差异 取值范围：0或1
u32Intra16x16Penalty	默认为0
u32Intra4x4Penalty	默认为0
bIntraPlanarPenalty	默认为0

- 相关数据类型及接口

[MI_VENC_SetH264IntraPred](#) [#233-mi_venc_seth264intrapred]

[MI_VENC_GetH264IntraPred](#) [#234-mi_venc_geth264intrapred]

3.27. MI_VENC_ParamH264Trans_t

- 描述

定义H.264协议编码通道变换、量化结构体。

- 定义

```
typedef struct MI_VENC_ParamH264Trans_s
{
    MI_U32 u32IntraTransMode;

    MI_U32 u32InterTransMode;

    MI_S32 s32ChromaQpIndexOffset;
}MI_VENC_ParamH264Trans_t;
```

- 成员

成员名称	描述
u32IntraTransMode	帧内预测的变换模式：保留接口，目前暂未支持。
u32InterTransMode	帧间预测的变换模式：保留接口，目前暂未支持。
s32ChromaQpIndexOffset	具体含义请参见H.264协议关于slice_qp_offset_index的解释。系统默认值为0。取值范围：[-12,12]。

- 相关数据类型及接口

[MI_VENC_SetH264Trans](#) [#235-mi_venc_seth264trans]

[MI_VENC_GetH264Trans](#) [#236-mi_venc_geth264trans]

3.28. MI_VENC_ParamH264Entropy_t

- 描述

定义H.264协议编码通道熵编码结构体。

- 定义

```
typedef struct MI_VENC_ParamH264Entropy_s
{
    MI_U32 u32EntropyEncModeI;

    MI_U32 u32EntropyEncModeP;
}MI_VENC_ParamH264Entropy_t;
```

- 成员

成员名称	描述
u32EntropyEncModel	I帧熵编码模式 0：cavlc 1：cabac >=2没有意义 Baseline不支持cabac。
u32EntropyEncModeP	P帧熵编码模式。 0：cavlc 1：cabac >=2没有意义 Baseline不支持cabac。

- 相关数据类型及接口
[MI_VENC_SetH264Entropy](#) [#237-mi_venc_seth264entropy]
[MI_VENC_GetH264Entropy](#) [#238-mi_venc_geth264entropy]

3.29. MI_VENC_ParamH265InterPred_t

- 描述
定义H.265协议编码通道帧间预测结构体。
- 定义

```
typedef struct MI_VENC_ParamH265InterPred_s
{
    /*search window*/

    MI_U32 u32HWSIZE;

    MI_U32 u32VWSIZE;

    MI_BOOL bInter16x16PredEn;

    MI_BOOL bInter16x8PredEn;

    MI_BOOL bInter8x16PredEn;

    MI_BOOL bInter8x8PredEn;

    MI_BOOL bInter8x4PredEn;

    MI_BOOL bInter4x8PredEn;

    MI_BOOL bInter4x4PredEn;

    MI_U32 u32Inter32x32Penalty;

    MI_U32 u32Inter16x16Penalty;

    MI_U32 u32Inter8x8Penalty;

    MI_BOOL bExtedgeEn;
}MI_VENC_ParamH265InterPred_t;
```

- 成员

成员名称	描述
u32HWSIZE	水平搜索窗大小
u32VWSIZE	垂直搜索窗大小
bInter16x16PredEn	16x16帧间预测使能开关，默认使能
bInter16x8PredEn	16x8帧间预测使能开关，默认使能
bInter8x16PredEn	8x16帧间预测使能开关，默认使能
bInter8x8PredEn	8x8帧间预测使能开关，默认使能
bInter8x4PredEn	8x4帧间预测使能开关，默认使能
bInter4x8PredEn	4x8帧间预测使能开关，默认使能
bInter4x4PredEn	4x4帧间预测使能开关，默认使能
u32Inter32x32Penalty	32x32帧间预测选中的概率，值越大选中的概率越低 默认为0。取值范围[0,65535]
u32Inter16x16Penalty	16x16帧间预测选中的概率，值越大选中的概率越低 默认为0。取值范围[0,65535]
u32Inter8x8Penalty	8x8帧间预测选中的概率，值越大选中的概率越低 默认为0。取值范围[0,65535]
bExtedgeEn	当搜索遇见图像边界时，超出了图像范围，是否进行补搜的使能开关，默认使能

- 相关数据类型及接口

[MI_VENC_SetH265InterPred](#) [#239-mi_venc_seth265interpred]

[MI_VENC_GetH265InterPred](#) [#240-mi_venc_geth265interpred]

3.30. MI_VENC_ParamH265IntraPred_t

- 描述

定义H.265协议编码通道帧内预测结构体。

- 定义

```
typedef struct MI_VENC_ParamH265IntraPred_s
{
    MI_BOOL bIntra32x32PredEn;

    MI_BOOL bIntra16x16PredEn;

    MI_BOOL bIntra8x8PredEn;

    MI_BOOL bConstrainedIntraPredFlag;

    MI_U32 u32Intra32x32Penalty;

    MI_U32 u32Intra16x16Penalty;

    MI_U32 u32Intra8x8Penalty;
```

```
}MI_VENC_ParamH265IntraPred_t;
```

- 成员

成员名称	描述
bIntra32x32PredEn	32x32帧间预测使能开关·默认使能
bIntra16x16PredEn	16x16帧间预测使能开关·默认使能
bIntra8x8PredEn	8x8帧间预测使能开关·默认使能
bConstrainedIntraPredFlag	默认为0 取值范围：0或1 请参见H.265协议关于constrained_intra_pred_flag的解释
u32Intra32x32Penalty	32x32块帧内预测被选中概率·值越大概率越低 默认为0
u32Intra16x16Penalty	16x16块帧内预测被选中概率·值越大概率越低 默认为0
u32Intra8x8Penalty	8x8块帧内预测被选中概率·值越大概率越低 默认为0

- 相关数据类型及接口

[MI_VENC_SetH265IntraPred](#) [#241-mi_venc_seth265intrapred]

[MI_VENC_GetH265IntraPred](#) [#242-mi_venc_geth265intrapred]

3.31. MI_VENC_ParamH265Trans_t

- 描述

定义H.265协议编码通道变换、量化结构体。

- 定义

```
typedef struct MI_VENC_ParamH265Trans_s
{
    MI_U32  u32IntraTransMode;

    MI_U32  u32InterTransMode;

    MI_S32  s32ChromaQpIndexOffset;

}MI_VENC_ParamH265Trans_t;
```

- 成员

成员名称	描述
u32IntraTransMode	帧内预测的变换模式：保留接口，目前暂未支持。
u32InterTransMode	帧间预测的变换模式：保留接口，目前暂未支持。
s32ChromaQpIndexOffset	具体含义请参见H.265协议关于slice_cb_qp_offset和slice_cr_qp_offset的解释。系统默认值为0。取值范围：[-12,12]。

- 相关数据类型及接口
[MI_VENC_SetH265Trans](#) [#243-mi_venc_seth265trans]
[MI_VENC_GetH265Trans](#) [#244-mi_venc_geth265trans]

3.32. MI_VENC_ParamH264Dbld_t

- 描述
定义H.264协议编码通道Dbld结构体。
- 定义

```
typedef struct MI_VENC_ParamH264Dbld_s
{
    MI_U32  disable_deblocking_filter_idc;

    MI_S32  slice_alpha_c0_offset_div2;

    MI_S32  slice_beta_offset_div2;

}MI_VENC_ParamH264Dbld_t;
```

- 成员
- | 成员名称 | 描述 |
|-------------------------------|--------------------------------|
| disable_deblocking_filter_idc | 取值范围[0,2],默认值0。具体含义请参见H.264协议 |
| slice_alpha_c0_offset_div2 | 取值范围[-6,6],默认值0。具体含义请参见H.264协议 |
| slice_beta_offset_div2 | 取值范围[-6,6],默认值0。具体含义请参见H.264协议 |

- 相关数据类型及接口
[MI_VENC_SetH264Dbld](#) [#245-mi_venc_seth264dbld]
[MI_VENC_GetH264Dbld](#) [#246-mi_venc_geth264dbld]

3.33. MI_VENC_ParamH265Dbld_t

- 描述
定义H.265协议编码通道Dbld结构体。
- 定义

```
typedef struct MI_VENC_ParamH265Dbld_s
```

```
{

    MI_U32 disable_deblocking_filter_idc; //special naming for CODEC ISO SPEC.

    MI_S32 slice_tc_offset_div2; //special naming for CODEC ISO SPEC.

    MI_S32 slice_beta_offset_div2; //special naming for CODEC ISO SPEC.

} MI_VENC_ParamH265Dbk_t;
```

- 成员

成员名称	描述
disable_deblocking_filter_idc	取值范围[0,2],默认值0 · 具体含义请参见H.265协议关于slice_deblocking_filter_disabled_flag的解释
slice_tc_offset_div2	取值范围[-6,6],默认值0 · 具体含义请参见H.265协议
slice_beta_offset_div2	取值范围[-6,6],默认值0 · 具体含义请参见H.265协议

- 相关数据类型及接口

[MI_VENC_SetH265Dbk](#) [#247-mi_venc_seth265dbk]

[MI_VENC_GetH265Dbk](#) [#248-mi_venc_geth265dbk]

3.34. MI_VENC_ParamH264Vui_t

- 描述

定义H.264协议编码通道Vui结构体。

- 定义

```
typedef struct MI_VENC_ParamH264Vui_s

{

    MI_VENC_ParamH264VuiAspectRatio_t stVuiAspectRatio;

    MI_VENC_ParamH264VuiTimeInfo_t stVuiTimeInfo;

    MI_VENC_ParamH264VuiVideoSignal_t stVuiVideoSignal;

}MI_VENC_ParamH264Vui_t;
```

- 成员

成员名称	描述
stVuiAspectRatio	具体含义请参见H.264协议中Annex E Video usability information的介绍
stVuiTimeInfo	具体含义请参见H.264协议中Annex E Video usability information的介绍
stVuiVideoSignal	具体含义请参见H.264协议中Annex E Video usability information的介绍

- 相关数据类型及接口

[MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]

[MI_VENC_GetH264Vui](#) [#250-mi_venc_geth264vui]

3.35. MI_VENC_ParamH264VuiAspectRatio_t

- 描述
定义H.264协议编码通道Vui中AspectRatio信息的结构体。
- 定义

```
typedef struct MI_VENC_ParamH264VuiAspectRatio_s
{
    MI_U8  u8AspectRatioInfoPresentFlag;

    MI_U8  u8AspectRatioIdc;

    MI_U8  u8OverscanInfoPresentFlag;

    MI_U8  u8OverscanAppropriateFlag;

    MI_U16  u16SarWidth;

    MI_U16  u16SarHeight;

}MI_VENC_ParamH264VuiAspectRatio_t;
```

- 成员

成员名称	描述
u8AspectRatioInfoPresentFlag	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为0。 取值范围：0或1
u8AspectRatioIdc	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为1。 取值范围：[0,255],17~254保留
u8OverscanInfoPresentFlag	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为0。 取值范围：0或1
u8OverscanAppropriateFlag	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为0。 取值范围：0或1
u16SarWidth	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为1。 取值范围：(0,65535],并且与u16SarHeight互质
u16SarHeight	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为1。 取值范围：(0,65535],并且与u16SarWidth互质

- 相关数据类型及接口
[MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]

3.36. MI_VENC_ParamH264VuiTimeInfo_t

- 描述
定义H.264协议编码通道Vui中TIME_INFO信息的结构体。
- 定义

```
typedef struct MI_VENC_ParamH264VuiTimeInfo_s
{
```



```
MI_U8 u8TimingInfoPresentFlag;

MI_U8 u8FixedFrameRateFlag;

MI_U32 u32NumUnitsInTick;

MI_U32 u32TimeScale;

}MI_VENC_ParamH264VuiTimeInfo_t;
```

- 成员

成员名称	描述
u8TimingInfoPresentFlag	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为0。取值范围：0或1。
u32NumUnitsInTick	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为1。取值范围：大于0。
u32TimeScale	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为60。取值范围：大于0。
u8FixedFrameRateFlag;	具体含义请参见H.264协议中Annex E Video usability information的介绍，系统默认为1。取值范围：0或1。

- 相关数据类型及接口

[MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]

[MI_VENC_GetH264Vui](#) [#250-mi_venc_geth264vui]

3.37. MI_VENC_ParamH264VuiVideoSignal_t

- 描述

定义H.264协议编码通道Vui中VIDEO_SIGNAL信息的结构体。

- 定义

```
typedef struct MI_VENC_ParamVuiVideoSignal_s
{
    MI_U8 u8VideoSignalTypePresentFlag;

    MI_U8 u8VideoFormat;

    MI_U8 u8VideoFullRangeFlag;

    MI_U8 u8ColourDescriptionPresentFlag;

    MI_U8 u8ColourPrimaries;

    MI_U8 u8TransferCharacteristics;

    MI_U8 u8MatrixCoefficients;

}MI_VENC_ParamH264VuiVideoSignal_t;
```

- 成员

成员名称	描述
u8VideoSignalTypePresentFlag	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为1。取值范围：0或1。
u8VideoFormat	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为5。取值范围：[0,7]。
u8VideoFullRangeFlag	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为1。取值范围：0或1。
u8ColourDescriptionPresentFlag;	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为1。取值范围：0或1。
u8ColourPrimaries	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为1。取值范围：[0,255]。
u8TransferCharacteristics	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为1。取值范围：[0,255]。
u8MatrixCoefficients	具体含义请参见H.264协议中Annex E Video usability information的介绍。系统默认为1。取值范围：[0,255]。

- 相关数据类型及接口

[MI_VENC_SetH264Vui](#) [#249-mi_venc_seth264vui]

3.38. MI_VENC_ParamH265Vui_t

- 描述

定义H.265协议编码通道Vui结构体。

-定义

```
typedef struct MI_VENC_ParamH265Vui_s
{
    MI_VENC_ParamH265VuiAspectRatio_t stVuiAspectRatio;

    MI_VENC_ParamH265VuiTimeInfo_t stVuiTimeInfo;

    MI_VENC_ParamH265VuiVideoSignal_t stVuiVideoSignal;
}MI_VENC_ParamH265Vui_t;
```

- 成员

成员名称	描述
stVuiAspectRatio	具体含义请参见H.265协议中Annex E Video usability information的介绍。
stVuiTimeInfo	具体含义请参见H.265协议中Annex E Video usability information的介绍。
stVuiVideoSignal	具体含义请参见H.265协议中Annex E Video usability information的介绍。

- 相关数据类型及接口

[MI_VENC_SetH265Vui](#) [#251-mi_venc_seth265vui]

3.39. MI_VENC_ParamH265VuiAspectRatio_t

- 描述
定义H.265协议编码通道Vui中AspectRatio信息的结构体。
- 定义

```
typedef struct MI_VENC_ParamH265VuiAspectRatio_s
{
    MI_U8  u8AspectRatioInfoPresentFlag;

    MI_U8  u8AspectRatioIdc;

    MI_U8  u8OverscanInfoPresentFlag;

    MI_U8  u8OverscanAppropriateFlag;

    MI_U16 u16SarWidth;

    MI_U16 u16SarHeight;

}MI_VENC_ParamH265VuiAspectRatio_t;
```

- 成员

成员名称	描述
u8AspectRatioInfoPresentFlag	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为0。取值范围：0或1。
u8AspectRatioIdc	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：[0,255],17~254保留。
u8OverscanInfoPresentFlag	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为0。取值范围：0或1。
u8OverscanAppropriateFlag	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为0。取值范围：0或1。
u16SarWidth	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：(0,65535],并且与u16SarHeight互质。
u16SarHeight	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：(0,65535],并且与u16SarWidth互质。

- 相关数据类型及接口

3.40. MI_VENC_ParamH265VuiTimeInfo_t

- 描述
定义H.265协议编码通道Vui中TIME_INFO信息的结构体。
- 定义

```
typedef struct MI_VENC_ParamH265VuiTimeInfo_s
{
```

```
MI_U8 u8TimingInfoPresentFlag;

MI_U8 u8FixedFrameRateFlag;

MI_U32 u32NumUnitsInTick;

MI_U32 u32TimeScale;

}MI_VENC_ParamH265VuiTimeInfo_t;
```

- 成员

成员名称	描述
u8TimingInfoPresentFlag	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为0。取值范围：0或1。
u32NumUnitsInTick	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：大于0。
u32TimeScale	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为60。取值范围：大于0。
u8FixedFrameRateFlag;	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：0或1。

- 相关数据类型及接口

[MI_VENC_SetH265Vui](#) [#251-mi_venc_seth265vui]

[MI_VENC_GetH265Vui](#) [#252-mi_venc_geth265vui]

3.41. MI_VENC_ParamH265VuiVideoSignal_t

- 描述

定义H.265协议编码通道Vui中VIDEO_SIGNAL信息的结构体。

- 定义

```
typedef struct MI_VENC_ParamVuiVideoSignal_s
{
    MI_U8 u8VideoSignalTypePresentFlag;

    MI_U8 u8VideoFormat;

    MI_U8 u8VideoFullRangeFlag;

    MI_U8 u8ColourDescriptionPresentFlag;

    MI_U8 u8ColourPrimaries;

    MI_U8 u8TransferCharacteristics;

    MI_U8 u8MatrixCoefficients;

}MI_VENC_ParamH265VuiVideoSignal_t;
```

- 成员

成员名称	描述
u8VideoSignalTypePresentFlag	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：0或1。
u8VideoFormat	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为5。取值范围：[0,7]。
u8VideoFullRangeFlag	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：0或1。
u8ColourDescriptionPresentFlag;	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：0或1。
u8ColourPrimaries	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：[0,255]。
u8TransferCharacteristics	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：[0,255]。
u8MatrixCoefficients	具体含义请参见H.265协议中Annex E Video usability information的介绍，系统默认为1。取值范围：[0,255]。

- 相关数据类型及接口

[MI_VENC_SetH265Vui](#) [#251-mi_venc_seth265vui]

3.42. MI_VENC_ParamJpeg_t

- 描述

定义JPEG协议编码通道高级参数结构体。

- 定义

```
typedef struct MI_VENC_ParamJpeg_s
{
    MI_U32  u32Qfactor;

    MI_U8  au8YQt[64];

    MI_U8  au8CbCrQt[64];

    MI_U32  u32McuPerEcs;
} MI_VENC_ParamJpeg_t;
```

- 成员

成员名称	描述
u32Qfactor	具体含义请参见RFC2435协议，系统默认为90。 取值范围：[1,90]
au8YQt	Y量化表。 取值范围：[1,255]
au8CbCrQt	CbCr量化表。 取值范围：[1,255]
u32MCUPerEcs	每个ECS中包含多少个MCU，系统默认为0，表示不划分Ecs u32MCUPerECS： $[0, \text{picwidth} \backslash +15 \gg 4 \times \text{picheight} \backslash +15 \gg 4 \times 2]$

- 相关数据类型及接口

[MI_VENC_SetJpegParam](#) [#255-mi_venc_setjpegparam]]

[MI_VENC_GetJpegParam](#) [#256-mi_venc_getjpegparam]

3.43. MI_VENC_RoiCfg_t

- 描述

定义编码感兴趣区域信息。

- 定义

```
typedef struct MI_VENC_RoiCfg_s
{
    MI_U32 u32Index;

    MI_B00L bEnable;

    MI_B00L bAbsQp;

    MI_S32 s32Qp;

    MI_VENC_Rect_t stRect;
}MI_VENC_RoiCfg_t;
```

- 成员

成员名称	描述
u32Index	ROI区域的索引，系统支持的索引范围为[0,7]，不支持超出这个范围的索引
bEnable	是否使能这个ROI区域
bAbsQp	ROI区域QP模式 FALSE：相对QP TURE：绝对QP
s32Qp	QP值，当QP模式为MI_FALSE时，s32Qp为宏块QP偏移，当QP模式为TRUE时，s32Qp为宏块QP值
stRect	ROI区域

- 注意

从Pudding 、Ispahan开始，ROI区域的QP模式仅支持相对QP（ bAbsQp = MI_FALSE ）；同时，s32Qp值的可支持范围修改为[-32,31]。

- 相关数据类型及接口

[MI_VENC_SetRoiCfg](#) [#225-mi_venc_setroicfg]

[MI_VENC_GetRoiCfg](#) [#226-mi_venc_getroicfg]

3.44. MI_VENC_RoiBgFrameRate_t

- 描述

定义非编码感兴趣区域帧率。

- 定义

```
typedef struct MI_VENC_RoiBgFrameRate_s
{
    MI_S32 s32SrcFrmRate;

    MI_S32 s32DstFrmRate;
}MI_VENC_RoiBgFrameRate_t;
```

- 成员

成员名称	描述
s32SrcFrmRate	非Roi区域的源帧率
s32DstFrmRate	非Roi区域的目标帧率

- 相关数据类型及接口

[MI_VENC_SetRoiBgFrameRate](#) [#227-mi_venc_setroibgframerate]

[MI_VENC_GetRoiBgFrameRate](#) [#228-mi_venc_getroibgframerate]

3.45. MI_VENC_ParamRef_t

- 描述

定义H.264/H.265编码的高级跳帧参考参数。

- 定义

```
typedef struct MI_VENC_ParamRef_s
{
    MI_U32 u32Base;

    MI_U32 u32Enhance;

    MI_BOOL bEnablePred;
}MI_VENC_ParamRef_t;
```

- 成员

成员名称	描述
u32Base	base层的周期。取值范围：(0 · +∞)
u32Enhance	enhance层的周期。取值范围：[0,255]
bEnablePred	代表base层的帧是否被base层其他帧用作参考。当为 MI_FALSE时，base层的所有帧都参考IDR帧。

- 相关数据类型及接口
[MI_VENC_GetRefParam](#) [#260-mi_venc_getrefparam]
[MI_VENC_SetRefParam](#) [#259-mi_venc_setrefparam]

3.46. MI_VENC_RcAttr_t

- 描述
定义编码通道码率控制器属性。
- 定义

```
typedef struct MI_VENC_RcAttr_s
{
    MI_VENC_RcMode_e    eRcMode;

    union
    {
        MI_VENC_AttrH264Cbr_t      stAttrH264Cbr;
        MI_VENC_AttrH264Vbr_t      stAttrH264Vbr;
        MI_VENC_AttrH264FixQp_t    stAttrH264FixQp;
        MI_VENC_AttrH264Abr_t      stAttrH264Abr;
        MI_VENC_AttrH264Avbr_t     stAttrH264Avbr;
        MI_VENC_AttrMjpegCbr_t     stAttrMjpegCbr;
        MI_VENC_AttrMjpegFixQp_t   stAttrMjpegFixQp;
        MI_VENC_AttrH265Cbr_t      stAttrH265Cbr;
        MI_VENC_AttrH265Vbr_t      stAttrH265Vbr;
        MI_VENC_AttrH265FixQp_t    stAttrH265FixQp;
        MI_VENC_AttrH265Avbr_t     stAttrH265Avbr;
    };

    MI_VOID*    pRcAttr;
}MI_VENC_RcAttr_t;
```

- 成员

成员名称	描述
enRcMode	RC 模式
stAttrH264Cbr	H.264协议编码通道Cbr模式属性
stAttrH264Vbr	H.264协议编码通道Vbr模式属性
stAttrH264FixQp	H.264协议编码通道Fixqp模式属性
stAttrH264Abr	H.264协议编码通道Abr模式属性（ 暂不支持 ）
stAttrH264Avbr	H.264协议编码通道Avbr模式属性
stAttrH265Cbr	H.265协议编码通道Cbr模式属性
stAttrH265Vbr	H.265协议编码通道Vbr模式属性
stAttrH265FixQp	H.265协议编码通道Fixqp模式属性
stAttrH265Avbr	H.265协议编码通道Avbr模式属性

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.47. MI_VENC_RcMode_e

- 描述

定义编码通道码率控制器模式。

- 定义

```
typedef enum
{
    E_MI_VENC_RC_MODE_H264CBR = 1,
    E_MI_VENC_RC_MODE_H264VBR,
    E_MI_VENC_RC_MODE_H264ABR,
    E_MI_VENC_RC_MODE_H264FIXQP,
    E_MI_VENC_RC_MODE_H264AVBR,
    E_MI_VENC_RC_MODE_MJPEGCBR,
    E_MI_VENC_RC_MODE_H265CBR,
    E_MI_VENC_RC_MODE_H265VBR,
    E_MI_VENC_RC_MODE_H265FIXQP,
    E_MI_VENC_RC_MODE_H265AVBR,
    E_MI_VENC_RC_MODE_MAX,
}MI_VENC_RcMode_e;
```

- 成员

成员名称	描述
E_MI_VENC_RC_MODE_H264CBR	H.264 CBR模式
E_MI_VENC_RC_MODE_H264VBR	H.264 VBR模式
E_MI_VENC_RC_MODE_H264ABR	H.264 ABR模式（暂时不提供）
E_MI_VENC_RC_MODE_H264FIXQP	H.264 FixQp模式
E_MI_VENC_RC_MODE_H264AVBR	H.264 AVBR模式
E_MI_VENC_RC_MODE_MJPEGCBR	MJPEG CBR模式
E_MI_VENC_RC_MODE_MJPEGFIXQP	MJPEG FixQp模式
E_MI_VENC_RC_MODE_H265CBR	H.265 CBR模式
E_MI_VENC_RC_MODE_H265VBR	H.265 VBR模式
E_MI_VENC_RC_MODE_H265FIXQP	H.265 FixQp模式
E_MI_VENC_RC_MODE_H265AVBR	H.265 AVBR模式

- 相关数据类型及接口
- [MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.48. MI_VENC_AttrH264Cbr_t

- 描述
- 定义H.264编码通道CBR属性结构。
- 定义

```
typedef struct MI_VENC_AttrH264Cbr_s
{
    MI_U32 u32Gop;

    MI_U32 u32StatTime;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32BitRate;

    MI_U32 u32FluctuateLevel;
}MI_VENC_AttrH264Cbr_t;
```

- 成员

成员名称	描述
u32Gop	H.264 gop值。 取值范围：[1,65536]
u32StatTime	CBR码率统计时间，以秒为单位。 取值范围：[1,60]
u32SrcFrmRateNum	编码器帧率分子，以整数为单位
u32SrcFrmRateDen	编码器帧率分母，以整数为单位
u32BitRate	平均bitrate，以bps为单位。 取值范围：[2000,102400000]
u32FluctuateLevel	最大码率相对平均码率的波动等级，暂不支持。

- 注意
SrcFrmRate应该设置为输入编码器的实际帧率，RC需要根据SrcFrmRate进行码率控制。
u32SrcFrmRateNum/u32SrcFrmRateDen范围在(0,30]之间。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.49. MI_VENC_AttrH264Vbr_t

- 描述
定义H.264编码通道VBR属性结构。
- 定义

```
typedef struct MI_VENC_AttrH264Vbr_s
{
    MI_U32 u32Gop;

    MI_U32 u32StatTime;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32MaxBitRate;

    MI_U32 u32MaxQp;

    MI_U32 u32MinQp;
}MI_VENC_AttrH264Vbr_t;
```

- 成员

成员名称	描述
u32Gop	H.264 gop值。取值范围：[1,65536]
u32StatTime	VBR码率统计时间，以秒为单位。取值范围：[1,60]
u32SrcFrmRateNum	编码器帧率分子，以整数为单位
u32SrcFrmRateDen	编码器帧率分母，以整数为单位
u32MaxBitRate	编码器输出最大码率，以bps为单位。取值范围：[2000,102400000]。
u32MaxQp	编码器支持图像最大QP。取值范围：(u32MinQp,48]
u32MinQp	编码器支持图像最小QP。取值范围：[012,u32MaxQp)

- 注意
请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.50. MI_VENC_AttrH264FixQp_t

- 描述
定义H.264编码通道Fixqp属性结构。
- 定义

```
typedef struct MI_VENC_AttrH264FixQp_s
{
    MI_U32 u32Gop;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32IQp;

    MI_U32 u32PQp;
}MI_VENC_AttrH264FixQp_t;
```

- 成员

成员名称	描述
u32Gop	H.264 gop值。取值范围：[1,65536]。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32IQp	I帧所有宏块Qp值。取值范围：[12,48]。
u32PQp	P帧所有宏块Qp值。取值范围：[12,48]。

- 注意
请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.51. MI_VENC_AttrH264Abr_t

- 描述
定义H.264编码通道ABR属性结构。
- 定义

```
typedef struct MI_VENC_AttrH264Abr_s
{
    MI_U32  u32Gop;

    MI_U32  u32StatTime;

    MI_U32  u32SrcFrmRateNum;

    MI_U32  u32SrcFrmRateDen;

    MI_U32  u32AvgBitRate;

    MI_U32  u32MaxBitRate;
}MI_VENC_AttrH264Abr_t;
```

- 成员

成员名称	描述
u32Gop	H.264 gop值。取值范围：[1,65536]。
u32StatTime	ABR码率统计时间，以秒为单位。取值范围：[1,60]。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32AvgBitRate	平均码率，单位为bps 取值范围[2000,u32MaxBitRate)
u32MaxBitRate	最大码率。取值范围[2000,102400000]

- 注意
请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.52. MI_VENC_AttrH264Avbr_t

- 描述
定义H.264编码通道AVBR属性结构。

- 定义

```
typedef struct MI_VENC_AttrH264Avbr_s
{
    MI_U32 u32Gop;

    MI_U32 u32StatTime;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32MaxBitRate;

    MI_U32 u32MaxQp;

    MI_U32 u32MinQp;
} MI_VENC_AttrH264Avbr_t;
```

- 成员

成员名称	描述
u32Gop	gop值。取值范围：[1,65536]。
u32StatTime	码率统计时间，以秒为单位。取值范围：[1,60]。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32MaxBitRate	编码器输出最大码率，以bps为单位。取值范围：[2000,102400000]。
u32MaxQp	编码器支持图像最大QP。取值范围：(u32MinQp,48]。
u32MinQp	编码器支持图像最小QP。取值范围：[12, u32MaxQp)。

- 注意

请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.53. MI_VENC_AttrMjpegCbr_t

- 描述

定义MJPEG编码通道CBR属性结构。

- 定义

```
typedef struct MI_VENC_AttrMjpegCbr_s
{
    MI_U32 u32BitRate;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;
```

```
} MI_VENC_AttrMjpegCbr_t;
```

- 成员

成员名称	描述
u32BitRate	码率，以bps为单位 取值范围[2000, 102400000]
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。

- 注意

请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.54. MI_VENC_AttrMjpegFixQp_t

- 描述

定义MJPEG编码通道FixQp属性结构。

- 定义

```
typedef struct MI_VENC_AttrMjpegFixQp_s
{
    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32Qfactor;
} MI_VENC_AttrMjpegFixQp_t;
```

- 成员

成员名称	描述
u32Qfactor	Qfactor值，取值范围[1, 90]
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。

- 注意

请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.55. MI_VENC_AttrH265Cbr_t

- 描述

定义H.265编码通道CBR属性结构。

- 定义

```
typedef struct MI_VENC_AttrH265Cbr_s
{
    MI_U32 u32Gop;

    MI_U32 u32StatTime;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32BitRate;

    MI_U32 u32FluctuateLevel;
}MI_VENC_AttrH265Cbr_t;
```

- 成员

成员名称	描述
u32Gop	H.265 gop值。取值范围：[1,65536]。
u32StatTime	CBR码率统计时间，以秒为单位。取值范围：[1,60]。暂不支持。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32BitRate	平均bitrate，以bps为单位。取值范围：[2000,102400000]。
u32FluctuateLevel	最大码率相对平均码率的波动等级，暂不支持。

- 注意

请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRate和fr32DstFrmRate的说明。

- 相关数据类型及接口

[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.56. MI_VENC_AttrH265Vbr_t

- 描述

定义H.265编码通道VBR属性结构。

- 定义

```
typedef struct MI_VENC_AttrH265Vbr_s
{
    MI_U32 u32Gop;

    MI_U32 u32StatTime;

    MI_U32 u32SrcFrmRateNum;
```



```
MI_U32 u32SrcFrmRateDen;

MI_U32 u32MaxBitRate;

MI_U32 u32MaxQp;

MI_U32 u32MinQp;

}MI_VENC_AttrH265Vbr_t;
```

- 成员

成员名称	描述
u32Gop	H.265 gop值。取值范围：[1,65536]。
u32StatTime	VBR码率统计时间，以秒为单位。取值范围：[1,60]。暂不支持。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32MaxBitRate	编码器输出最大码率，以bps为单位。取值范围：[2000,102400000]。
u32MaxQp	编码器支持图像最大QP。取值范围：(u32MinQp,48]。
u32MinQp	编码器支持图像最小QP。取值范围：[12,u32MaxQp]。

- 注意
请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRate和fr32DstFrmRate的说明。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.57. MI_VENC_AttrH265FixQp_t

- 描述
定义H.265编码通道Fixqp属性结构。
- 定义

```
typedef struct MI_VENC_AttrH265FixQp_s
{
    MI_U32 u32Gop;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32IQp;

    MI_U32 u32PQp;

}MI_VENC_AttrH265FixQp_t;
```

- 成员

成员名称	描述
u32Gop	H.265 gop值。取值范围：[1,65536]。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32IQp	I帧所有宏块Qp值。取值范围：[12,48]。
u32PQp	P帧所有宏块Qp值。取值范围：[12,48]。

- 注意
请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRate和fr32DstFrmRate的说明。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.58. MI_VENC_AttrH265Avbr_t

- 描述
定义H.265编码通道AVBR属性结构。
- 定义

```
typedef struct MI_VENC_AttrH265Avbr_s
{
    MI_U32 u32Gop;

    MI_U32 u32StatTime;

    MI_U32 u32SrcFrmRateNum;

    MI_U32 u32SrcFrmRateDen;

    MI_U32 u32MaxBitRate;

    MI_U32 u32MaxQp;

    MI_U32 u32MinQp;
} MI_VENC_AttrH265Avbr_t;
```

- 成员

成员名称	描述
u32Gop	gop值。取值范围：[1,65536]。
u32StatTime	码率统计时间，以秒为单位。取值范围：[1,60]。暂不支持。
u32SrcFrmRateNum	编码器帧率分子，以整数为单位。
u32SrcFrmRateDen	编码器帧率分母，以整数为单位。
u32MaxBitRate	编码器输出最大码率，以bps为单位。取值范围：[2000,102400000]。
u32MaxQp	编码器支持图像最大QP。取值范围：(u32MinQp,48]。
u32MinQp	编码器支持图像最小QP。取值范围：[12,u32MaxQp]。

- 注意
请参见[MI_VENC_AttrH264Cbr_t](#) [#348-mi_venc_attrh264cbr_t]关于u32SrcFrmRateNum和u32SrcFrmRateDen的说明。
- 相关数据类型及接口
[MI_VENC_CreateChn](#) [#25-mi_venc_createchn]

3.59. MI_VENC_SuperFrmMode_e

- 描述
定义码率控制中超大帧处理模式。
- 定义

```
typedef enum
{
    E_MI_VENC_SUPERFRM_NONE,
    E_MI_VENC_SUPERFRM_DISCARD,
    E_MI_VENC_SUPERFRM_REENCODE,
    E_MI_VENC_SUPERFRM_MAX
}MI_VENC_SuperFrmMode_e;
```

- 成员

成员名称	描述
E_MI_VENC_SUPERFRM_NONE	无特殊策略
E_MI_VENC_SUPERFRM_DISCARD	丢弃超大帧
E_MI_VENC_SUPERFRM_REENCODE	重编超大帧

- 相关数据类型及接口
[MI_VENC_GetSuperFrameCfg](#) [#266-mi_venc_getsuperframecfg]
[MI_VENC_SetSuperFrameCfg](#) [#265-mi_venc_setsuperframecfg]

3.60. MI_VENC_ParamH264Vbr_t

- 描述
定义H264协议编码通道VBR码率控制模式高级参数配置。
- 定义

```
typedef struct MI_VENC_ParamH264Vbr_s
{
    MI_S32  s32IPQPDelta;

    MI_S32  s32ChangePos;

    MI_U32  u32MaxIQp;

    MI_U32  u32MinIQp;

    MI_U32  u32MaxIPProp;

    MI_U32  u32MaxISize;

    MI_U32  u32MaxPSize;
}MI_VENC_ParamH264Vbr_t;
```

- 成员

成员名称	描述
s32IPQPDelta	IPQP变化值。取值范围：[-12,12]
s32ChangePos	VBR开始调整Qp时的码率相对于最大码率的比例。取值范围：[5,100]
u32MaxIQp	I帧的最大QP。用于控制I帧的最小bits数。取值范围：(MinQp,MaxQp]
u32MinIQp	I帧的最小QP。用于控制I帧的最大bits数。取值范围：[MinQp,MaxQp)
u32MaxIPProp	最大IP帧码率的比值。取值范围[5,100]
u32MaxISize	I帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。取值范围：[0, 0xFFFFFFFF] 默认值：0
u32MaxPSize	P帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。取值范围：[0, 0xFFFFFFFF] 默认值：0

- 相关数据类型及接口
[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]
[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

3.61. MI_VENC_ParamH264Avbr_t

- 说明
定义H264协议编码通道AVBR码率控制模式高级参数配置。
- 定义 typedef struct MI_VENC_ParamH264Avbr_s

```
{  
  
    MI_S32 s32IPQPDelta;  
  
    MI_S32 s32ChangePos;  
  
    MI_U32 u32MinIQp;  
  
    MI_U32 u32MaxIPProp;  
  
    MI_U32 u32MaxIQp;  
  
    MI_U32 u32MaxISize;  
  
    MI_U32 u32MaxPSize;  
  
    MI_U32 u32MinStillPercent;  
  
    MI_U32 u32MaxStillIQp;  
  
    MI_U32 u32MotionSensitivity;  
  
} MI_VENC_ParamH264Avbr_t;
```

- 成员

成员名称	描述
s32IPQPDelta	IPQP变化值 取值范围：[-12,12] 默认值：0
s32ChangePos	AVBR开始调整Qp时的码率相对于最大码率的比例 取值范围：[5,100] 默认值：80
u32MinIQp	I帧的最小QP。用于控制I帧的最大bits数 取值范围：[MinQp,MaxQp) 默认值：12
u32MaxIPProp	最大IP帧码率的比值 取值范围[5,100]
u32MaxIQp	I帧的最大QP。用于控制I帧的最小bits数 取值范围：(MinQp,MaxQp] 默认值：48
u32MaxISize	I帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。 取值范围：[0, 0xFFFFFFFF] 默认值：0
u32MaxPSize	P帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。 取值范围：[0, 0xFFFFFFFF] 默认值：0
u32MinStillPercent	静止状态下目标码率的最小百分比。若此变量设置为100，AVBR将不会在判别为静止时主动调低目标码率，AVBR的表现将和VBR一致。 取值范围：[5,100] 默认值：25
u32MaxStillQp	静止状态最大I帧QP 取值范围：[MinIQp,MaxIQp] 暂不支持
u32MotionSensitivity	根据画面运动程度调整码率的灵敏度 取值范围：[0,100] 默认值：100

- 相关数据类型及接口

[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]

[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

3.62. MI_VENC_ParamMjpegCbr_t

- 描述

定义MJPEG协议编码通道CBR码率控制模式高级参数配置。

- 定义

```
typedef struct MI_VENC_ParamMjpegCbr_s
{
    MI_U32 u32MaxQfactor;

    MI_U32 u32MinQfactor;
```

```
} MI_VENC_ParamMjpegCbr_t;
```

- 成员

成员名称	描述
u32MaxQfactor	帧最大Qfactor。用于控制图像质量 取值范围：(u32MinQfactor,90]。
u32MinQfactor	帧最小Qfactor。用于控制图像质量 取值范围：[1, u32MaxQfactor)。

- 相关数据类型及接口

[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]

[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

3.63. MI_VENC_ParamH264Cbr_t

- 描述

定义H264协议编码通道CBR码率控制模式高级参数配置。

- 定义

```
typedef struct MI_VENC_ParamH264Cbr_s
{
    MI_U32 u32MaxQp;

    MI_U32 u32MinQp;

    MI_S32 s32IPQPDelta;

    MI_U32 u32MaxIQp;

    MI_U32 u32MinIQp;

    MI_U32 u32MaxIPProp;

    MI_U32 u32MaxISize;

    MI_U32 u32MaxPSize;

}MI_VENC_ParamH264Cbr_t;
```

- 成员

成员名称	描述
u32MaxQp	帧最大QP。用于钳位质量。取值范围：(u32MinQp,48]。
u32MinQp	帧最小QP。用于钳位码率波动。取值范围：[12,u32MaxQp)。
s32IPQPDelta	IPQP变化值。取值范围：[-12,12]。
u32MaxIQp	I帧的最大QP。用于控制I帧的最小bits数。取值范围：(u32MinIQp,48]。
u32MinIQp	I帧的最小QP。用于控制I帧的最大bits数。取值范围：[12,u32MaxIQp)。
u32MaxIPProp	最大IP帧码率的比值。取值范围[5,100]。
u32MaxISize	I帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。 取值范围：[0, 0xFFFFFFFF] 默认值：0
u32MaxPSize	P帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。 取值范围：[0, 0xFFFFFFFF] 默认值：0

- 相关数据类型及接口
 - [MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]
 - [MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

3.64. MI_VENC_ParamH265Vbr_t

- 描述

定义H265协议编码通道VBR码率控制模式高级参数配置。
- 定义

```
typedef struct MI_VENC_ParamH265Vbr_s
{
    MI_S32 s32IPQPDelta;

    MI_S32 s32ChangePos;

    MI_U32 u32MaxIQp;

    MI_U32 u32MinIQP;

    MI_U32 u32MaxIPProp;

    MI_U32 u32MaxISize;

    MI_U32 u32MaxPSize;
}MI_VENC_ParamH265Vbr_t;
```

- 成员

成员名称	描述
s32IPQPDelta	IPQP变化值。取值范围：[-12,12]。
s32ChangePos	VBR开始调整Qp时的码率相对于最大码率的比例。取值范围：[5,100]。
u32MaxIQp	I帧的最大QP。用于控制I帧的最小bits数。取值范围：(MinQp,MaxQp]。
u32MinIQp	I帧的最小QP。用于控制I帧的最大bits数。取值范围：[MinQp,MaxQp)。
u32MaxIPProp	最大IP帧码率的比值。取值范围[5,100]。
u32MaxISize	I帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。 取值范围：[0, 0xFFFFFFFF] 默认值：0
u32MaxPSize	P帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。 取值范围：[0, 0xFFFFFFFF] 默认值：0

- 相关数据类型及接口

[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]

[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

.....

3.65. MI_VENC_ParamH265Avbr_t

- 描述

定义H265协议编码通道AVBR码率控制模式高级参数配置。

- 定义

```
typedef struct MI_VENC_ParamH265Avbr_s
{
    MI_S32 s32IPQPDelta;

    MI_S32 s32ChangePos;

    MI_U32 u32MinIQp;

    MI_U32 u32MaxIPProp;

    MI_U32 u32MaxIQp;

    MI_U32 u32MaxISize;

    MI_U32 u32MaxPSize;

    MI_U32 u32MinStillPercent;

    MI_U32 u32MaxStillQp;

    MI_U32 u32MotionSensitivity;
} MI_VENC_ParamH265Avbr_t;
```

- 成员

成员名称	描述
s32IPQPDelta	IPQP变化值。取值范围：[-12,12] 默认值：0
s32ChangePos	AVBR开始调整Qp时的码率相对于最大码率的比例。取值范围：[5,100] 默认值：80
u32MinIQp	I帧的最小QP。用于控制I帧的最大bits数。 取值范围：[MinQp,MaxQp] 默认值：12
u32MaxIPProp	最大IP帧码率的比值。取值范围[5,100]
u32MaxIQp	I帧的最大QP。用于控制I帧的最小bits数。取值范围：(MinQp,MaxQp] 默认值：48
u32MaxISize	I帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制 取值范围：[0,0xFFFFFFFF] 默认值：0
u32MaxPSize	P帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制 取值范围：[0,0xFFFFFFFF] 默认值：0
u32MinStillPercent	静止状态下最小码率相对于调节阈值码率的百分比。若此变量设置为100，AVBR将不会在判别为静止时主动调低目标码率，AVBR的表现将和VBR一致 取值范围：[5,100] 默认值：25
u32MaxStillQp	静止状态最大I帧QP 取值范围：[MinIQp,MaxIQp] 暂不支持
u32MotionSensitivity	根据画面运动程度调整码率的灵敏度 取值范围：[0,100] 默认值：100

- 相关数据类型及接口

[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]

[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

3.66. MI_VENC_ParamH265Cbr_t

- 描述

定义H265协议编码通道CBR新版码率控制模式高级参数配置。

- 定义

```
typedef struct MI_VENC_ParamH265Cbr_s
{
    MI_U32  u32MaxQp;

    MI_U32  u32MinQp;

    MI_S32  s32IPQPDelta;

    MI_U32  u32MaxIQp;
```

```
MI_U32 u32MinIQp;

MI_U32 u32MaxIPProp;

MI_U32 u32MaxISize;

MI_U32 u32MaxPSize;

}MI_VENC_ParamH265Cbr_t;
```

- 成员

成员名称	描述
u32MaxQp	帧最大QP。用于钳位质量。取值范围：(u32MinQp,48]
u32MinQp	帧最小QP。用于钳位码率波动。取值范围：[12,u32MaxQp)
s32IPQPDelta	IPQP变化值。取值范围：[-12,12]
u32MaxIQp	I帧的最大QP。用于控制I帧的最小bits数。取值范围：(u32MinIQp,48]。
u32MinIQp	I帧的最小QP。用于控制I帧的最大bits数。取值范围：[12,u32MaxIQp)。
u32MaxIPProp	最大IP帧码率的比值。取值范围[5,100]
u32MaxISize	I帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。取值范围：[0, 0xFFFFFFFF]默认值：0
u32MaxPSize	P帧的最大size。rate control会尽量让target size不超过max size。若为0，则代表未加限制。取值范围：[0, 0xFFFFFFFF]默认值：0

- 相关数据类型及接口

[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]

[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

3.67. MI_VENC_RcParam_t

- 描述

定义编码通道的码率控制高级参数。

- 定义

```
typedef struct MI_VENC_RcParam_s
{
    MI_U32 u32ThrdI[RC_TEXTURE_THR_SIZE];

    MI_U32 u32ThrdP[RC_TEXTURE_THR_SIZE];

    MI_U32 u32RowQpDelta;

    union
    {
        MI_VENC_ParamH264Cbr_t stParamH264Cbr;

        MI_VENC_ParamH264Vbr_t stParamH264VBR;
    }
};
```

```
MI_VENC_ParamH264Avbr_t stParamH264Avbr;

MI_VENC_ParamMjpegCbr_t stParamMjpegCbr;

MI_VENC_ParamH265Cbr_t stParamH265Cbr;

MI_VENC_ParamH265Vbr_t stParamH265Vbr;

MI_VENC_ParamH265Avbr_t stParamH265Avbr;

};

MI_VOID*pRcParam;

}MI_VENC_RcParam_t;
```

- 成员

成员名称	描述
u32ThrdI	I帧宏块级码率控制的mad门限。取值范围：[0,255]。暂不支持。
u32ThrdP	P帧宏块级码率控制的mad门限。取值范围：[0,255]。暂不支持。
u32RowQpDelta	行级码率控制。取值范围：[0,10]。暂不支持。
stParamH264Cbr	H.264通道CBR（ConstantBitRate）码率控制模式高级参数。
stParamH264Vbr	H.264通道VBR（VariableBitRate）码率控制模式高级参数。
stParamH264Avbr	H.264通道AVBR（AdaptiveVariableBitRate）码率控制模式高级参数。
stParamH265Cbr	H.265通道CBR（ConstantBitRate）码率控制模式高级参数。
stParamH265Vbr	H.265通道VBR（VariableBitRate）码率控制模式高级参数。
stParamH265Avbr	H.265通道AVBR（AdaptiveVariableBitRate）码率控制模式高级参数。
pRcParam	保留参数，目前没有使用到。

- 相关数据类型及接口

[MI_VENC_SetRcParam](#) [#257-mi_venc_setrcparam]

[MI_VENC_GetRcParam](#) [#258-mi_venc_getrcparam]

3.68. MI_VENC_CropCfg_t

- 描述

定义通道截取（Crop）参数。

- 定义

```
typedef struct MI_VENC_CropCfg_s
{
    MI_BOOL bEnable; /* Crop region enable */

    MI_VENC_Rect_t stRect; /* Crop region, 注意: s32X must be multi of 16 */
} MI_VENC_CropCfg_t;
```

- 成员

成员名称	描述
bEnable	是否进行裁剪 TRUE：使能裁剪 FALSE：不使能裁剪
stRect	裁剪的区域 stRect.u32Left：H.264必须16像素对齐，H.265必须32像素对齐。 stRect.u32Top：H.264/H.265皆必须2像素对齐。 stRect.u32Width，s32Rect.u32Height：H.264必须16像素对齐，H.265必须32像素对齐。

- 相关数据类型及接口

[MI_VENC_SetCrop](#) [#261-mi_venc_setcrop]

[MI_VENC_GetCrop](#) [#262-mi_venc_getcrop]

3.69. MI_VENC_RecvPicParam_t

- 描述

接收指定帧数图像编码。

- 定义

```
typedef struct MI_VENC_RecvPicParam_s
{
    MI_S32 s32RecvPicNum;
}MI_VENC_RecvPicParam_t;
```

- 成员

成员名称	描述
s32RecvPicNum	编码通道连续接收并编码的帧数

- 相关数据类型及接口

[MI_VENC_StartRecvPicEx](#) [#29-mi_venc_startrecvpicex]

3.70. MI_VENC_H264eIdrPicIdMode_e

- 描述

设置IDR帧或I帧的idr_pic_id的模式。

- 定义

```
typedef enum
{
    E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR,
}MI_VENC_H264eIdrPicIdMode_e;
```

- 成员

成员名称	描述
E_MI_VENC_H264E_IDR_PIC_ID_MODE_USR	用户模式；即idr_pic_id由用户来设置。

- 相关数据类型及接口

[MI_VENC_SetH264IdrPicId](#) [#221-mi_venc_seth264idrpicid]

3.71. MI_VENC_H264IdrPicIdCfg_t

- 描述

IDR帧或I帧的idr_pic_id参数。

- 定义

```
typedef struct MI_VENC_H264IdrPicIdCfg_s
{
    MI_VENC_H264eIdrPicIdMode_e eH264eIdrPicIdMode;

    MI_U32 u32H264eIdrPicId;
}MI_VENC_H264IdrPicIdCfg_t;
```

- 成员

成员名称	描述
enH264eIdrPicIdMode	设置idr_pic_id的模式
u32H264eIdrPicId	idr_pic_id的值。取值范围为：[0,65535]

- 相关数据类型及接口

[MI_VENC_SetH264IdrPicId](#) [#221-mi_venc_seth264idrpicid]

3.72. MI_VENC_FrameLostMode_e

- 描述

瞬时码率超过阈值时的丢帧模式。

- 定义

```
typedef enum
{
    E_MI_VENC_FRMLOST_NORMAL,

    E_MI_VENC_FRMLOST_PSKIP,

    E_MI_VENC_FRMLOST_MAX,
}MI_VENC_FrameLostMode_e;
```

- 成员

成员名称	描述
E_MI_VENC_FRMLOST_NORMAL	瞬时码率超过阈值时正常丢帧
E_MI_VENC_FRMLOST_PSKIP	瞬时码率超过阈值时编码pskip帧

- 相关数据类型及接口
[MI_VENC_SetFrameLostStrategy](#) [#263-mi_venc_setframeLoststrategy]
[MI_VENC_GetFrameLostStrategy](#) [#264-mi_venc_getframeLoststrategy]

3.73. MI_VENC_ParamFrameLost_t

- 描述
瞬时码率超过阈值时的丢帧策略参数。
- 定义

```
typedef struct MI_VENC_ParamFrameLost_s
{
    MI_BOOL bFrmLostOpen;

    MI_U32 u32FrmLostBpsThr;

    MI_VENC_FrameLostMode_e eFrmLostMode;

    MI_U32 u32EncFrmGaps;
} MI_VENC_ParamFrameLost_t;
```

- 成员

成员名称	描述
bFrmLostOpen	瞬时码率超过阈值时丢帧开关
u32FrmLostBpsThr	丢帧阈值。（单位为bit/s）
enFrmLostMode	瞬时码率超过阈值时丢帧模式
u32EncFrmGaps	丢帧间隔，默认为0。取值范围：[0,65535]

- 相关数据类型及接口
[MI_VENC_SetFrameLostStrategy](#) [#263-mi_venc_setframeLoststrategy]

3.74. MI_VENC_SuperFrameCfg_t

- 描述
超大帧处理策略参数。
- 定义

```
typedef struct MI_VENC_SuperFrameCfg_s
{
```

```
MI_VENC_SuperFrmMode_e eSuperFrmMode;

MI_U32 u32SuperIFrmBitsThr;

MI_U32 u32SuperPFrmBitsThr;

MI_U32 u32SuperBFrmBitsThr;

}MI_VENC_SuperFrameCfg_t;
```

- 成员

成员名称	描述
eSuperFrmMode	超大帧处理模式
u32SuperIFrmBitsThr	I帧超大阈值，默认为0。取值范围：大于等于0
u32SuperPFrmBitsThr	P帧超大阈值，默认为0。取值范围：大于等于0
u32SuperBFrmBitsThr	B帧超大阈值，默认为0。取值范围：大于等于0

- 相关数据类型及接口

[MI_VENC_SetSuperFrameCfg](#) [#265-mi_venc_setsuperframecfg]

3.75. MI_VENC_RcPriority_e

- 描述

码率控制优先级枚举。

- 定义

```
typedef enum
{
    E_MI_VENC_RC_PRIORITY_BITRATE_FIRST=1,
    E_MI_VENC_RC_PRIORITY_FRAMEBITS_FIRST,
    E_MI_VENC_RC_PRIORITY_MAX,
}MI_VENC_RcPriority_e;
```

- 成员

成员名称	描述
E_MI_VENC_RC_PRIORITY_BITRATE_FIRST	目标码率优先
E_MI_VENC_RC_PRIORITY_FRAMEBITS_FIRST	超大帧阈值优先

3.76. MI_VENC_ModParam_t

- 描述

编码相关模块参数。

- 定义

```
typedef struct MI_VENC_ModParam_s
{
    MI_VENC_ModType_e eVencModType;

    union
    {
        //MI_VENC_ParamModVenc_t stVencModParam; //not defined yet

        //MI_VENC_ParamModH264e_t stH264eModParam; //not defined yet

        MI_VENC_ParamModH265e_t stH265eModParam;

        MI_VENC_ParamModJpege_t stJpegeModParam;
    };
} MI_VENC_ModParam_t;
```

- 成员

成员名称	描述
enVencModType	设置或者获取模块参数的类型。
stH265eModParam/stJpegeModParam	mi_h265.ko/ mi_jpege.ko模块参数结构。

3.77. MI_VENC_ModType_e

- 描述

编码相关模块参数类型。

- 定义

```
typedef enum
{
    E_MI_VENC_MODTYPE_VENC=1,

    E_MI_VENC_MODTYPE_H264E,

    E_MI_VENC_MODTYPE_H265E,

    E_MI_VENC_MODTYPE_JPEGE,

    E_MI_VENC_MODTYPE_MAX

}MI_VENC_ModType_e;
```

- 成员

成员名称	描述
E_MI_VENC_MODTYPE_VENC	mi_venc.ko模块参数类型
E_MI_VENC_MODTYPE_H264E	mi_h264e.ko模块参数类型
E_MI_VENC_MODTYPE_H265E	mi_h265e.ko模块参数类型
E_MI_VENC_MODTYPE_JPEGE	mi_jpege.ko模块参数类型

- 相关数据类型及接口
[MI_VENC_ModParam_t](#) [#376-mi_venc_modparam_t]

3.78. MI_VENC_ParamModH265e_t

- 描述
mi_h265.ko模块参数。
- 定义

```
typedef struct MI_VENC_ParamModH265e_s
{
    MI_U32  u32OneStreamBuffer;

    MI_U32  u32H265eMiniBufMode;
}MI_VENC_ParamModH265e_t;
```

- 成员

成员名称	描述
u32OneStreamBuffer	mi_h265.ko模块中u32OneStreamBuffer参数。 0：多包模式 1：单包模式
u32H265eMiniBufMode	mi_h265.ko模块中u32H265eMiniBufMode参数。 0：码流buffer根据分辨率分配 1：码流buffer下限为32k，用户保证合理

- 相关数据类型及接口
[MI_VENC_ModParam_t](#) [#376-mi_venc_modparam_t]

3.79. MI_VENC_ParamModJpege_t

- 描述
mi_jpeg.ko模块中参数。
- 定义

```
typedef struct MI_VENC_ParamModJpege_s
{
```

```
MI_U32 u32OneStreamBuffer;

MI_U32 u32JpegeMiniBufMode;

}MI_VENC_ParamModJpege_t;
```

- 成员

成员名称	描述
u32OneStreamBuffer	mi_jpeg.ko模块中u32OneStreamBuffer参数。 0：多包模式 1：单包模式
u32JpegeMiniBufMode	mi_jpeg.ko模块中u32JpegeMiniBufMode参数。 0：码流buffer根据分辨率分配 1：码流buffer下限为32k，用户保证合理

- 相关数据类型及接口

[MI_VENC_ModParam_t](#) [#376-mi_venc_modparam_t]

3.80. MI_VENC_AdvCustRcAttr_t

- 描述
自定义的高级码控相关功能的开关参数。
- 定义

```
typedef struct MI_VENC_AdvCustRcAttr_s
{
    MI_BOOL bEnableQMap;

    MI_BOOL bAbsQP;

    MI_BOOL bEnableModeMap;

    MI_BOOL bEnabelHistoStaticInfo;

}MI_VENC_AdvCustRcAttr_t;
```

- 成员

成员名称	描述
bEnableQPMaP	使能QPMaP功能的开关 TRUE：开 FALSE：关
bAbsQP	配置QPMaP是否使用绝对QP值 TRUE：是 FALSE：否，使用相对QP值
bEnableModeMap	使能ModeMap功能的开关 TRUE：开 FALSE：关
bEnabelHistoStaticInfo	使能获取帧编码后相关统计数据的开关 TRUE：开 FALSE：关

- 相关数据类型及接口
- [MI_VENC_SetAdvCustRcAttr](#) [#273-mi_venc_setadvcustrcattr]

3.81. MI_VENC_FrameHistoStaticInfo_t

- 描述
- 图像帧编码的相关配置属性信息。
- 定义

```
typedef struct MI_VENC_FrameHistoStaticInfo_s
{
    MI_U8  u8PicSkip;

    MI_U16 u16PicType;

    MI_U32 u32PicPoc;

    MI_U32 u32PicSliNum;

    MI_U32 u32PicNumIntra;

    MI_U32 u32PicNumMerge;

    MI_U32 u32PicNumSkip;

    MI_U32 u32PicAvgCtuQp;

    MI_U32 u32PicByte;

    MI_U32 u32GopPicIdx;

    MI_U32 u32PicNum;

    MI_U32 u32PicDistLow;

    MI_U32 u32PicDistHigh;
} MI_VENC_FrameHistoStaticInfo_t;
```

- 成员

成员名称	描述
u8PicSkip	图像帧skip flag
u16PicType	编码帧的类型： 0：I 1：P 2：B
u32PicPoc	当前编码帧的图片顺序计数（POC）
u32PicSliNum	片段（Slice Segment）总数
u32PicNumIntra	当前编码帧的Intra块的数量（8x8 unit）
u32PicNumMerge	当前编码帧的Merge块的数量（8x8 unit）
u32PicNumSkip	当前编码帧的Skip块的数量（8x8 unit）
u32PicAvgCtuQp	当前编码帧所有CTU的平均QP值
u32PicByte	当前编码图像帧的大小（bytes）
u32GopPicIdx	GOP中该图像帧的index
u32PicNum	当前已经编码的图片帧数
u32PicDistLow	SSD的低32bit
u32PicDistHigh	SSD的高32bit

- 相关数据类型及接口

MI_VENC_GetLastHistoStaticInfo

3.82. MI_VENC_InputSourceConfig_t

- 描述
输入配置参数。
- 定义

```
typedef struct MI_VENC_InputSourceConfig_s
{
    MI_VENC_InputSrcBufferMode_e eInputSrcBufferMode;
} MI_VENC_InputSourceConfig_t;
```

- 成员

成员名称	描述
eInputSrcBufferMode	输入模式

- 相关数据类型及接口

[MI_VENC_InputSrcBufferMode_e](#) [#383-mi_venc_inputsrcbuffermode_e]

3.83. MI_VENC_InputSrcBufferMode_e

- 描述
定义H.264和H.265输入buffer模式枚举。
- 定义

```
typedef enum
{
    E_MI_VENC_INPUT_MODE_NORMAL_FRMBASE = 0, /*Handshake with input by about 3 buffers in frame mode*/

    E_MI_VENC_INPUT_MODE_RING_ONE_FRM, /*Handshake with input by one buffer in ring mode*/

    E_MI_VENC_INPUT_MODE_RING_HALF_FRM, /*Handshake with input by half buffer in ring mode*/

    E_MI_VENC_INPUT_MODE_HW_AUTO_SYNC, /*Handshake with input by hw auto sync in ring mode*/

    E_MI_VENC_INPUT_MODE_MAX
} MI_VENC_InputSrcBufferMode_e;
```

- 成员

成员名称	描述
E_MI_VENC_INPUT_MODE_NORMAL_FRMBASE	默认与前级通过三张左右frame交互
E_MI_VENC_INPUT_MODE_RING_ONE_FRM	与前级通过一张frame以ring的形式交互
E_MI_VENC_INPUT_MODE_RING_HALF_FRM	与前级通过半张frame以ring的形式交互
E_MI_VENC_INPUT_MODE_HW_AUTO_SYNC	使用此标志来区分是ring mode还是hw auto sync mode

- 相关数据类型及接口
[MI_VENC_InputSourceConfig_t](#) [#382-mi_venc_inputsourceconfig_t]
[MI_VENC_SetInputSourceConfig](#) [#274-mi_venc_setinputsourceconfig]

3.84. VENC_MAX_SAD_RANGE_NUM

- 描述
定义智能检测相关之统计SAD值落入区间范围的最大区间个数。
- 定义

```
#define VENC_MAX_SAD_RANGE_NUM 16
```

3.85. MI_VENC_SmartDetType_e

- 描述

定义智能侦测类型。

- 定义

```
typedef enum
{
    E_MI_VENC_MD_DET=1,
    E_MI_VENC_ROI_DET,
    E_MI_VENC_SMART_DET_MAX,
} MI_VENC_SmartDetType_e;
```

- 成员

成员名称	描述
E_MI_VENC_MD_DET	运动侦测类型
E_MI_VENC_ROI_DET	ROI侦测类型

3.86. MI_VENC_MdInfo_t

- 描述

定义运动侦测相关统计信息。

- 定义

```
typedef struct MI_VENC_MdInfo_s
{
    MI_U16 u16SadRangeRatio[VENC_MAX_SAD_RANGE_NUM];
} MI_VENC_MdInfo_t;
```

- 成员

成员名称	描述
u16SadRangeRatio	每一帧中所有MB之SAD落入每个区间的占比

- 注意

SAD值的取值范围为[0, 255]，所有SAD值被分为16个区间：[0,10) · [10,15) · [15,20) · [20,25) · [25,30) · [30,40) · [40,50) · [50,60) · [60,70) · [70,80) · [80,90) · [90,100) · [100,120) · [120,140) · [140,160) · [160, 255]。此处统计一帧中所有MB(8*8)落入以上区间的占总的MB数量的比例。

- 相关数据类型及接口

VENC_MAX_SAD_RANGE_NUM [384-venc_max_sad_range_num]

3.87. MI_VENC_SmartDetInfo_t

- 描述

定义智能侦测算法所需的统计信息。

- 定义

```
typedef struct MI_VENC_SmartDetInfo_s
{
    MI_VENC_SmartDetType_e eSmartDetType;

    union
    {
        MI_VENC_MdInfo_t stMdInfo;

        MI_BOOL bRoiExist;
    };

    MI_U8 u8ProtectFrmNum;
} MI_VENC_SmartDetInfo_t;
```

- 成员

成员名称	描述
eSmartDetType	智能侦测类型
stMdInfo	运动侦测统计信息
bRoiExist	ROI区域是否存在信息 MI_TRUE: 当前帧有ROI存在； MI_FALSE: 当前帧无ROI存在；
u8ProtectFrmNum	保护帧数

3.88. MI_VENC_IntraRefresh_t

- 描述

支持P帧刷Islice的控制参数。

- 定义

```
typedef struct MI_VENC_IntraRefresh_s
{
    MI_BOOL bEnable;

    MI_U32 u32RefreshLineNum;

    MI_U32 u32ReqIQp;
}MI_VENC_IntraRefresh_t;
```

- 成员

成员名称	描述
bEnable	Enable venc Intra Refresh模式
u32RefreshLineNum	每个gop序列里面需要刷新的Islice总数
u32ReqIQp	是不是要强制编码I帧

- 注意
只支持H265,H264.
- 相关数据类型及接口
[MI_VENC_SetIntraRefresh](#) [#276-mi_venc_setintrarefresh]

3.89. MI_VENC_InitParam_t

- 说明
Venc设备初始化参数。
- 定义

```
typedef struct MI_VENC_InitParam_s
{
    MI_U32 u32MaxWidth;

    MI_U32 u32MaxHeight;
}MI_VENC_InitParam_t;
```

- 成员

成员名称	描述
u32MaxWidth	设备使用的最大分辨率的宽度
u32MaxHeight	设备使用的最大分辨率的高度

- 注意事项
Pudding 、Ispahan和Ikayaki系列根据实际需要设置最大宽高。
- 相关数据类型及接口
[MI_VENC_InitDev](#) [#278-mi_venc_initdev]

4. 错误码

视频编码API错误码如表所示。

Error Code	Macro Definition	Description
0xa0022000	MI_VENC_OK	success
0xa0022001	MI_ERR_VENC_INVALID_DEVID	invalid device ID
0xa0022002	MI_ERR_VENC_INVALID_CHNID	invalid channel ID
0xa0022003	MI_ERR_VENC_ILLEGAL_PARAM	at lease one 参数 is illegal
0xa0022004	MI_ERR_VENC_EXIST	channel exists
0xa0022005	MI_ERR_VENC_UNEXIST	channel unexist
0xa0022006	MI_ERR_VENC_NULL_PTR	using a NULL point
0xa0022007	MI_ERR_VENC_NOT_CONFIG	try to enable or initialize device or channel, before configuring attribute
0xa0022008	MI_ERR_VENC_NOT_SUPPORT	operation is not supported by NOW
0xa0022009	MI_ERR_VENC_NOT_PERM	operation is not permitted
0xa002200C	MI_ERR_VENC_NOMEM	failure caused by malloc memory
0xa002200D	MI_ERR_VENC_NOBUF	failure caused by malloc buffer
0xa002200E	MI_ERR_VENC_BUF_EMPTY	no data in buffer
0xa002200F	MI_ERR_VENC_BUF_FULL	no buffer for new data
0xa0022010	MI_ERR_VENC_NOTREADY	System is not ready
0xa0022011	MI_ERR_VENC_BADADDR	bad address
0xa0022012	MI_ERR_VENC_BUSY	resource is busy
0xa0022013	MI_ERR_VENC_CHN_NOT_STARTED	channel not start
0xa0022014	MI_ERR_VENC_CHN_NOT_STOPPED	channel not stop
0xa002201F	MI_ERR_VENC_UNDEFINED	unexpected error

5. PROCFS介绍

5.1. cat

- 调试信息

```
#cat /proc/mi_modules/mi_venc/mi_vencN
```

N表示不同的device · 不同chip的N取值如下：

表4-1 不同chip的N取值

Chip	N取值	
	H.264/H.265	JPEG
Pretzel	0	1
Macaron	0	1
Pudding	0	1
Ispahan	0	1
Ikayaki	0	1

```
----- ALL VENC Dev info -----
----- VENC 0 Dev info -----
DevId  IrqNum  IsrTotalCnt  IsrFrmDoneCnt  IsrBufFullCnt  IsrSliceDoneCnt  IsrRingFullCnt  IsrTimeoutCnt  IsrOtherCnt
0  26/ 0  1049  0  0  0  0  0  0
DevId  MaxTaskCnt  WorkingTaskCnt  UtilHw  UtilMi  PeakHw  PeakMi  FPS  MbRate  %
0  0  1  0  0  0  0  29.99  972000  146
DevId  SupportRing  SupportImi  NeedPadding  CmdqHandle  CmdqBufSize  ResetTime  MaxW  MaxH  AlignMaxW  AlignMaxH
0  0  0  0  (null)  0  0  3840  2160  3840  2176

----- VENC 0 CHN info -----
ChnId  DevId  bStart  RefMemPA  RefMemSize  ALMemPA  ALMemVA  ALMemSize
0  0  1  0  0  0  (null)  0
ChnId  State  EnPred  base  enhance  MaxStreamCnt  FrameIdx  Gardient  Fps_1s  kbps  Fps10s  kbps
0  0  0  1  0  20  1  929  30.18  8456  29.99  7917
ChnId  RingBufStartLine  RingBufHeight  QueryCnt  GetStreamCnt  RlsStreamCnt  PollReadyCnt  PollFailCnt
0  0  0  1049  1049  1049  1049  1049
ChnId  BufWidth  BufHeight  ContentWidth  ContentHeight  AlignMaxW  AlignMaxH
0  3840  2176  3840  2176  3840  2176
----- InputPort of dev: 0 -----
ChnId  Width  Height  SrcFrmRate  MaxW  MaxH  FrameCnt  DropCnt  BlockCnt
0  3840  2160  30/1  3840  2160  1049  1  0
----- OutputPort of dev: 0 -----
ChnId  CODEC  Profile  BufSize  MinAllocSize  RefNum  bByFrame  FrameCnt  DropCnt  ReEncCnt  RingUnreadCnt  RingTotalCnt  UsrLockedCnt
0  H264  2  4147200  33528  0  1  1049  0  0  0  0  0
ChnId  RateCtl  GOP  MaxBitrate  IPQPDelta  MaxOp  MinOp  MaxIQp  MinIQp  QpMap  AbsQp  ModeMap
0  CBR  60  8000000  -2  48  15  48  15  0  0  0
```

- 调试信息分析
记录当前某VENC device的使用状况，以及device属性、layer属性、inputport属性，可以动态地获取到这些信息，方便调试和测试。
- 参数说明

参数		描述
Dev info	DevId	设备号，参考表4-1 [#N]
	IrqNum	中断号
	IsrTotalCnt	各种类型的irq总数
	IsrFrmDoneCnt	frame done类型的irq总数
	IsrBufFullCnt	buffer full类型的irq总数
	IsrSliceDoneCnt	slice done的isr 总数
	IsrRingFullCnt	ring buffer full类型的irq总数
	IsrReEncCnt	重编码类型的irq总数
	IsrTimeoutCnt	超时的isr总数
	IsrOtherCnt	other类型的irq总数
	MaxTaskCnt	支持同时处理的最多task个数
	WorkingTaskCnt	正在处理的task个数

working taskCnt		正在处理的task个数
	SupportRing	是否支持HW RING
	SupportImi	是否支持HW REALTIME
	NeedPadding	是否需要MI SYS clear padding
	CmdqHandle	cmdq handle value
	CmdqBufSize	cmdq buffer size
	ResetTime	设备重启所花时间
	MaxW	设备使用最大分辨率的宽度
	MaxH	设备使用最大分辨率的高度
	AlignMaxW	对齐的最大宽度
	AlignMaxH	对齐的最大高度
CHN info	ChnId	Channel ID 通道号
	IsrBufFullCnt	buffer full类型的irq总数
	IsrRingFullCnt	ring buffer full类型的irq总数
	IsrReEncCnt	重编码类型的irq总数
	bStart	是否已开始接收数据
	RefMemPA	底层driver要求的硬件物理位址
	RefMemSize	底层driver要求的硬件内存大小
	AlMemPA	底层driver要求的CPU物理位址
	AlMemVA	底层driver要求的CPU虚拟位址
	AlMemSize	底层driver要求的CPU内存大小
	State	底层处理状态: typedef enum { E_MI_VENC_BUFSTATE_IDLE = 0, E_MI_VENC_BUFSTATE_ENCODE, E_MI_VENC_BUFSTATE_WAIT_CMDQ, E_MI_VENC_BUFSTATE_CHECK, E_MI_VENC_BUFSTATE_DROP, E_MI_VENC_BUFSTATE_DROP_FORCE, E_MI_VENC_BUFSTATE_RECOVER, E_MI_VENC_BUFSTATE_BUF_FULL, E_MI_VENC_BUFSTATE_MAX } MI_VENC_BufferState_e;
	EnPred	代表base层的帧是否被base层其它帧用作参考
	base	base层的周期，取值范围：(0, +∞)
	enhance	enhance层的周期，取值范围：[0, 255]
	MaxStreamCnt	最大输出缓存张数

	Fps_1s	统计1s内的输出帧率
	kbps1s	统计1s内的输出码率
	Fps10s	统计10s内的输出帧率
	kbps10s	统计10s内的输出码率
	QueryCnt	APP调用MI_VENC_Query的次数
	GetStreamCnt	APP调用MI_VENC_GetStream的次数
	RlsStreamCnt	APP调用MI_VENC_ReleaseStream的次数
	PollReadyCnt	APP调用select/poll成功的次数
	PollFailCnt	APP调用select/poll失败的次数
	Frameldx	当前frame的序号： Pudding之前 · 取值为：1 · 2 · 3 · 4 Pudding及之后 · 取值为：1 · 2 · 4
	BufW	input yuv buffer的宽
	BufH	input yuv buffer的高
	Stride	input yuv buffer的stride
	ContentW	input yuv buffer的有效宽
	ContentH	input yuv buffer的有效高
	RingStartLine	ring start line in ring mode
	RingRealTotalHeight	ring real total height in ring mode
	YBufSize	Y buffer size
	BufSize	YUV total buffer size
	Gradient	前端isp传递过来的梯度值
chn pipeline delay in us	Flow	流程点：GetStream:app取流
	Min	Buffer产生到该chn对应Flow所经过的最小延时
	Max	Buffer产生到该chn对应Flow所经过的最大延时
	Avg	Buffer产生到该chn对应Flow所经过的平均延时
	Diff	Buffer产生到该chn对应Flow所经过的当前延时
Inputport info	ChnId	Channel ID 通道号
	Width	画面宽度 (pixels)
	Height	画面高度 (pixels)
	SrcFrmRate	来源每秒帧数 · 通常以此作为rate control的参数
	MaxW	最大画面宽度 (pixels)

	MaxH	最大画面高度 (pixels)
	FrameCnt	已处理的帧数
	DropFrameCnt	已丢弃的帧数
	BlockCnt	已接收到的不过由于当前还在处理上一张input picture从而rewind的input picture总数
Outputport info	ChnId	Channel ID 通道号
	CODEC	使用何种CODEC: H264,H265,JPEG
	Profile	Profile值
	BufSize	由user设定的 output pool total size (bytes)
	MinAllocSize	允许分配的最小 output buffer size
	RefNum	最大参考帧数
	bByFrame	后级是否一次取走一帧？ 0：否，通常代表按slice取数据 1：是
	FrameCnt	已编码并release的帧数
	DropCnt	已编码但丢掉的帧数
	ReEncCnt	重编码的帧数
	RingUnreadCnt	output pool里还未被取走的帧数
	RingTotalCnt	output pool里的帧数
	UsrLockedCnt	user正拿住的帧数
	RateCtl	Rate Control的算法：CBR/VBR/AVBR/FixQP等
	GOP	Group of Picture，I帧间隔
	MaxBitrate	CBR/VBR/AVBR时，最大的目标码率(bits per second)
	IPQPDelta	平均QP值与当前I帧QP的差值
	MaxQp	VBR/AVBR时，允许的最大QP
	MinQp	VBR/AVBR时，允许的最小QP
	MaxIQP	VBR/AVBR时，允许的最大的I帧QP
	MinIQP	VBR/AVBR时，允许的最小的I帧QP
	MaxISize	I帧的最大size
	MaxPSize	P帧的最大size
	ChangePos	VBR/AVBR开始调整Qp时的码率相对于最大码率的比例
	MinStPct	AVBR时，静止状态下目标码率的最小百分比
	MaxStillQp	AVBR时，静止状态最大I帧QP
	MotionSense	AVBR时，根据画面运动程度调整码率的灵敏度

	QpMap	QPMAP模式下允许用户自由决定码率控制的策略
	AbsQp	使用绝对Qp的值
	ModeMap	0 decided byhw ; 1 force skip ; 2 force intra
	Qfactor	JPEG FixQP下，量化系数
	MaxQfactor	JPEG CBR下，最大量化系数
	MinQfactor	JPEG CBR下，最小量化系数

5.2. echo

功能	Dump input buffer
命令	echo dump_in [ChnId] [num] [dump_path] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId]：通道号 [num]：需要dump_in的总张数 [dump_path]：保存文件的路径 [DevID]：设备号，参考表4-1 [#N]
举例	echo dump_in 0 10 /mnt > /proc/mi_modules/mi_venc/mi_venc0

功能	Dump output stream
命令	echo dump_out [ChnId] [num] [dump_path] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId]：通道号 [num]：需要dump_out的总张数 [dump_path]：保存文件的路径 [DevID]：设备号，参考表4-1 [#N]
举例	echo dump_out 0 100 /mnt > /proc/mi_modules/mi_venc/mi_venc0

功能	Show all used device info
命令	echo show all [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[Status]：0-1 0:Show only requested device info 1:Show all used device info. (with created channels) [DevID]：设备号，参考表4-1 [#N]
举例	echo show_all 1 > /proc/mi_modules/mi_venc/mi_venc0

功能	Show output frame count
命令	echo frame_cnt [ChnId] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo frame_cnt 0 > /proc/mi_modules/mi_venc/mi_venc0

功能	Drop/Release the corresponding channel output task
命令	echo drop_out [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号,为a时代表所有通道 [Status] : r : release the corresponding channel output task d : drop the corresponding channel output task [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo drop_out 0 d > /proc/mi_modules/mi_venc/mi_venc0 说明 : drop channel 0 output task echo drop_out 0 r > /proc/mi_modules/mi_venc/mi_venc0 说明 : release channel 0 output task echo drop_out a d > /proc/mi_modules/mi_venc/mi_venc0 说明 : drop the whole channel output task echo drop_out a r > /proc/mi_modules/mi_venc/mi_venc0 说明: release the whole channel output task

功能	Drop/Release the corresponding channel input task
命令	echo drop_in [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号,为a时代表所有通道 [Status] : r : release the corresponding channel input task d : drop the corresponding channel input task [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo drop_in 0 d > /proc/mi_modules/mi_venc/mi_venc0 说明 : drop channel 0 input task echo drop_in 0 r > /proc/mi_modules/mi_venc/mi_venc0 说明 : release channel 0 input task echo drop_in a d > /proc/mi_modules/mi_venc/mi_venc0 说明 : drop the whole channel input task echo drop_in a r > /proc/mi_modules/mi_venc/mi_venc0 说明: release the whole channel input task

功能	Set debug level for super frame flow
命令	echo dbg_supfrm [ChnId] [debug_level] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号
功能	[debug_level] : 0:none 1:warn 2:info Set debug level for super frame flow [DevID] : 设备号 · 参考表4-1 [#N]

举例	echo dbg_supfrm 0 2 > /proc/mi_modules/mi_venc/mi_venc0
----	---

功能	Set debug level for frame lost flow
命令	echo dbg_frmlost [ChnId] [debug_level] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [debug_level] : 0.none 1.warn 2.info [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo dbg_frmlost 0 2 > /proc/mi_modules/mi_venc/mi_venc0

功能	Set debug level for get frame flow
命令	echo dbg_getout [ChnId] [debug_level] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [debug_level] : 0.none 1.warn 2.info [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo dbg_getout 0 2 > /proc/mi_modules/mi_venc/mi_venc0

功能	Dump dump customer qp map
命令	echo dump_map [ChnId] [dump_times] [dump_path] > /proc/mi_modules/mi_venc/mi_venc0
参数说明	[ChnId] : 通道号 [dump_times] : dump 的次数 [dump_path] : 保存文件的路径 [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo dump_map 0 1 /mnt > /proc/mi_modules/mi_venc/mi_venc0

功能	Dump the internal buffer
命令	echo dump_dram [ChnId] [dump_path] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [dump_path] : 保存文件的路径 [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo dump_dram 0 /mnt > /proc/mi_modules/mi_venc/mi_venc0

功能	Enable to run shell script when hang
命令	echo run_sh [Status] [path]> /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[Status] : 1:enable 0:disable [path] : shell script的路径 [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo run_sh 1 /mnt/dump_mhaI_venc_status_hook.sh > /proc/mi_modules/mi_venc/mi_venc0

功能	Enable to recover when hang
命令	echo en_recover [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[Status] : 1:enable 0:disable 2:show [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo en_recover 0 > /proc/mi_modules/mi_venc/mi_venc0

功能	Show output stream info
命令	echo show_out [ChnId] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo show_out 0 > /proc/mi_modules/mi_venc/mi_venc0

功能	Dynamic debug sed roi info or sad info
命令	echo debug_sed [ChnId] [SedType] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [SedType] : 0: ive detector algorithm roi info 1: cnn detector algorithm roi info 2: motion detector algorithm roi info [Status] : ture:enable false:disable [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo debug_sed 0 0 true > /proc/mi_modules/mi_venc/mi_venc0 说明 : show ive detector algorithm roi info echo debug_sed 0 1 true > /proc/mi_modules/mi_venc/mi_venc0 说明 : show cnn detector algorithm roi info echo debug_sed 0 2 true > /proc/mi_modules/mi_venc/mi_venc0 说明 : show motion detector algorithm roi info

功能	Enable to show sram info
命令	echo debug_sram [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [Status] : ture:enable false:disable [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo debug_sram 0 true > /proc/mi_modules/mi_venc/mi_venc0

功能	Enable to show channel function stat info
命令	echo flow_stat [ChnId] [Status] > /proc/mi_modules/mi_venc/mi_venc[DevID]
参数说明	[ChnId] : 通道号 [Status] : 1:enable 0:disable [DevID] : 设备号 · 参考表4-1 [#N]
举例	echo flow_stat 0 1 > /proc/mi_modules/mi_venc/mi_venc0

